БРЯНСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

КУРСОВОЙ ПРОЕКТ

по дисциплине «Технология разработки программного продукта»

Разработка автоматизированной информационной системы приёмной комиссии БГТУ

Выполнила: студентка группы 23-ИСП-1-СПО

Зарубецкая Анастасия Алексеевна

Руководитель:

Сизый Денис Андреевич

Брянск — 2026

Содержание

Введение....3

Термины и сокращения....4

1 Бизнес-требования и анализ предметной области....5

2 Требования к продукту....30

3 Прототипирование и интерфейсы....51

4 Модель жизненного цикла и паттерны проектирования....71

5 Планирование разработки....74

6 Техническое задание....86

7 Архитектура реализованной системы....97

8 Проектирование базы данных....119

9 Реализация программного продукта....126

10 Тестирование....132

11 Сопровождение и внедрение....136

12 Документация и руководства....139

Заключение....162

Список используемых источников....163

Приложение А Листинги программного кода....165

Приложение Б SQL-скрипты создания базы данных....198

Приложение В Структура проекта и результаты проверки....203

Приложение Г План разработки по спринтам....205

Введение

Приёмная кампания требует согласованного учёта заявлений, конкурсных списков и договоров. Раздельное ведение электронных таблиц увеличивает объём ручной работы и риск расхождений. Актуальность проекта определяется необходимостью объединить эти процессы и обеспечить поступающим доступ к актуальной обезличенной информации.

Цель работы — разработать и проверить автоматизированную информационную систему приёмной комиссии БГТУ, включающую Python-ядро, внутренний сайт сотрудников и открытый портал абитуриента с общей базой данных.

Для достижения цели решены задачи анализа предметной области и аналогов, формулирования требований, проектирования интерфейсов, архитектуры и базы данных, реализации импорта Excel, рейтингов и договорного учёта, тестирования и подготовки эксплуатационной документации.

Объект исследования — процессы сопровождения приёмной кампании. Предмет исследования — программные средства учёта заявлений и публикации конкурсной информации. Использованы методы анализа требований, моделирования UML, реляционного проектирования и автоматизированного тестирования.

Практический результат — работающий прототип версии 1 с двумя связанными сайтами. Публичное представление ограничено разрешёнными полями; обработка персональных данных требует соблюдения законодательства [3, 4]. Развитие обученного прогнозирования и промышленное внедрение рассматриваются отдельно от подтверждённых возможностей прототипа.

Термины и сокращения

Таблица Т.1 — Основные термины

Термин	Определение
АИС / ПК	Автоматизированная информационная система / приёмная комиссия
ЕПГУ	Единый портал государственных и муниципальных услуг; источник исходной Excel-выгрузки
Абитуриент	Физическое лицо с устойчивым строковым номером в системе
Заявление	Выбор направления, года, кампании, формы обучения и категории конкурса одним абитуриентом
MVP	Минимально жизнеспособная версия продукта; в отчёте — версия 1
JWT	Подписанный токен доступа сотрудника; срок действия — восемь часов
ORM / Unit of Work	Объектно-реляционное отображение / единая граница транзакции
CR / Drop Rate	Конверсия между этапами / доля потерь на этапе воронки
SP	Story Points — относительная плановая оценка сложности
Оценка положения	Детерминированное сравнение текущего рейтинга с планом мест, без числовой вероятности

1 Бизнес-требования и анализ предметной области

Описания структуры организации и объёмов кампании в этой главе используются как исходная проектная модель, а не как подтверждённая штатная или статистическая справка БГТУ.

1.1 Описание предметной области

1.1.1 Организационная структура приёмной комиссии университета

Приёмная комиссия БГТУ является структурным подразделением, обеспечивающим организацию и проведение приёма абитуриентов на все уровни образования: бакалавриат, специалитет и магистратуру. Ежегодно через приёмную комиссию проходят порядка 10 000 абитуриентов, из которых около 2 000 зачисляются на обучение.

Организационная структура приёмной комиссии имеет матричный вид: Директор ПК одновременно получает указания от трёх руководителей (Ректора, Проректора по учебной работе и Юриста), а Бухгалтерия, Отдел кадров и Преподаватели СПО являются параллельными подразделениями, взаимодействующими с приёмной комиссией по функциональным вопросам (рисунок 1.1).



Рисунок 1.1 — Проектная модель взаимодействия участников приёмной кампании

Описание уровней:

— Ректор БГТУ — высшее руководство вуза. Утверждает правила приёма, подписывает приказы о зачислении, контролирует общие показатели приёмной кампании. Директор ПК подчиняется Ректору по стратегическим вопросам.

— Проректор по учебной работе — курирует работу приёмной комиссии, согласовывает спорные вопросы, утверждает конкурсные списки. Директор ПК подчиняется Проректору по оперативным вопросам.

— Юрист — разрабатывает шаблоны договоров, консультирует по правовым вопросам, проверяет соответствие процедур законодательству. Директор ПК подчиняется Юристу по вопросам юридической корректности документов.

— Директор приёмной комиссии — непосредственный руководитель приёмной кампанией. Организует работу секретарей и практикантов, контролирует корректность данных, отвечает за формирование конкурсных списков.

— Секретари приёмной комиссии — основной персонал, работающий с абитуриентами: приём документов, внесение данных в систему, формирование договоров, обзвон платников.

— Студенты-практиканты — привлекаются в период приёмной кампании для выполнения вспомогательных операций. Имеют тот же уровень доступа к системе, что и секретари.

— Бухгалтерия — проверяет и подписывает договоры на платное обучение, контролирует поступление оплаты, формирует ежедневную Excel-выгрузку об оплатах.

— Отдел кадров — ставит печать на договоры, ведёт учёт зачисленных студентов.

— Преподаватели СПО — вносят результаты внутренних экзаменов для абитуриентов, поступающих после среднего профессионального образования.

1.1.2 Основные бизнес-процессы приёмной комиссии, подлежащие автоматизации

Деятельность приёмной комиссии включает следующие ключевые бизнес-процессы:

— Приём и обработка заявлений абитуриентов. Абитуриенты подают заявления через портал Госуслуг (ЕПГУ). Данные о поданных заявлениях ежедневно выгружаются в формате Excel и передаются в приёмную комиссию для загрузки в систему. Структура Excel-выгрузки включает: номер абитуриента (уникален по РФ), ФИО, паспортные данные, СНИЛС, контактные данные, баллы ЕГЭ, индивидуальные достижения, направления, квота, дата подачи заявления.

— Ведение реестра абитуриентов. Формирование и хранение полных данных о каждом абитуриенте: паспортные данные, СНИЛС, документы об образовании (аттестат или диплом СПО), индивидуальные достижения (медаль, ГТО, олимпиады), льготы, контактные данные, данные родителя (если оплата производится им).

— Распределение абитуриентов по квотам. В зависимости от указанных льгот и выбранного направления абитуриент попадает в одну из пяти категорий:

— особая квота (основания определяются действующими правилами приёма);

— отдельная квота (основания определяются действующими правилами приёма);

— целевое направление;

— основной набор (бюджет);

— платное обучение.

— Формирование конкурсных списков. Расчёт итогового рейтингового балла для каждого абитуриента (сумма баллов ЕГЭ и индивидуальных достижений) и формирование пяти отдельных конкурсных списков по каждому направлению подготовки.

— Учёт результатов внутренних экзаменов для СПО. Абитуриенты, поступающие после среднего профессионального образования, сдают внутренние экзамены. Преподаватели вносят результаты в систему, которые затем учитываются при расчёте рейтинга.

— Заключение и сопровождение договоров на платное обучение. Создание договора по шаблону Word с автозаполнением данными абитуриента, печать, прохождение договора через бухгалтерию, ректора и отдел кадров, подписание абитуриентом и подтверждение оплаты.

— Обзвон абитуриентов, поступающих на платное обучение. Формирование списков для обзвона в последнюю неделю изменения направления (когда абитуриент не проходит на бюджет) и в последнюю неделю до закрытия оплаты.

— Формирование отчётности. Подготовка рейтинговых списков, статистики по квотам, отчётов о переполненных направлениях, ведомостей для бухгалтерии.

— Архивация данных. Хранение информации о приёме в течение 6 лет с возможностью поиска по ФИО, году поступления и номеру договора.

— Выгрузка данных в 1С. После издания приказа о зачислении передача данных о зачисленных студентах в систему 1С для дальнейшего ведения учебного процесса.

1.1.3 Проблемы существующего процесса учёта

В настоящее время учётные процессы приёмной комиссии БГТУ организованы с использованием Google-таблиц и бумажных носителей, что порождает ряд существенных проблем:

— Дублирование и расхождение данных. Информация об абитуриентах вводится вручную в разные Google-таблицы, что приводит к появлению дублей и расхождений. По оценкам сотрудников, до 15–20% записей содержат неточности.

— Отсутствие единой базы данных. Данные о заявлениях, договорах, оплатах и индивидуальных достижениях хранятся в разных файлах, не связанных между собой. Это затрудняет получение целостной картины по конкретному абитуриенту.

— Потеря и путаница с договорами. Договоры на платное обучение распечатываются и передаются по цепочке: приёмная комиссия → бухгалтерия → ректор → отдел кадров → приёмная комиссия → абитуриент. Бумажные документы раскладываются по направлениям, что затрудняет поиск. Нередки случаи утери договоров и задержек их подписания.

— Ручной расчёт рейтинга. Формирование конкурсных списков и расчёт итоговых баллов выполняется вручную в Excel. Это занимает много времени и сопровождается ошибками при суммировании баллов ЕГЭ и индивидуальных достижений.

— Отсутствие контроля за статусом договора. Невозможно оперативно определить, на каком этапе находится конкретный договор (в бухгалтерии, у ректора,

в отделе кадров). Секретари вынуждены уточнять эту информацию по телефону или лично.

— Задержки при обзвоне платников. Списки для обзвона формируются вручную, что приводит к задержкам и неполному охвату абитуриентов. Нет возможности приоритизировать звонки по вероятности заключения договора.

— Отсутствие статистики по переполненным направлениям. Информация о конкурсной ситуации собирается разрозненно, что не позволяет оперативно выявлять переполненные направления и принимать управленческие решения.

— Трудоемкость отчётности. Формирование отчётов для руководства и бухгалтерии занимает значительное время из-за необходимости собирать данные из разных источников.

— Отсутствие прозрачности для абитуриентов. Абитуриенты не имеют возможности самостоятельно отслеживать своё место в рейтинге и оценивать шансы на поступление, что приводит к большому количеству звонков в приёмную комиссию.

— Негибкий парсер Excel. Текущий парсер жёстко привязан к позициям столбцов в выгрузке из ЕПГУ. При изменении структуры выгрузки (столбцы меняются местами, добавляются новые или исчезают) парсер перестаёт работать, и требуется ручная обработка.

Перечисленные затруднения связаны не только с объёмом данных, но и с отсутствием единого правила их обновления. Если сотрудники ведут несколько несогласованных копий, даже корректное исправление одной таблицы не гарантирует согласованность остальных. Автоматизация должна закреплять источник каждого изменения и сохранять связь между человеком, заявлением и договором. При этом организационные решения о праве на льготу, порядке работы комиссии и действительности документов остаются за уполномоченными сотрудниками; информационная система обеспечивает учёт и проверяемость их действий.

1.1.4 Обоснование необходимости автоматизации

Автоматизация процессов приёмной комиссии позволит решить перечисленные проблемы и достичь следующих результатов:

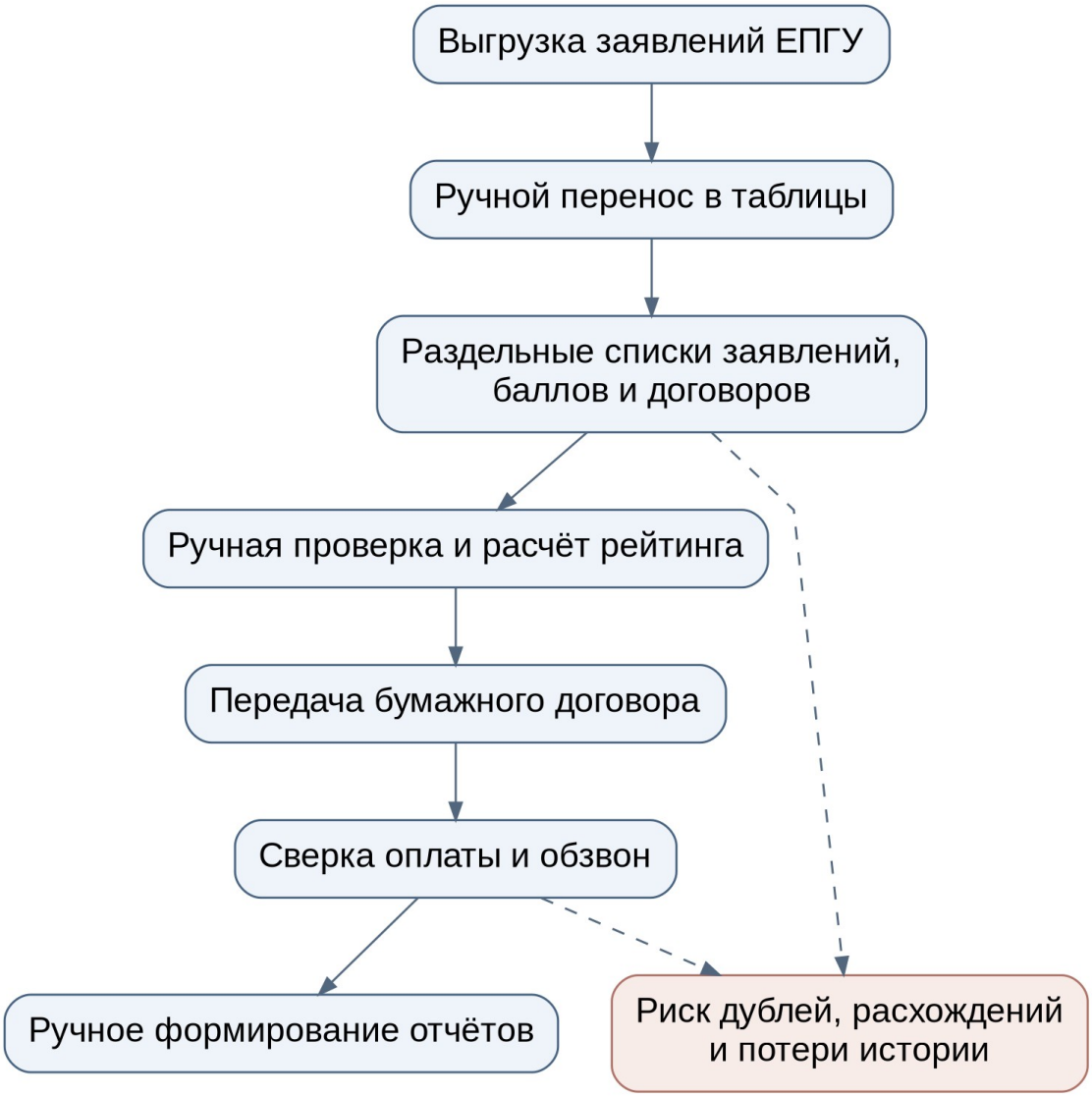


Рисунок 1.2 — Исходный процесс обработки заявлений

— Централизация данных. Создание единой базы данных, в которой хранится вся информация об абитуриентах, договорах, оплатах и статусах.

— Автоматизация расчёта рейтинга. Система автоматически рассчитывает итоговый балл и формирует пять конкурсных списков.

— Контроль жизненного цикла договора. Внедрение системы статусов («ГДЕ?» и «НА КАКОМ ЭТАПЕ?») позволит оперативно отслеживать, где находится каждый договор.

— Оптимизация обзвона. Автоматическое формирование списков для обзвона с приоритезацией по вероятности заключения договора.

— Ускорение отчётности. Автоматическое формирование отчётов и статистики сократит время на подготовку документов.

— Обеспечение сохранности данных. Архивация в течение 6 лет и регулярное резервное копирование.

— Интеграция с внешними системами. Ежедневная загрузка данных из ЕПГУ и выгрузка в 1С.

— Применение ИИ для прогнозирования. Использование модели машинного обучения для прогноза вероятности заключения договора и шансов поступления.

— Гибкий парсер Excel. Семантическое сопоставление столбцов, устойчивое к изменениям структуры выгрузки.

— Соответствие 152-ФЗ. Открытый Сайт 2 с обезличенными данными для абитуриентов.

Таким образом, автоматизация приёмной комиссии БГТУ является необходимой мерой, направленной на повышение эффективности работы, снижение трудозатрат и улучшение качества обслуживания абитуриентов.

1.2 Бизнес-требования

1.2.1 Описание бизнес-проблемы

В настоящее время процессы учёта приёмной комиссии выполняются разрозненно в Google-таблицах и бумажных журналах, что приводит к:

— затрате времени — до 25 часов в неделю на согласование данных между секретарями, бухгалтерией и отделом кадров;

— ошибкам в данных — 15–20% расхождений в конкурсных списках и реестрах;

— финансовым потерям — до 600 000 рублей в год из-за незаключённых договоров на платное обучение и потерянных абитуриентов;

— задержкам в принятии решений — до 3 дней на сбор информации о конкурсной ситуации по направлению.

Автоматизация позволит сократить время на обработку данных на 70%, уменьшить ошибки учёта на 90%, сэкономить до 500 000 рублей в год за счёт увеличения заключённых договоров на платное обучение.

1.2.2 Цели бизнеса

Таблица 1.1 — Цели и целевые критерии проекта

Цель	Описание	Критерии успеха
Повышение прозрачности приёмной кампании	Создание единого информационного пространства для всех участников	Все отделы работают в единой системе, данные доступны в реальном времени
Снижение ошибок учёта	Автоматический расчёт рейтинга и распределения по квотам	Снижение количества ошибок на 90%
Ускорение формирования отчётности	Автоматическая генерация рейтинговых списков и статистики	Время формирования отчётов сокращено на 80%
Повышение конверсии в договоры	ИИ-прогнозирование и приоритизация обзвона платников	Увеличение заключённых договоров на 15%
Оптимизация трудозатрат секретарей	Автоматизация рутины, чат-бот для абитуриентов	Сокращение звонков в ПК на 40%

Критерии в таблице являются целевыми. Снижение ошибок, рост конверсии и сокращение звонков не измерялись в опытной эксплуатации.

1.2.3 Заинтересованные стороны (Stakeholders)

Таблица 1.2 — Заинтересованные стороны

Стейкхолдер	Роль	Интересы	Влияние
Ректор	Заказчик	Контроль показателей приёмной кампании, выполнение контрольных цифр приёма	Высокое
Директор приёмной комиссии	Ключевой пользователь	Прозрачность процессов, оперативное управление, снижение ошибок	Высокое

Стейкхолдер	Роль	Интересы	Влияние
Секретари ПК	Пользователи	Удобство работы, снижение рутины, быстрый доступ к данным	Высокое
Студенты-практиканты	Пользователи	Простой интерфейс, минимум обучения	Низкое
IT-отдел	Поддержка	Надёжность системы, простота администрирования	Низкое
Абитуриенты	Внешние пользователи	Быстрый доступ к информации	Среднее

1.2.4 Бизнес-ценность продукта

Количественная ценность:

— экономия до 600 000 рублей в год за счёт увеличения заключённых договоров на платное обучение;

— сокращение трудозатрат секретарей на 20 часов в неделю;

— снижение количества ошибок в данных на 90%;

— сокращение времени формирования отчётов с 3 дней до 30 минут.

Качественная ценность:

— повышение скорости принятия управленческих решений;

— улучшение качества обслуживания абитуриентов;

— стандартизация процессов приёмной кампании;

— повышение удовлетворённости сотрудников за счёт устранения рутины.

1.2.5 Обоснование экономической эффективности

Разработка ведётся в рамках курсового проекта, внедрение планируется силами IT-отдела БГТУ, что исключает затраты на закупку лицензии. Рассмотрим экономию в период приёмной кампании (июнь–август, 3 месяца).

Таблица 1.3 — Условная модель экономического эффекта (исправленная арифметика)

Статья экономии	Расчёт	Сумма, руб.
Экономия рабочего времени секретарей	5 секретарей × 20 ч/нед × 13 нед × 150 руб/ч	195 000
Экономия на бумаге (договоры, списки, отчёты)	~5 000 листов × 0,3 руб/лист × 3 копии	4 500
Экономия на картриджах для принтеров	4 картриджа × 3 000 руб	12 000
Снижение потерь от незаключённых договоров	+15% к 3 500 платникам × 180 000 руб/год × 10% маржи	9 450 000
Всего в условной модели	211 500 + 9 450 000	9 661 500

Допущения: 5 × 20 × 13 × 150 = 195 000 руб.; 5 000 × 0,3 × 3 = 4 500 руб.; 4 × 3 000 = 12 000 руб. Ресурсная экономия равна 211 500 руб. Условный прирост маржи: 0,15 × 3 500 × 180 000 × 0,10 = 9 450 000 руб. Это сценарный расчёт, а не подтверждённый финансовый результат. В исходном черновике эта строка ошибочно равнялась 94 500 руб.

Ресурсная часть условной экономии составляет 211 500 руб. за кампанию. Потенциальный прирост маржи рассматривается отдельно и не считается достигнутым результатом. Стоимость разработки определяется отдельным расчётом; относительная сложность в SP не переводится напрямую в денежные затраты.

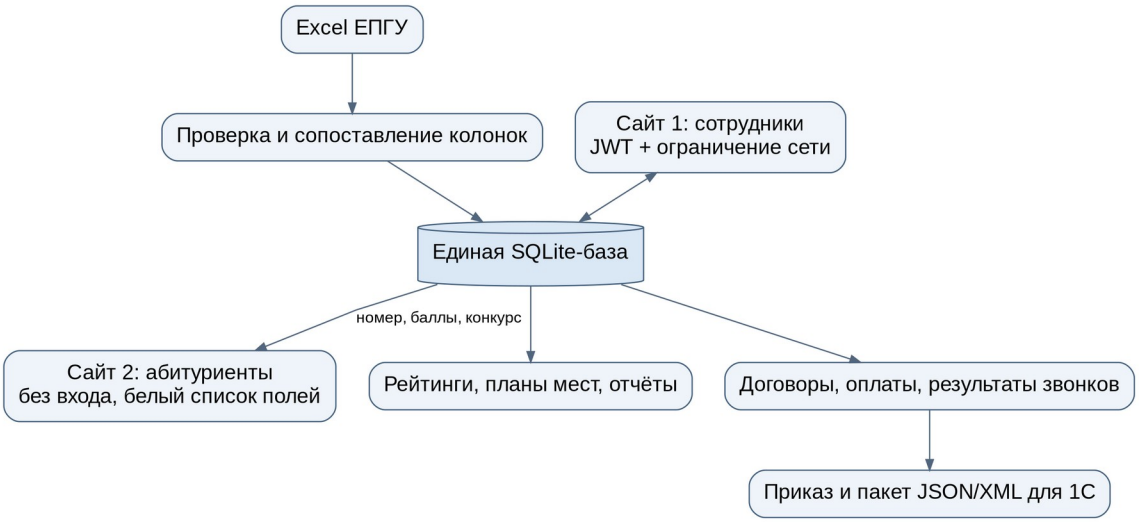


Рисунок 1.3 — Целевой поток данных двух связанных сайтов

1.3 Продуктовые метрики и воронки приёмной кампании

1.3.1 Продукт и его целевое действие

АИС Приёмной комиссии — это цифровой продукт, который автоматизирует путь абитуриента от подачи заявления до зачисления и оплаты договора. Продукт состоит из двух связанных интерфейсов:

— Сайт 1 — рабочий инструмент для секретарей приёмной комиссии;

— Сайт 2 — открытый информационный портал для абитуриентов.

Главное целевое действие — успешное зачисление абитуриента (для бюджетных мест) или успешное заключение и оплата договора на платное обучение.

Целевая аудитория:

— Внутренняя: секретари приёмной комиссии, студенты-практиканты, руководство вуза;

— Внешняя: абитуриенты и их родители (около 10 000 человек ежегодно).

Бизнес-модель: государственный вуз, финансируемый из бюджета. Дополнительный доход формируется за счёт договоров на платное обучение (около 30% от общего числа зачисленных).

1.3.2 Описание трёх продуктовых воронок

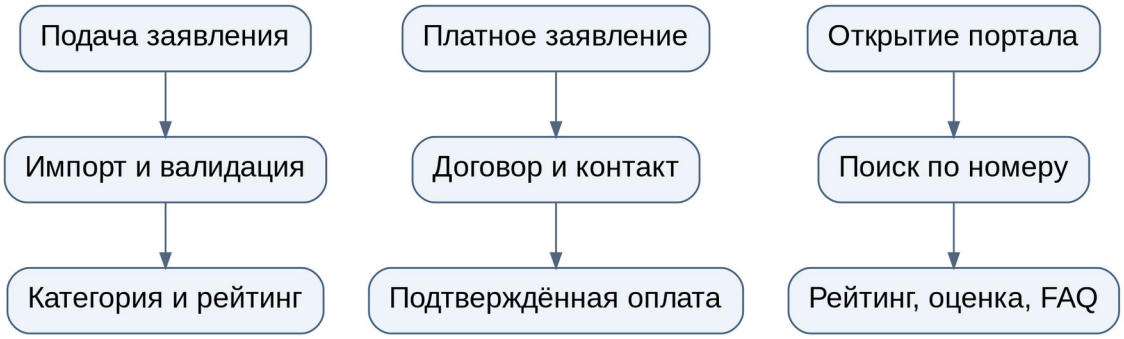


Рисунок 1.4 — Три учебные продуктовые воронки

Для анализа эффективности приёмной кампании выделены три ключевые продуктовые воронки.

Воронка №1. Подача заявления и попадание в конкурсный список

Эта воронка описывает путь абитуриента от подачи заявления через Госуслуги до попадания в конкурсный список по выбранному направлению.

Таблица 1.4 — Учебная воронка обработки заявлений

№	Событие (Event)	Описание	Пользователей	CR	Drop Rate	Lost Users
1	application_submitted	Абитуриент подал заявление через ЕПГУ	10 000	100%	-	-
2	data_loaded	Данные загружены из Excel-выгрузки в систему	9 800	98%	2%	200
3	data_validated	Данные прошли валидацию	9 200	93,9%	6,1%	600
4	quota_assigned	Абитуриент распределён по одной из 5 квот	9 100	98,9%	1,1%	100

Продолжение таблицы 1.4

№	Событие (Event)	Описание	Пользователей	CR	Drop Rate	Lost Users
5	rating_calculated	Рассчитан итоговый балл (ЕГЭ + ИД)	8 900	97,8%	2,2%	200
6	in_competitive_list	Абитуриент попал в конкурсный список	8 500	95,5%	4,5%	400

Числа сохранены из исходной учебной модели. Они не являются статистикой пользователей работающей версии и не совпадают с количеством строк фактической Excel-выгрузки.

Воронка №2. Заключение договора на платное обучение

Эта воронка описывает путь абитуриента, не прошедшего на бюджет, от попадания в список платного обучения до оплаты договора.

Таблица 1.5 — Учебная воронка платного обучения

№	Событие (Event)	Описание	Пользователей	CR	Drop Rate	Lost Users
1	in_paid_list	Абитуриент попал в список на платное обучение	3 500	100%	-	-
2	contract_created	Договор создан по шаблону Word	3 200	91,4%	8,6%	300
3	contract_signed_internal	Договор подписан внутри вуза	2 900	90,6%	9,4%	300
4	contract_issued	Договор выдан абитуриенту	2 600	89,7%	10,3%	300
5	contact_attempted	Секретарь связался с абитуриентом	2 100	80,8%	19,2%	500
6	payment_confirmed	Оплата подтверждена	1 800	85,7%	14,3%	300

Числа сохранены из исходной учебной модели. Они не являются статистикой пользователей работающей версии и не совпадают с количеством строк фактической Excel-выгрузки.

Итоговая конверсия: 51,4% (1 800 из 3 500).

Воронка №3. Использование абитуриентом Сайта 2

Эта воронка описывает путь абитуриента на открытом сайте для просмотра своего места в рейтинге и прогноза шансов.

Таблица 1.6 — Учебная воронка публичного портала

№	Событие (Event)	Описание	Пользователей	CR	Drop Rate	Lost Users
1	site2_opened	Абитуриент открыл Сайт 2	7 500	100%	-	-
2	number_entered	Ввёл номер абитуриента в поиск	6 200	82,7%	17,3%	1 300
3	rating_viewed	Посмотрел своё место в рейтинге	5 400	87,1%	12,9%	800
4	forecast_viewed	Посмотрел прогноз шансов поступления	3 800	70,4%	29,6%	1 600
5	bot_question_asked	Задал вопрос чат-боту	1 900	50,0%	50%	1 900

Числа сохранены из исходной учебной модели. Они не являются статистикой пользователей работающей версии и не совпадают с количеством строк фактической Excel-выгрузки.

Итоговая конверсия: 25,3% (1 900 из 7 500).

1.3.3 Ключевые продуктовые метрики

Таблица 1.7 — Плановые продуктовые метрики

Метрика	Определение	Целевое значение
Final CR (Воронка 2)	Конверсия из списка платников в оплаченные договоры	≥ 60%
Drop Rate (Воронка 2, шаг 5)	Доля потерянных платников на этапе обзвона	≤ 15%
Time to Contract	Среднее время от создания договора до оплаты	≤ 7 дней
Call Success Rate	Доля успешных дозвонов секретарям	≥ 70%

Продолжение таблицы 1.7

Метрика	Определение	Целевое значение
Site 2 MAU	Уникальные посетители Сайта 2 в месяц	≥ 8 000
Bot Resolution Rate	Доля вопросов, решённых чат-ботом без звонка в ПК	≥ 65%
Error Rate (загрузка Excel)	Доля ошибочных записей при загрузке из ЕПГУ	≤ 2%
System Uptime	Доступность системы в рабочее время	≥ 99,5%

1.3.4 Обоснование выбора приоритетной воронки

Для оптимизации в первую очередь выбрана Воронка №2 (Заключение договора на платное обучение) по следующим причинам:

Прямое влияние на доход вуза. Каждый дополнительный заключённый договор приносит вузу в среднем 180 000 рублей за год обучения.

Высокий потенциал улучшения. Текущая конверсия 51,4% может быть повышена до 60% за счёт ИИ-приоритизации обзвона и улучшения UX.

Наличие явных проблем. Этап «дозвон до абитуриента» имеет высокий Drop Rate (19,2%), что указывает на неэффективный обзвон.

Возможность измерения эффекта. Результат легко измеряется в рублях и количестве заключённых договоров.

1.3.5 Сегментный анализ Воронки №2

Для выявления скрытых проблем проведён сегментный анализ по трём разрезам.

Таблица 1.8 — Учебная сегментация по платформе

Сегмент	Вход	Цель	CR	Потеря
Web	1 400	792	56,6%	608

Продолжение таблицы 1.8

Сегмент	Вход	Цель	CR	Потеря
Мобильный	1 400	648	46,3%	752
Итого	2 800	1 440	51,4%	1 360

Числа сохранены из исходной учебной модели. Они не являются статистикой пользователей работающей версии и не совпадают с количеством строк фактической Excel-выгрузки.

Таблица 1.9 — Учебная сегментация по новизне пользователя

Сегмент	Вход	Цель	CR	Потеря
Новый	1 750	700	40,0%	1 050
Повторный	1 050	740	70,5%	310
Итого	2 800	1 440	51,4%	1 360

Числа сохранены из исходной учебной модели. Они не являются статистикой пользователей работающей версии и не совпадают с количеством строк фактической Excel-выгрузки.

Таблица 1.10 — Учебная сегментация по каналу привлечения

Сегмент	Вход	Цель	CR	Потеря
Рекламный	1 400	560	40,0%	840
Органический	1 400	880	62,9%	520
Итого	2 800	1 440	51,4%	1 360

Числа сохранены из исходной учебной модели. Они не являются статистикой пользователей работающей версии и не совпадают с количеством строк фактической Excel-выгрузки.

Выявленные аномалии:

Мобильные пользователи конвертируются значительно хуже (46,3% против 56,6% на Web). Возможная причина: неудобный интерфейс Сайта 2 на смартфонах.

Новые пользователи имеют низкую конверсию (40,0% против 70,5% у повторных). Возможная причина: отсутствие доверия к процессу, необходимость дополнительных объяснений.

Рекламный трафик показывает низкую конверсию (40,0% против 62,9% у органического). Возможная причина: несоответствие ожиданий от рекламы реальному процессу.

1.4 Обзор программ-аналогов

Для обоснования необходимости разработки собственной автоматизированной информационной системы был проведён анализ существующих программных продуктов, применяемых в приёмных комиссиях вузов Российской Федерации. Рассмотрены четыре наиболее распространённых решения: государственная платформа «Госуслуги» (ЕПГУ), коммерческая система «1С:Университет», система «Деканат» (ММИС) и информационная система «Контингент» по указанным пользователем официальным адресам.

1.4.1 Личный кабинет абитуриента на платформе «Госуслуги» (ЕПГУ)

Разработчик: Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации.

Назначение: Единая государственная платформа для подачи заявлений в вузы, колледжи и школы.

Основной функционал:

— Подача заявлений на поступление в вузы через портал Госуслуг;

— Отслеживание статуса поданного заявления в режиме реального времени;

— Просмотр конкурсных списков и рейтингов абитуриентов;

— Подача согласия на зачисление;

— Интеграция с государственными реестрами (ЕГРЮЛ, ФНС, ПФР);

— Автоматическая проверка документов через межведомственное взаимодействие.

— Преимущества:

— Централизованная платформа, охватывающая все вузы страны;

— Бесплатность для абитуриентов и вузов;

— Высокий уровень доверия пользователей благодаря государственному статусу;

— Интеграция с государственными реестрами и базами данных;

— Соответствие требованиям законодательства РФ;

— Мобильное приложение для удобства абитуриентов.

— Недостатки:

— Не автоматизирует внутреннюю работу приёмной комиссии вуза;

— Отсутствует функционал генерации договоров на платное обучение;

— Нет инструментов для обзвона абитуриентов и CRM-функций;

— Отсутствует аналитика и отчётность для руководства вуза;

— Нет ИИ-прогнозирования шансов поступления;

— Не учитывает внутреннюю специфику приёмной кампании конкретного вуза;

— Ограниченные возможности кастомизации под нужды конкретного университета.

Стоимость: бесплатно для вузов и абитуриентов (государственная платформа).

Целевая аудитория: абитуриенты, поступающие в вузы Российской Федерации.

Рисунок 1.5 — Личный кабинет абитуриента на портале «Госуслуги» (ЕПГУ)

1.4.2 «1С:Университет»

Разработчик: фирма «1С» (Россия).

Назначение: Комплексная автоматизация учебного процесса в вузе, включая приём, обучение, выпуск и послевузовское образование.

Основной функционал:

— Управление приёмной кампанией (подача заявлений, формирование списков);

— Ведение контингента студентов и академических групп;

— Учёт успеваемости и формирование ведомостей;

— Планирование учебного процесса (расписание, нагрузка преподавателей);

— Финансовый учёт (стипендии, платное обучение);

— Формирование отчётности для Министерства образования;

— Интеграция с другими продуктами 1С (Бухгалтерия, Зарплата и кадры).

— Преимущества:

— Глубокая интеграция с другими продуктами 1С (бухгалтерия, кадры, зарплата);

— Полное соответствие российским стандартам и законодательству;

— Широкая распространённость — многие вузы уже используют продукты 1С;

— Наличие сертифицированных специалистов по внедрению и поддержке;

— Регулярные обновления в соответствии с изменениями законодательства;

— Масштабируемость — подходит для вузов любого размера.

— Недостатки:

— Очень высокая стоимость внедрения (от 500 000 рублей и выше);

— Сложный и перегруженный интерфейс, требующий длительного обучения;

— Избыточность функционала — система охватывает весь учебный процесс, а не только приёмную кампанию;

— Слабая специализация именно под приёмную кампанию — модуль приёма является лишь частью большой системы;

— Длительные сроки внедрения (6–12 месяцев);

— Зависимость от лицензий и подписки на обновления;

— Сложность доработки под специфические нужды конкретного вуза.

Стоимость: от 500 000 рублей (лицензия) + ежегодное обновление от 50 000 рублей.

Целевая аудитория: крупные и средние вузы, нуждающиеся в комплексной автоматизации всего учебного процесса.

Рисунок 1.6 — Интерфейс программы «1С:Университет»

1.4.3 «Деканат» (ММИС)

В обзоре рассматривается система «Деканат», представленная на официальном сайте ММИС: https://dekanat.mmis.ru/ [23]. Сведения о стоимости и возможностях ниже являются исходными авторскими оценками, а не подтверждённым коммерческим предложением поставщика.

Назначение: Специализированная система управления учебным процессом в вузе, охватывающая все этапы от приёма до выпуска.

Основной функционал:

— Приём и зачисление абитуриентов;

— Формирование учебных групп и потоков;

— Ведение успеваемости и рейтингов студентов;

— Планирование учебного процесса и нагрузки;

— Формирование приказов и отчётности;

— Интеграция с государственными информационными системами;

— Электронный деканат для студентов и преподавателей.

— Преимущества:

— Специализация именно под высшие учебные заведения;

— Учёт российской специфики учебного процесса;

— Поддержка всех этапов обучения (приём, учёба, выпуск);

— Интеграция с государственными системами (ЕПГУ, Рособрнадзор);

— Наличие технической поддержки и обучения;

— Соответствие требованиям Министерства образования.

— Недостатки:

— Высокая стоимость внедрения (от 300 000 рублей);

— Длительные сроки внедрения (4–8 месяцев);

— Устаревший интерфейс, не соответствующий современным стандартам UX;

— Отсутствие ИИ-модулей для прогнозирования и аналитики;

— Слабая адаптивность под мобильные устройства;

— Ограниченные возможности кастомизации;

— Сложность интеграции с современными веб-технологиями.

Стоимость: от 300 000 рублей (лицензия) + внедрение и обучение.

Целевая аудитория: средние и крупные вузы, нуждающиеся в комплексной системе управления учебным процессом.

Рисунок 1.7 — Интерфейс системы «Деканат» (ММИС)

1.4.4 Информационная система «Контингент»

В качестве аналога рассматривается информационная система «Контингент», официальный информационный ресурс которой расположен по адресу http://gis-kontingent.ru/information.html [24].

Основанием для предметного сопоставления является задача централизованного учёта данных в сфере образования. При сравнении выделены следующие параметры:

Параметры сравнения с разрабатываемой АИС:

— учёт контингента и образовательных организаций;

— состав сведений об обучающихся и поступающих;

— возможность обмена с внешними информационными системами;

— наличие средств контроля полноты и корректности данных;

— отчётность и разграничение доступа.

Подтверждённые данные о составе функций и условиях эксплуатации выбранной редакции в материалах проекта отсутствуют. Поэтому в сравнительной таблице для неё сохранён статус «Требует проверки».

Ранее приведённые сведения о подписке стоимостью 150 000 рублей в год, разработчике ООО «Контингент» и мобильном приложении исключены: их принадлежность выбранной системе не установлена.

Рисунок 1.8 — Интерфейс информационной системы «Контингент»

1.4.5 Сравнительный анализ аналогов

Таблица 1.11 — Исходный качественный обзор аналогов и уточнённый статус MVP

Характеристика	ЕПГУ	1С:Университет	Деканат	Контингент	Моя система
Стоимость	Бесплатно	Очень высокая	Высокая	Требует проверки по документации выбранной системы	Открытый код; эксплуатация требует ресурсов

Продолжение таблицы 1.11

Характеристика	ЕПГУ	1С:Университет	Деканат	Контингент	Моя система
Специализация под приёмную комиссию	Частичная	Общая	Общая	Требует проверки по документации выбранной системы	Полная
ИИ-прогнозирование шансов	Нет	Нет	Нет	Требует проверки по документации выбранной системы	Расчётная оценка положения
Очередь обзвона с приоритизацией	Нет	Нет	Нет	Требует проверки по документации выбранной системы	Эвристика; звонок делает сотрудник
Открытый сайт для абитуриентов	Да	Нет	Нет	Требует проверки по документации выбранной системы	Да
Интеграция с ЕПГУ и 1С	Да	Да	Частично	Требует проверки по документации выбранной системы	Обмен файлами XLSX / JSON / XML

Продолжение таблицы 1.11

Характеристика	ЕПГУ	1С:Университет	Деканат	Контингент	Моя система
Сроки внедрения	-	6–12 мес	4–8 мес	Требует проверки по документации выбранной системы	Локальный прототип; внедрение планируется
Простота освоения	Высокая	Сложно	Сложно	Требует проверки по документации выбранной системы	Высокая

Обзор аналогов отражает авторские оценки из исходного материала. Актуальные лицензии, цены и возможности конкретных конфигураций поставщиков требуют отдельной проверки. Для собственной системы указано состояние версии 1.

Ключевые преимущества разрабатываемой системы:

— Специализация именно на процессах приёмной комиссии — учтены все особенности приёмной кампании БГТУ (5 квот, внутренние экзамены СПО, жизненный цикл договора с двумя осями статусов).

— Учёт отраслевой специфики — гибкий парсер Excel, устойчивый к изменениям структуры выгрузки из ЕПГУ.

— ИИ-модуль прогнозирования — прогнозирование шансов поступления и вероятности заключения договора на базе Яндекс GPT / DeepSeek.

— Открытый Сайт 2 для абитуриентов — с поиском по номеру и соблюдением 152-ФЗ.

— Быстрое внедрение — силами IT-отдела БГТУ за 1–2 месяца.

— Интуитивный интерфейс — адаптирован под студентов-практикантов.

— Бесплатная разработка — курсовая работа без лицензионных отчислений.

1.5 Выводы по анализу предметной области

Основной объект учёта следует разделить на абитуриента и его заявления. Один номер ЕПГУ может соответствовать нескольким направлениям и формам обучения. Единый реестр и общий API устраняют необходимость независимо

наполнять два сайта; публичное представление должно формироваться отдельной схемой разрешённых полей.

В фактической выгрузке обнаружено 19 610 заявлений от 3 506 абитуриентов. Поэтому количество строк Excel нельзя использовать как количество физических лиц. Приоритет MVP — достоверность импорта, воспроизводимый рейтинг и изоляция персональных данных, а не неподтверждённая точность ИИ-прогноза.

2 Требования к продукту

В главе сформулированы целевые требования. Фактический статус каждого FR приведён в матрице трассировки после описания аналитических событий.

2.1 Функциональные требования

Функциональные требования определяют, какие именно действия должна выполнять система. Все требования сформулированы с учётом особенностей работы приёмной комиссии БГТУ: ежедневной Excel-выгрузки из ЕПГУ, работы с пятью квотами, жизненного цикла договора с двумя осями статусов, формирования списков для обзвона и наличия двух связанных сайтов (для секретарей и для абитуриентов).

2.1.1 Основные функции системы

FR-1: Ведение реестра абитуриентов

Описание: Система должна обеспечивать хранение, добавление, редактирование и поиск данных обо всех абитуриентах. Реестр содержит паспортные данные, СНИЛС, данные документов об образовании, индивидуальные достижения, льготы, контактные данные, данные родителя (если оплата производится им).

Входные данные: данные абитуриента (ФИО, дата рождения, паспорт, СНИЛС, адрес, телефон, email, документ об образовании, ИД, льготы, данные родителя).

Выходные данные: карточка абитуриента, список абитуриентов с фильтрацией по направлению, квоте, статусу.

Предусловия: пользователь авторизован (Сайт 1).

Постусловия: данные сохранены в базе, доступны для поиска и редактирования.

FR-2: Загрузка Excel-выгрузки из ЕПГУ

Описание: система должна обеспечивать загрузку ежедневной Excel-выгрузки из портала Госуслуг, содержащей данные о поданных заявлениях абитуриентов. Система автоматически обрабатывает файл, проверяет его на ошибки, сопоставляет данные с существующими записями по номеру абитуриента и обновляет реестр.

Парсер работает по семантическим признакам (названиям столбцов), а не по их позиции, что обеспечивает устойчивость к изменениям структуры выгрузки.

Входные данные: Excel-файл с данными абитуриентов (номер абитуриента, ФИО, паспортные данные, СНИЛС, контактные данные, баллы ЕГЭ, индивидуальные достижения, направления, квота, дата подачи заявления).

Выходные данные: обновлённый реестр абитуриентов, отчёт о загрузке (количество новых, обновлённых, ошибочных записей).

Предусловия: пользователь авторизован в системе (Сайт 1), файл выгрузки получен из ЕПГУ.

Постусловия: данные загружены в базу, записи обновлены, сформирован отчёт о загрузке.

FR-3: Распределение абитуриентов по квотам

Описание: система должна автоматически распределять абитуриентов по пяти категориям в зависимости от указанных льгот и выбранной формы обучения:

— особая квота (основания определяются действующими правилами приёма);

— отдельная квота (основания определяются действующими правилами приёма);

— целевое направление (с указанием организации-заказчика);

— основной набор (бюджет);

— платное обучение.

Входные данные: льготы абитуриента, выбранная форма обучения, наличие целевого направления.

Выходные данные: принадлежность абитуриента к одной из пяти категорий.

Предусловия: данные абитуриента загружены в систему.

Постусловия: абитуриент отнесён к соответствующей квоте, данные доступны для формирования списков.

FR-4: Расчёт рейтинга и формирование конкурсных списков

Описание: система должна формировать конкурсные списки по итоговым баллам. В импортируемом файле готовая «Сумма баллов» принимается как итог

конкурса, а не сумма всех предметных столбцов. При ручном пересчёте используются только выбранные экзамены и баллы индивидуальных достижений. Конкретные правила начисления ИД определяются действующими правилами приёма; льготы не прибавляются к итоговому баллу.

Входные данные: баллы ЕГЭ, индивидуальные достижения, квота, направление.

Выходные данные: пять конкурсных списков по каждому направлению (общий рейтинг, место абитуриента).

Предусловия: данные абитуриентов загружены, баллы ЕГЭ и ИД заполнены.

Постусловия: списки сформированы, доступны для просмотра и экспорта.

FR-5: Учёт результатов внутренних экзаменов для СПО

Описание: система должна позволять преподавателям вносить результаты внутренних экзаменов для абитуриентов, поступающих после СПО. Результаты учитываются при расчёте рейтинга наравне с баллами ЕГЭ.

Входные данные: результаты внутренних экзаменов (баллы по предметам), номер абитуриента.

Выходные данные: обновлённый рейтинг абитуриента.

Предусловия: абитуриент зарегистрирован в системе, экзамен проведён.

Постусловия: результаты внесены, рейтинг пересчитан.

FR-6: Жизненный цикл договора

Описание: система должна обеспечивать создание договора по шаблону Word с автозаполнением данными абитуриента, печать, отслеживание двух видов статусов:

— «ГДЕ?» — местоположение договора (в приёмной комиссии, в бухгалтерии, у ректора, в отделе кадров, у абитуриента, архив);

— «НА КАКОМ ЭТАПЕ?» — этап подписания (создан, на внутреннем подписании, готов к выдаче, выдан, подписан, ожидает оплаты, оплачен, отказ от места).

Входные данные: данные абитуриента, шаблон договора, действия пользователя по смене статуса.

Выходные данные: сгенерированный договор в формате .docx, история изменения статусов.

Предусловия: абитуриент выбрал платное обучение, шаблон договора утверждён юристом.

Постусловия: договор создан, статусы обновляются, история сохраняется.

FR-7: Подтверждение оплаты договора

Описание: система должна позволять секретарю отмечать факт оплаты договора путём установки флажка «Оплачен» в карточке договора. Файлы чеков не загружаются — секретарь ставит галочку после сверки с Excel-выгрузкой из бухгалтерии.

Входные данные: номер договора или номер абитуриента, факт подтверждения оплаты секретарём.

Выходные данные: обновлённый статус договора на «Оплачен», запись в истории изменений.

Предусловия: договор создан, оплата фактически произведена абитуриентом.

Постусловия: статус договора обновлён, запись об оплате сохранена в базе.

FR-8: Формирование списков для обзвона

Описание: система должна автоматически формировать списки абитуриентов, поступающих на платное обучение, для обзвона:

— в последнюю неделю изменения направления (когда абитуриент не проходит на бюджет);

— в последнюю неделю до закрытия оплаты (для напоминания).

Список содержит ФИО, номер телефона, направление, итоговый балл, текущий статус договора.

Входные данные: данные абитуриентов, статусы договоров, даты.

Выходные данные: список для обзвона с возможностью отметки результатов звонка.

Предусловия: данные загружены, договоры созданы.

Постусловия: список сформирован, доступен для работы секретарей.

FR-9: ИИ-прогнозирование шансов поступления

Описание: система должна обеспечивать прогнозирование вероятности зачисления абитуриента на выбранное направление. Прогноз пересчитывается раз в день (ночью, после загрузки свежей Excel-выгрузки). Прогноз доступен как секретарям (для приоритизации обзвона), так и абитуриентам (на Сайте 2).

Входные данные: баллы ЕГЭ, ИД, квота, направление, исторические данные за прошлые годы, текущая конкурсная ситуация.

Выходные данные: вероятность зачисления (в процентах), объяснение прогноза (какие факторы повлияли).

Предусловия: данные загружены, модель обучена на исторических данных.

Постусловия: прогноз сохранён, доступен на обоих сайтах.

FR-10: Сайт для абитуриентов (Сайт 2)

Описание: система должна предоставлять открытый доступ абитуриентам (без пароля) для:

— поиска по номеру абитуриента (уникален по всей РФ, находится во вкладке «Заявления» на Госуслугах);

— просмотра своего места в рейтинге по каждому направлению;

— просмотра обезличенных рейтингов по любому направлению;

— просмотра прогноза шансов поступления.

Персональные данные (ФИО, паспорт, СНИЛС) на сайте НЕ отображаются в соответствии с Федеральным законом № 152-ФЗ «О персональных данных».

Входные данные: номер абитуриента.

Выходные данные: информация о месте в рейтинге, прогноз шансов, обезличенный список.

Предусловия: данные загружены в систему.

Постусловия: абитуриент получил информацию.

FR-11: Чат-бот для абитуриентов

Описание: на Сайте 2 должен быть реализован скриптовый чат-бот (rule-based), который отвечает на типовые вопросы абитуриентов по ключевым словам и может подтягивать персональные данные по номеру абитуриента:

«Когда будут списки?»

«Какой проходной балл на направление?»

«Какое у меня место?»

«Что такое особая квота?»

Входные данные: текст вопроса абитуриента, номер абитуриента (при необходимости).

Выходные данные: ответ чат-бота из базы знаний.

Предусловия: Сайт 2 доступен, база знаний чат-бота заполнена.

Постусловия: абитуриент получил ответ.

FR-12: Отчётность и статистика

Описание: система должна формировать отчёты:

— рейтинговые списки по направлениям и квотам;

— статистика приёма (подано, принято, зачислено);

— статистика переполненных направлений;

— ведомость для бухгалтерии по оплатам;

— отчёт для Министерства образования.

Отчёты экспортируются в Excel и PDF.

Входные данные: период, тип отчёта, параметры фильтрации.

Выходные данные: отчёт в выбранном формате.

Предусловия: данные за период заполнены.

Постусловия: отчёт сформирован и доступен для скачивания.

FR-13: Архивация данных

Описание: система должна хранить данные о приёме в течение 6 лет с возможностью поиска по ФИО, году поступления, номеру договора.

Входные данные: данные за прошлые годы.

Выходные данные: найденные записи из архива.

Предусловия: данные за прошлые годы загружены в архив.

Постусловия: данные доступны для поиска.

FR-14: Выгрузка данных в 1С

Описание: после издания приказа о зачислении система должна сформировать пакет данных для передачи в 1С (ФИО, паспорт, СНИЛС, направление, форма обучения, дата зачисления, номер договора). Выгрузка выполняется по нажатию кнопки.

Входные данные: приказ о зачислении, данные зачисленных студентов.

Выходные данные: файл выгрузки (JSON/XML) для 1С.

Предусловия: приказ издан, данные подтверждены.

Постусловия: данные переданы в 1С.

FR-15: Аутентификация пользователей (Сайт 1)

Описание: система должна обеспечивать вход в Сайт 1 по логину и паролю. Все пользователи (секретари и практиканты) имеют одинаковый уровень доступа. Пароли хранятся в хешированном виде (bcrypt).

Входные данные: логин, пароль.

Выходные данные: доступ к системе.

Предусловия: пользователь зарегистрирован в системе.

Постусловия: пользователь авторизован, сессия создана.

2.1.2 Пользовательские истории (User Stories)

Высокий приоритет:

— US-1: Как секретарь приёмной комиссии, я хочу загружать ежедневную Excel-выгрузку из Госуслуг, чтобы получать актуальные данные о заявлениях абитуриентов без ручного ввода (Story Points: 8).

— US-2: Как секретарь, я хочу видеть полный реестр абитуриентов с поиском и фильтрацией, чтобы быстро находить нужную информацию (Story Points: 5).

— US-3: Как секретарь, я хочу, чтобы система автоматически формировала пять конкурсных списков, чтобы не считать рейтинг вручную (Story Points: 13).

— US-4: Как секретарь, я хочу создавать договор по шаблону Word с автозаполнением, чтобы не тратить время на ручной ввод данных абитуриента (Story Points: 8).

— US-5: Как секретарь, я хочу отслеживать статус договора по двум осям («ГДЕ?» и «НА КАКОМ ЭТАПЕ?»), чтобы всегда знать, где находится документ (Story Points: 13).

— US-6: Как секретарь, я хочу получать список для обзвона платников, чтобы своевременно связаться с абитуриентами (Story Points: 8).

Средний приоритет:

— US-7: Как секретарь, я хочу видеть прогноз вероятности заключения договора для каждого платника, чтобы приоритизировать звонки (Story Points: 13).

— US-8: Как абитуриент, я хочу найти своё место в рейтинге по номеру, чтобы понимать свои шансы на поступление (Story Points: 5).

— US-9: Как абитуриент, я хочу видеть прогноз своих шансов на поступление, чтобы принять решение о выборе направления (Story Points: 8).

Низкий приоритет:

— US-10: Как абитуриент, я хочу задать вопрос чат-боту, чтобы быстро получить ответ без звонка в приёмную комиссию (Story Points: 5).

— US-11: Как сотрудник приёмной комиссии, я хочу выгружать данные в 1С после приказа, чтобы не вводить их повторно (Story Points: 8).

— US-12: Как администратор, я хочу видеть статистику по переполненным направлениям, чтобы принимать управленческие решения (Story Points: 5).

— US-13: Как секретарь, я хочу отмечать оплату договора галочкой после сверки с Excel из бухгалтерии, чтобы не загружать файлы чеков (Story Points: 3).

— US-14: Как секретарь, я хочу формировать отчёты в Excel и PDF, чтобы передавать данные руководству и бухгалтерии (Story Points: 8).

— US-15: Как секретарь, я хочу вносить результаты внутренних экзаменов для абитуриентов СПО, чтобы учитывать их в рейтинге (Story Points: 5).

— US-16: Как абитуриент, я хочу видеть обезличенный рейтинг по своему направлению, чтобы сравнивать себя с другими абитуриентами (Story Points: 5).

— US-17: Как секретарь, я хочу искать абитуриента по ФИО, номеру или СНИЛС, чтобы быстро находить нужную запись (Story Points: 3).

— US-18: Как секретарь, я хочу видеть историю изменений статусов договора, чтобы понимать, кто и когда менял статус (Story Points: 5).

— US-19: Как секретарь, я хочу экспортировать конкурсные списки в Excel, чтобы передавать их руководству (Story Points: 3).

— US-20: Как администратор, я хочу архивировать данные за прошлые годы, чтобы хранить их 6 лет в соответствии с требованиями (Story Points: 8).

2.1.3 Варианты использования (Use Cases)

Use Case UC-1: Загрузка Excel-выгрузки из ЕПГУ и обновление реестра

Актёры: Секретарь приёмной комиссии, Система.

Цель: загрузить данные из ЕПГУ и обновить реестр абитуриентов.

Предусловия: секретарь авторизован, Excel-файл получен из ЕПГУ.

Основной сценарий:

— Секретарь выбирает «Загрузить выгрузку».

— Система отображает форму загрузки файла.

— Секретарь выбирает Excel-файл.

— Система проверяет формат и структуру файла.

— Система сопоставляет записи по номеру абитуриента.

— Система добавляет новые записи и обновляет существующие.

— Система формирует отчёт о загрузке (новые, обновлённые, ошибочные).

— Секретарь просматривает отчёт и подтверждает завершение.

— Альтернативные сценарии:

— А. Файл повреждён или имеет неверный формат — система выводит сообщение об ошибке, загрузка не выполняется.

— Б. В файле есть дубли — система помечает их и предлагает обработать вручную.

— B. Структура файла изменена (столбцы в другом порядке) — гибкий парсер распознаёт столбцы по семантическим признакам и продолжает загрузку.

Постусловия: данные загружены, реестр обновлён, отчёт сформирован.

Use Case UC-2: Формирование пяти конкурсных списков

Актёры: Секретарь, Система.

Цель: автоматически сформировать конкурсные списки по каждому направлению и квоте.

Предусловия: данные абитуриентов загружены, баллы ЕГЭ и ИД заполнены, результаты внутренних экзаменов внесены.

Основной сценарий:

— Секретарь выбирает «Формирование списков».

— Система рассчитывает итоговый балл для каждого абитуриента (ЕГЭ + ИД).

— Система распределяет абитуриентов по пяти квотам.

— Система сортирует абитуриентов по убыванию итогового балла.

— Система формирует пять списков по каждому направлению.

— Секретарь просматривает списки, проверяет корректность.

— Секретарь экспортирует списки в Excel или PDF.

— Альтернативные сценарии:

— A. Если у абитуриента не хватает данных (нет баллов ЕГЭ) — система помечает его как «Неполные данные» и исключает из списка.

— Б. Если конкурс на направление превышает количество мест — система выделяет направление как «Переполненное».

Постусловия: пять конкурсных списков сформированы и доступны для работы.

Use Case UC-3: Создание и отслеживание договора

Актёры: Секретарь, Система, Бухгалтерия, Ректор, Отдел кадров, Абитуриент.

Цель: создать договор, провести его по всем этапам подписания и отслеживать статус.

Предусловия: абитуриент выбрал платное обучение, шаблон договора утверждён юристом.

Основной сценарий:

— Секретарь выбирает абитуриента и нажимает «Создать договор».

— Система автозаполняет договор данными абитуриента и генерирует файл .docx.

— Секретарь распечатывает договор.

— Секретарь меняет статус на «В бухгалтерии / На внутреннем подписании».

— Бухгалтерия подписывает, статус меняется на «У ректора / На внутреннем подписании».

— Ректор подписывает, статус меняется на «В отделе кадров / На печати».

— Отдел кадров ставит печать, статус меняется на «В приёмной комиссии / Готов к выдаче».

— Секретарь выдаёт договор абитуриенту, статус «У абитуриента / Выдан».

— Абитуриент подписывает и приносит договор, секретарь сверяется с Excel-выгрузкой из бухгалтерии.

— Секретарь ставит галочку «Оплачен», статус «В приёмной комиссии / Оплачен».

— Альтернативные сценарии:

— А. Абитуриент отказался от платного обучения — статус «Отказ от места».

— Б. Абитуриент поступил в другой вуз — статус «Отказ от места», место освобождается.

Постусловия: договор проведён по всем этапам, оплата подтверждена, статус обновлён.

Use Case UC-4: Поиск абитуриентом своего места в рейтинге

Актёры: Абитуриент, Сайт 2.

Цель: найти своё место в рейтинге и узнать прогноз шансов.

Предусловия: данные загружены, абитуриент знает свой номер (из Госуслуг).

Основной сценарий:

— Абитуриент открывает Сайт 2.

— Абитуриент вводит номер абитуриента в поле поиска.

— Система находит запись.

— Система отображает список направлений, куда подал абитуриент.

— Система показывает место в рейтинге по каждому направлению.

— Система отображает прогноз шансов поступления.

— Абитуриент может задать вопрос чат-боту.

— Альтернативные сценарии:

— А. Номер не найден — система предлагает проверить номер и обратиться в приёмную комиссию.

— Б. Данные ещё не загружены — система сообщает о дате следующего обновления.

Постусловия: абитуриент получил информацию о своём месте и прогнозе.

При описании варианта использования основной сценарий рассматривается вместе с отказами и отсутствующими данными. Неверный формат файла, неизвестный номер или запрещённое изменение состояния не являются успешным завершением операции. Система должна возвращать понятный результат и сохранять согласованное состояние базы. Поэтому диаграмма вариантов использования показывает границы ответственности участников, а подробные проверки раскрываются в требованиях и диаграммах последовательностей. Такое разделение позволяет сопоставить действие пользователя с конкретной серверной операцией и проверить не только удобный пример, но и существенные альтернативные исходы.

2.1.4 Диаграмма вариантов использования

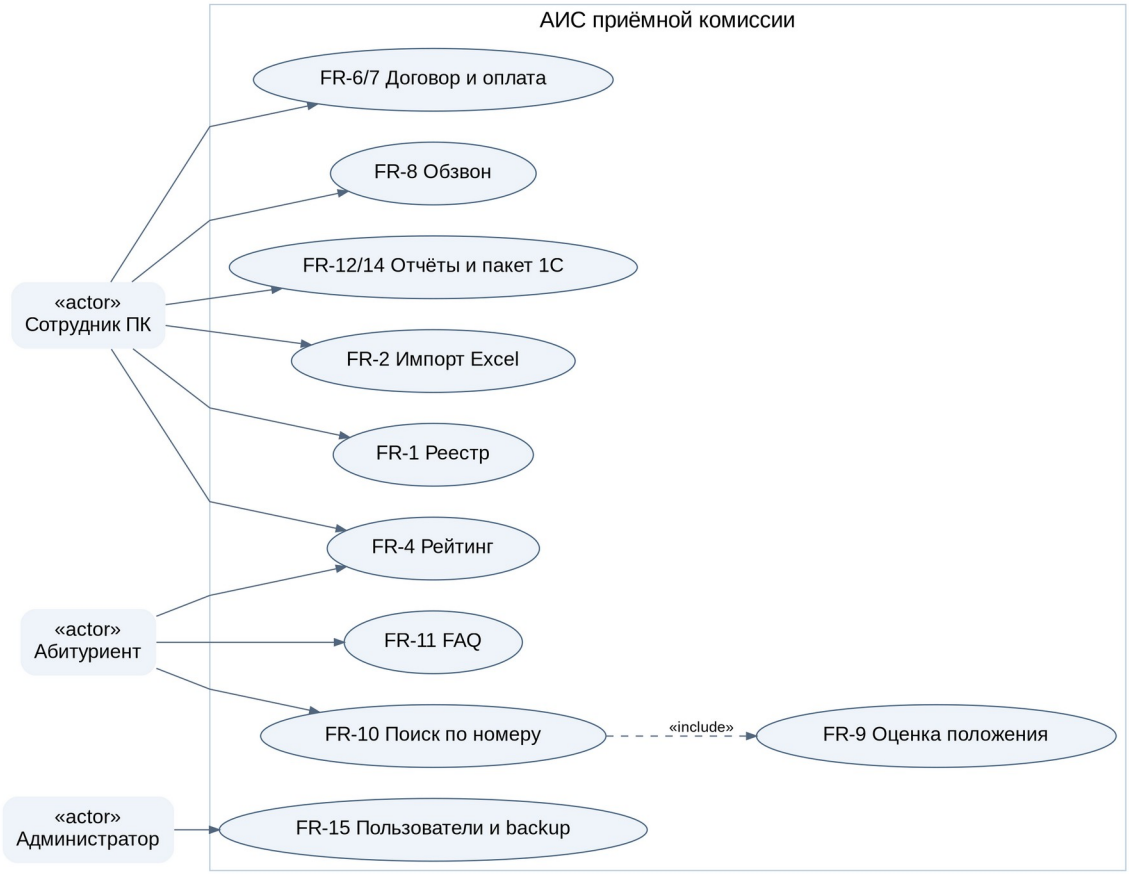


Рисунок 2.1 — Варианты использования АИС

2.2 Нефункциональные требования

2.2.1 Производительность

Время загрузки списка абитуриентов (до 10 000 записей) не должно превышать 3 секунд.

Поиск по реестру (по номеру абитуриента, ФИО, направлению) — не более 2 секунд.

Формирование пяти конкурсных списков по одному направлению — не более 5 секунд.

Формирование отчётов — не более 30 секунд.

Система должна поддерживать одновременную работу до 10 пользователей Сайта 1.

Сайт 2 должен выдерживать до 500 одновременных запросов от абитуриентов.

Генерация договора по шаблону Word — не более 10 секунд.

2.2.2 Безопасность

Все пароли пользователей Сайта 1 хранятся в хешированном виде с использованием алгоритма bcrypt.

Сайт 2 работает без авторизации, но отображает только обезличенные данные (номер абитуриента, баллы, квота, направление). Персональные данные (ФИО, паспорт, СНИЛС, контакты) на Сайте 2 НЕ отображаются в соответствии с Федеральным законом № 152-ФЗ «О персональных данных».

Резервные копии базы данных хранятся в зашифрованном виде.

Доступ к Сайту 1 — только из локальной сети БГТУ.

Защита от SQL-инъекций и XSS-атак на обоих сайтах.

2.2.3 Масштабируемость

Архитектура должна позволять добавление новых модулей (например, модуль взаимодействия с другими вузами).

Система должна поддерживать увеличение базы данных до 500 ГБ без потери производительности.

Возможность интеграции с 1С для обмена данными о зачисленных студентах.

Возможность добавления новых направлений подготовки и квот без изменения кода.

2.2.4 Надёжность

Общее время доступности системы — не менее 99,5% в рабочее время (8:00–20:00).

Автоматическое резервное копирование базы данных — ежедневно в 02:00.

Время восстановления из резервной копии — не более 1 часа.

Система должна обеспечивать целостность данных при одновременной работе нескольких пользователей.

При загрузке Excel-выгрузки система должна сохранять предыдущие версии данных для возможности отката.

2.2.5 Удобство использования (Usability)

Интерфейс Сайта 1 должен быть адаптирован для пользователей с разным уровнем компьютерной грамотности (включая студентов-практикантов, не владеющих Excel).

Основные функции должны быть доступны за 3 клика от главного экрана.

Интерфейс должен иметь контекстные подсказки для сложных операций (например, при смене статуса договора).

Сайт 2 должен быть адаптирован для работы на смартфонах (мобильная версия).

Время обучения нового пользователя Сайта 1 — не более 1 часа.

2.2.6 Совместимость

Сайт 1 и Сайт 2 должны корректно работать в браузерах: Google Chrome, Яндекс.Браузер, Mozilla Firefox (последние версии).

Сайт 1 должен работать в локальной сети БГТУ без необходимости установки дополнительного ПО на рабочие станции.

Сайт 2 должен корректно отображаться на устройствах с разрешением от 320 px (смартфоны) до 1920 px (десктопы).

2.2.7 Требования к данным

Система должна обеспечивать ежедневную загрузку Excel-выгрузки из ЕПГУ.

Система должна хранить данные о приёме в течение 6 лет.

Система должна обеспечивать возможность выгрузки данных в 1С после издания приказа о зачислении.

Формат Excel-выгрузки должен соответствовать формату, утверждённому ЕПГУ.

Шаблоны договоров хранятся в формате .docx и могут быть обновлены юристом без изменения кода системы.

2.3 Проектирование аналитических событий и свойств

2.3.1 Цели проектирования событий

Аналитические события проектируются для отслеживания поведения пользователей в системе и измерения эффективности продуктовых воронок, описанных в разделе 1.3. События позволяют:

— отслеживать конверсию на каждом шаге воронки;

— выявлять проблемные этапы с высокими потерями;

— проводить сегментный анализ по платформе, типу пользователя и источнику трафика;

— измерять эффект от внедрения новых функций (например, ИИ-прогноза или чат-бота).

2.3.2 Словарь аналитических событий

Таблица 2.1 — Проектный словарь аналитических событий

№	Событие (Event)	Описание	Свойства (Properties)
1	application_submitted	Абитуриент подал заявление через ЕПГУ	applicant_id, direction, quota_type, submission_date
2	data_loaded	Данные загружены из Excel-выгрузки в систему	file_name, records_count, new_records, updated_records, errors_count
3	data_validated	Данные прошли валидацию	applicant_id, validation_status, error_fields
4	quota_assigned	Абитуриент распределён по квоте	applicant_id, quota_type, direction

Продолжение таблицы 2.1

№	Событие (Event)	Описание	Свойства (Properties)
5	rating_calculated	Рассчитан итоговый балл	applicant_id, ege_score, id_score, total_score, direction
6	in_competitive_list	Абитуриент попал в конкурсный список	applicant_id, direction, quota_type, rank, total_score
7	in_paid_list	Абитуриент попал в список на платное обучение	applicant_id, direction, total_score
8	contract_created	Договор создан по шаблону Word	applicant_id, contract_number, direction, creation_date
9	contract_signed_internal	Договор подписан внутри вуза	contract_number, signing_stage, signed_by, timestamp
10	contract_issued	Договор выдан абитуриенту	contract_number, issue_date, issued_by
11	contact_attempted	Секретарь связался с абитуриентом	applicant_id, contract_number, call_result, call_date
12	payment_confirmed	Оплата подтверждена (галочка в системе)	contract_number, confirmation_date, confirmed_by
13	site2_opened	Абитуриент открыл Сайт 2	session_id, device_type, platform, timestamp
14	number_entered	Ввёл номер абитуриента в поиск	session_id, applicant_number, search_result
15	rating_viewed	Посмотрел своё место в рейтинге	applicant_id, direction, rank, view_date
16	forecast_viewed	Посмотрел прогноз шансов поступления	applicant_id, direction, forecast_value, view_date

Продолжение таблицы 2.1

№	Событие (Event)	Описание	Свойства (Properties)
17	bot_question_asked	Задал вопрос чат-боту	session_id, question_text, bot_response, response_time

2.3.3 Свойства событий (Properties)

Для каждого события определены свойства, которые позволяют проводить сегментный анализ:

Таблица 2.2 — Проектные свойства аналитических событий

Свойство	Тип данных	Описание	Пример значения
applicant_id	string	Уникальный номер абитуриента	"2026-001234"
direction	string	Направление подготовки	"09.03.01 Информатика и вычислительная техника"
quota_type	string	Тип квоты	"особая", "отдельная", "целевая", "основная", "платная"
platform	string	Платформа пользователя	"web", "mobile", "tablet"
device_type	string	Тип устройства	"desktop", "smartphone", "tablet"
source	string	Источник трафика	"organic", "referral", "direct"
user_type	string	Тип пользователя	"new", "returning"
call_result	string	Результат звонка	"success", "no_answer", "refused", "callback_later"
validation_status	string	Статус валидации	"passed", "failed", "partial"

Продолжение таблицы 2.2

Свойство	Тип данных	Описание	Пример значения
signing_stage	string	Этап подписания договора	"created", "internal_signing", "ready_to_issue", "issued", "signed", "awaiting_payment", "paid", "refused"

2.3.4 Интеграция событий с продуктовыми воронками

События напрямую соответствуют шагам трёх продуктовых воронок, описанных в разделе 1.3:

Воронка №1 (Подача заявления): application_submitted → data_loaded → data_validated → quota_assigned → rating_calculated → in_competitive_list

Воронка №2 (Заключение договора): in_paid_list → contract_created → contract_signed_internal → contract_issued → contact_attempted → payment_confirmed

Воронка №3 (Использование Сайта 2): site2_opened → number_entered → rating_viewed → forecast_viewed → bot_question_asked

2.3.5 Технические требования к сбору событий

События записываются в отдельную таблицу analytics_events в базе данных SQLite.

Каждое событие содержит временную метку (timestamp) с точностью до миллисекунд.

События не содержат персональных данных абитуриентов (только анонимизированные идентификаторы).

Данные событий хранятся в течение 2 лет для возможности ретроспективного анализа.

Для анализа событий используется встроенный SQL-инструментарий SQLite (оконные функции, агрегации).

2.4 Матрица трассировки требований к версии 1

Таблица 2.3 — Требования, реализация и границы MVP

ID	Функция	Статус	Подтверждённое поведение
FR-1	Реестр и карточка	Частично	Реестр, контакты, поиск и JSON-профиль; расширенные вкладки и отдельная форма родителя требуют развития.
FR-2	Импорт ЕПГУ	Реализовано	Алиасы колонок, сохранение номера, отдельные заявления, upsert и отчёт ошибок.
FR-3	Категории конкурса	Частично	Пять явных категорий; правовая квалификация льгот по документам не автоматизирована.
FR-4	Рейтинг	Реализовано	Итоговый балл, равные места, пять списков, планы мест; не заменяет юридические правила приказа.
FR-5	Внутренние экзамены СПО	Частично	Баллы сохраняются в exams; отдельного рабочего места преподавателя нет.
FR-6	Договор	Реализовано в MVP	DOCX, независимые location/stage и история; сложные юридические маршруты требуют настройки.
FR-7	Подтверждение оплаты	Реализовано	Сверка по номеру договора и признаку полной оплаты; частичные суммы не учитываются.
FR-8	Обзвон	Частично	Очередь по срокам и эвристике, результат звонка; нет автоматического набора номера и фонового расписания.
FR-9	Прогноз	Частично	rules_v1 по рейтингу и плану; числовая вероятность не выдаётся, обученная модель не подключена.
FR-10	Публичный сайт	Реализовано	Анонимный поиск, каталог, списки, CSV; персональные поля не публикуются.
FR-11	Чат-бот	Реализовано	Локальные правила FAQ, без внешнего ИИ API.
FR-12	Отчёты	Частично	Статистика, рейтинг XLSX/PDF, бухгалтерский XLSX; полный набор регламентных форм требует согласования.

Продолжение таблицы 2.3

ID	Функция	Статус	Подтверждённое поведение
FR-13	Архив 6 лет	Частично	Годовые данные сохраняются; регламент хранения и удаления не автоматизирован.
FR-14	Выгрузка в 1С	Частично	JSON/XML после приказа; формат и транспорт конкретной конфигурации 1С не интегрированы.
FR-15	Доступ сотрудников	Реализовано	bcrypt, JWT, сетевое ограничение, создание пользователя через CLI.

Идентификаторы FR соответствуют тексту главы 2. На исторических схемах архива встречается другая нумерация; для оценки реализованной версии используются данная матрица и актуальные диаграммы глав 7–9.

3 Прототипирование и интерфейсы

Все изображения в разделе 3.2 являются проектными экранами из исходного документа. Проценты ИИ-прогноза и отдельные элементы этих макетов не являются результатом обучения модели. Актуальные экраны работающих сайтов приведены в главе 9.

3.1 Методология прототипирования

Для разработки интерфейса автоматизированной информационной системы приёмной комиссии применяется итеративная методология прототипирования, состоящая из пяти последовательных этапов:

Исследование и анализ требований. Изучение рабочих процессов приёмной комиссии, интервью с секретарями и анализ «болевых точек» текущего процесса (работа с Excel-таблицами, бумажными договорами).

Создание wireframes (набросков). Разработка структурных схем основных экранов без детального дизайна для определения расположения ключевых элементов: навигации, таблиц, форм и кнопок действий.

Разработка дизайн-системы. Создание единого визуального языка: цветовой палитры, типографики и набора компонентов (кнопки, карточки, бейджи статусов).

Создание интерактивного прототипа. Объединение wireframes в кликабельный прототип с основными пользовательскими сценариями (User Flow).

Валидация с пользователями. Демонстрация прототипа ключевым пользователям (Директору ПК, секретарям) для сбора обратной связи перед началом программирования. Обратная связь используется для уточнения названий действий, порядка заполнения полей и сообщений об ошибках до реализации соответствующего сценария.

Особенность проекта: система состоит из двух связанных интерфейсов с разной целевой аудиторией:

Сайт 1 — рабочий инструмент для сотрудников ПК (плотный десктопный интерфейс 1440×900 px, ориентированный на работу с большими массивами данных).

Сайт 2 — открытый информационный портал для абитуриентов (адаптивный мобильный интерфейс 375×812 px, минималистичный, с акцентом на быстрый поиск по номеру).

3.2 Wireframes

Wireframes созданы с учётом контекста использования: высокая нагрузка в период приёмной кампании требует минимального количества кликов для частых операций и интуитивной понятности для пользователей с разным уровнем компьютерной грамотности (включая студентов-практикантов).

3.2.1 Экраны Сайта 1

— Главная панель (Dashboard)

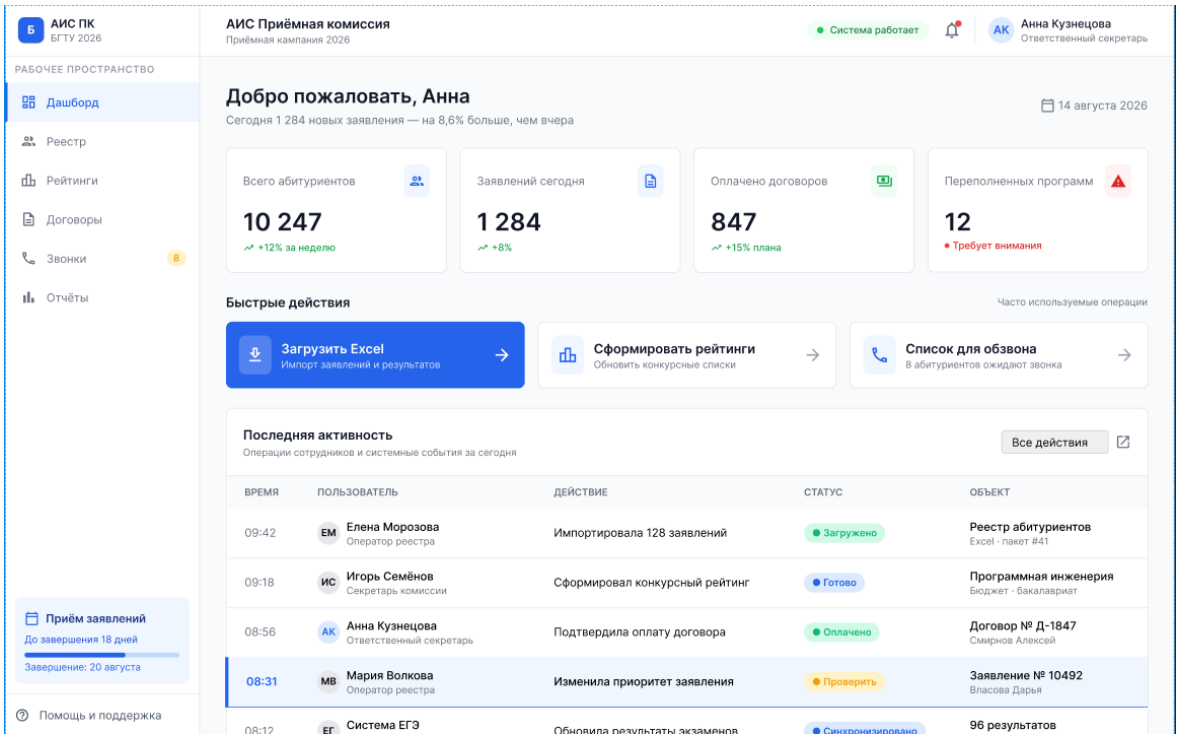


Рисунок 3.1 — Главная панель (исходный прототип)

Приветствие: «Добро пожаловать, [Имя]».

Ключевые виджеты: «Всего абитуриентов» (с динамикой за неделю), «Заявлений сегодня», «Оплачено договоров» (% от плана), «Переполненных программ» (с пометкой «Требует внимания»).

Быстрые действия: крупные кнопки «Загрузить Excel», «Сформировать рейтинги», «Список для обзвона» (с индикатором количества ожидающих звонка).

Таймер кампании: обратный отсчёт до завершения приёма заявлений.

Лента последних действий: таблица с временем, именем пользователя и выполненным действием (например, «Импортировала 128 заявлений», «Подтвердила оплату договора»).

— Реестр абитуриентов

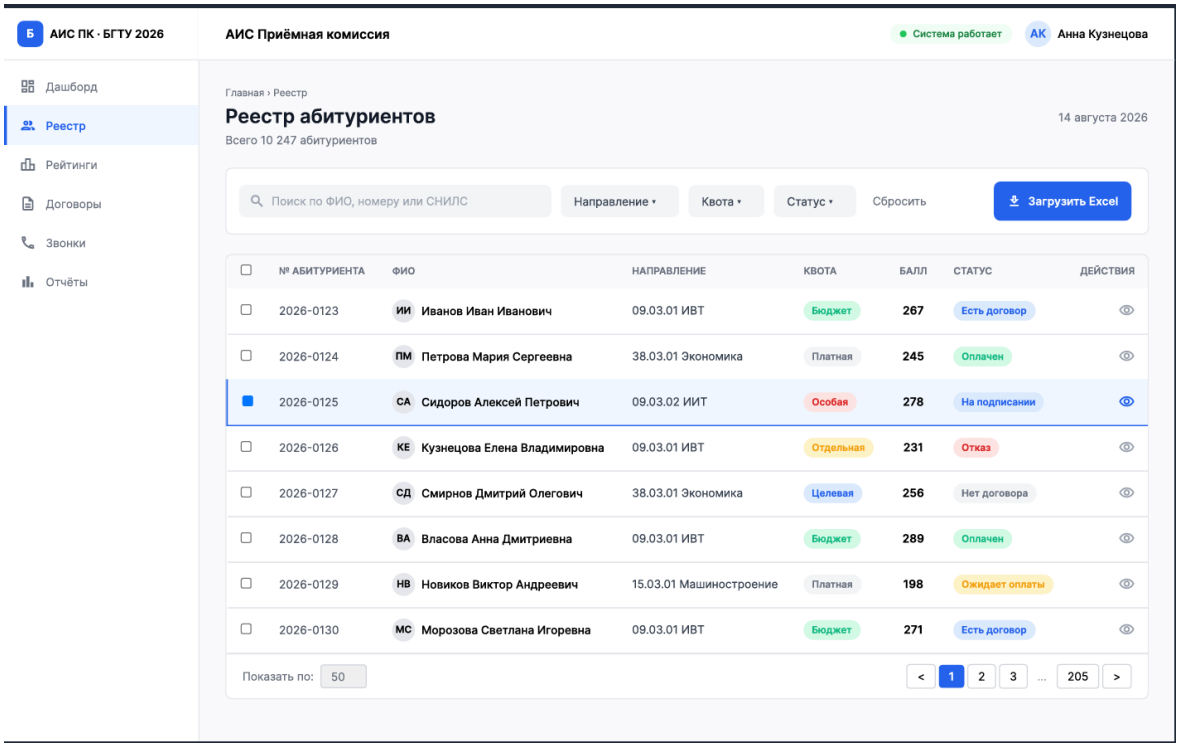


Рисунок 3.2 — Реестр абитуриентов (исходный прототип)

При работе с реестром важно различать поиск человека и выбор его заявления. Один поступающий может участвовать в нескольких конкурсах, поэтому совпадение фамилии не означает дублирование записи. Фильтры направления, формы обучения и квоты задают рабочую выборку, а карточка позволяет уточнить сведения конкретного человека. Такое разделение уменьшает риск изменения данных не того заявления. В прототипе показан состав элементов управления; фактические возможности сохранения и проверки данных оцениваются отдельно по реализованным сценариям.

Инструменты: строка поиска (по ФИО, номеру или СНИЛС), фильтры (по направлению, квоте, статусу), кнопка «Сбросить».

Действия: кнопка «Загрузить Excel» для импорта ежедневной выгрузки, пагинация («Показать по 50», навигация по страницам).

— Карточка абитуриента (модальное окно или отдельная страница)

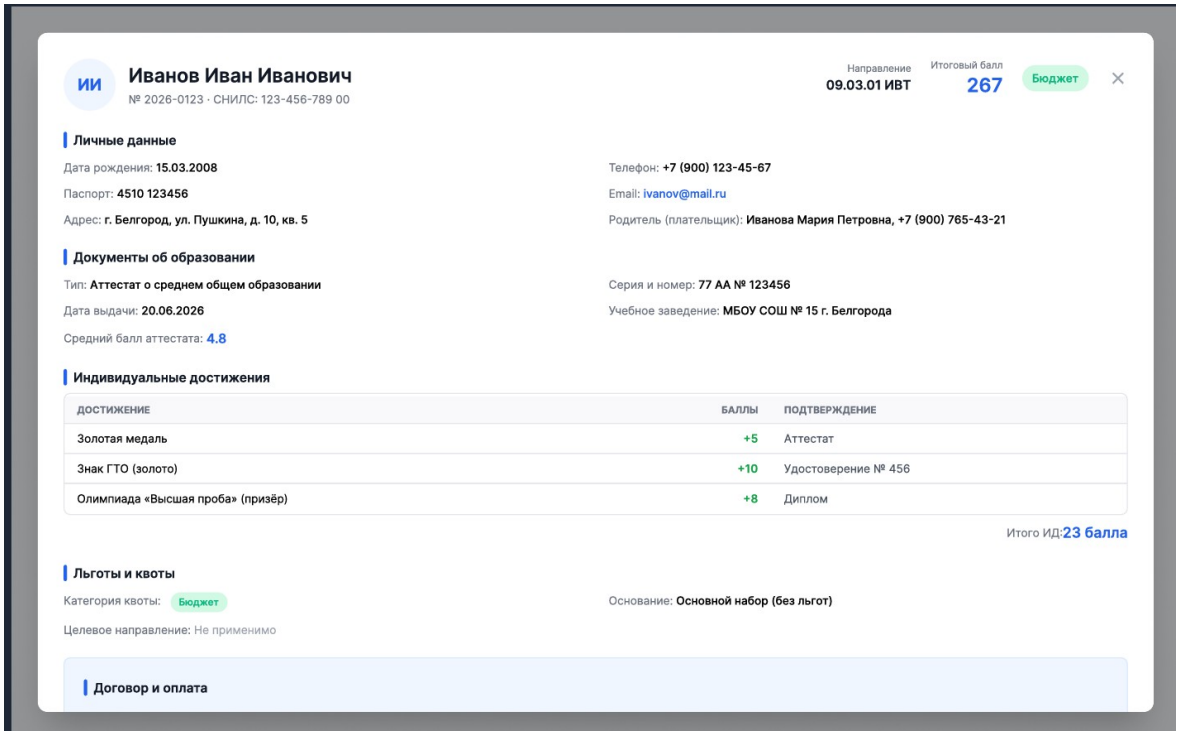


Рисунок 3.3 — Карточка абитуриента (исходный прототип)

Личные данные: ФИО, номер абитуриента, СНИЛС, дата рождения, паспорт, адрес, телефон, email, данные родителя-плательщика.

Документы об образовании: тип документа, дата выдачи, серия/номер, учебное заведение.

Индивидуальные достижения: таблица с колонками «Достижение», «Баллы», «Подтверждение» (например: Золотая медаль +5, Знак ГТО +10). Итоговая сумма баллов ИД.

Льготы и квоты: категория квоты, основание, организация-заказчик (для целевиков).

Блок «Договор и оплата»:

Селектор «ГДЕ?» (В ПК, Бухгалтерия, Ректор, Отдел кадров, У абитуриента).

Селектор «НА КАКОМ ЭТАПЕ?» (Создано, На подписании, Оплачен, Отказ).

Чекбокс «Оплачен» с текстовой подсказкой: «Отметить после сверки с Excel-выгрузкой бухгалтерии» (файлы чеков не загружаются).

Кнопки действий: «Сгенерировать docx», «Распечатать», «Изменить "ГДЕ?"», «Изменить этап», «История изменений».

— Управление договорами

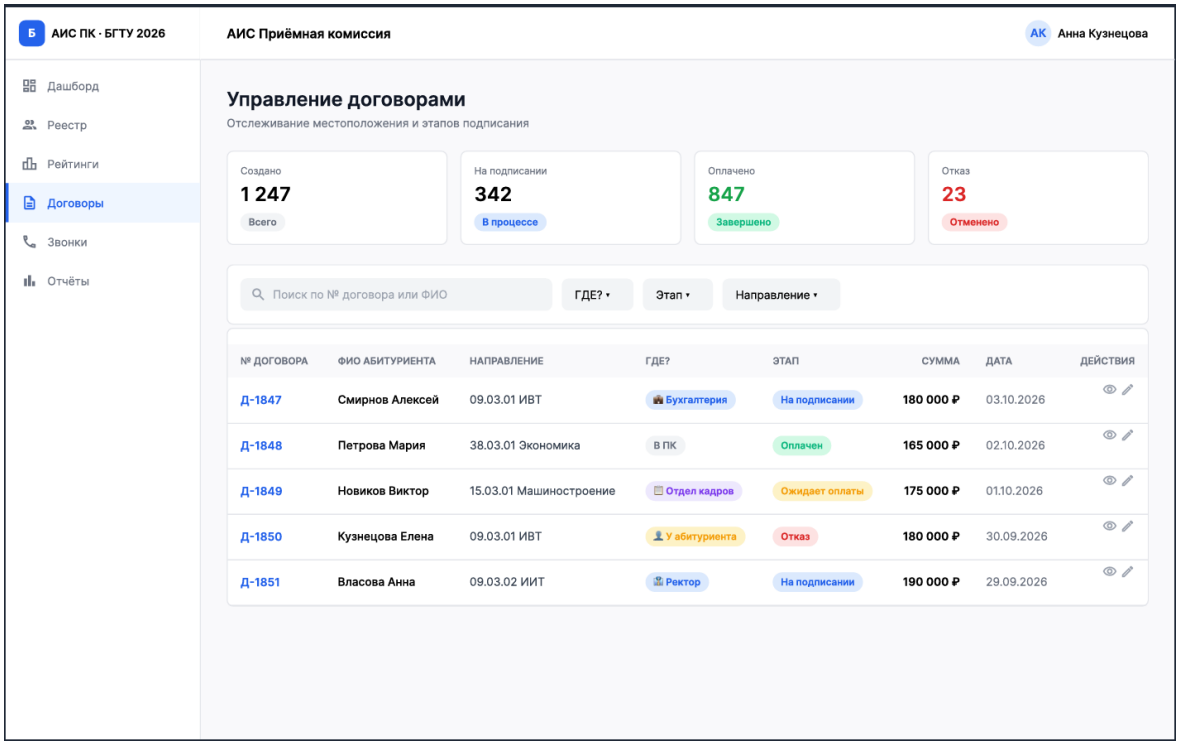


Рисунок 3.4 — Управление договорами (исходный прототип)

Сводные виджеты: «Всего», «В процессе», «Завершено», «Отменено».

Фильтры: по местоположению («ГДЕ?»), этапу подписания и направлению.

— Списки для обзвона

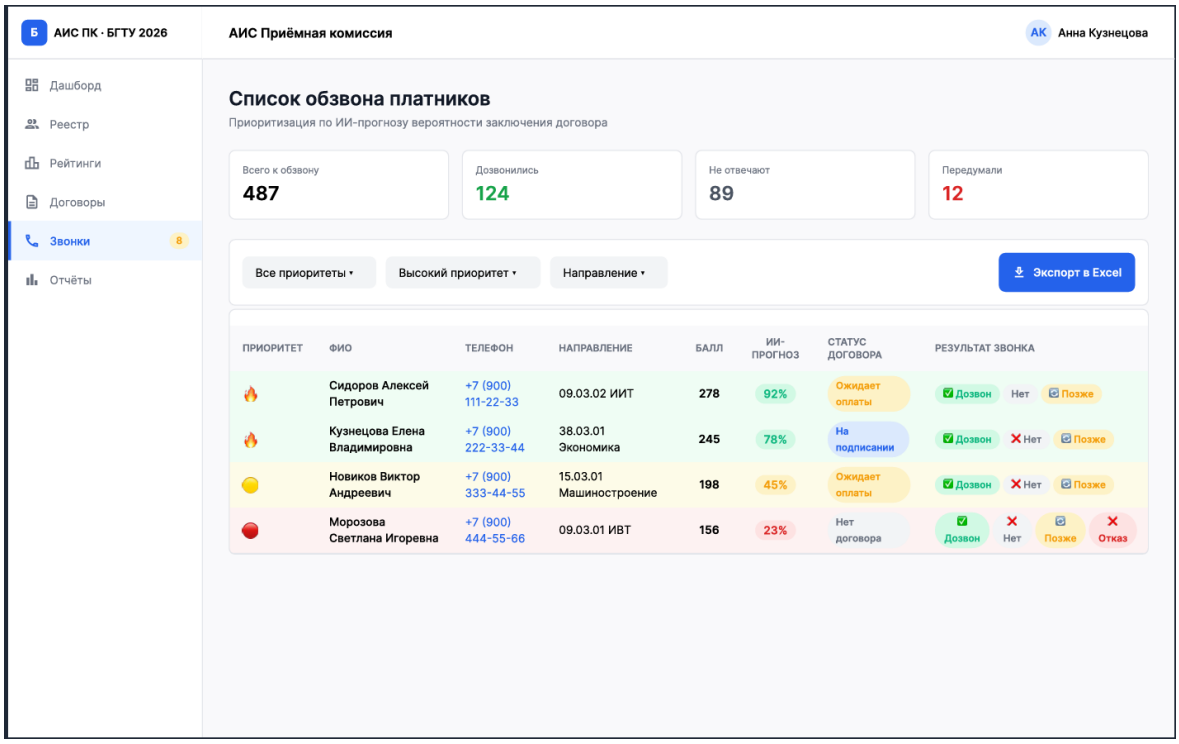


Рисунок 3.5 — Списки для обзвона (исходный прототип)

Статистика звонков: виджеты «Всего к обзвону», «Дозвонились», «Не отвечают», «Передумали».

Результат звонка: набор чекбоксов в строке («✅ Дозвон», «❌ Нет», «🔄 Позже», «❌ Отказ»).

Действия: кнопка «Экспорт в Excel».

— Конкурсные списки

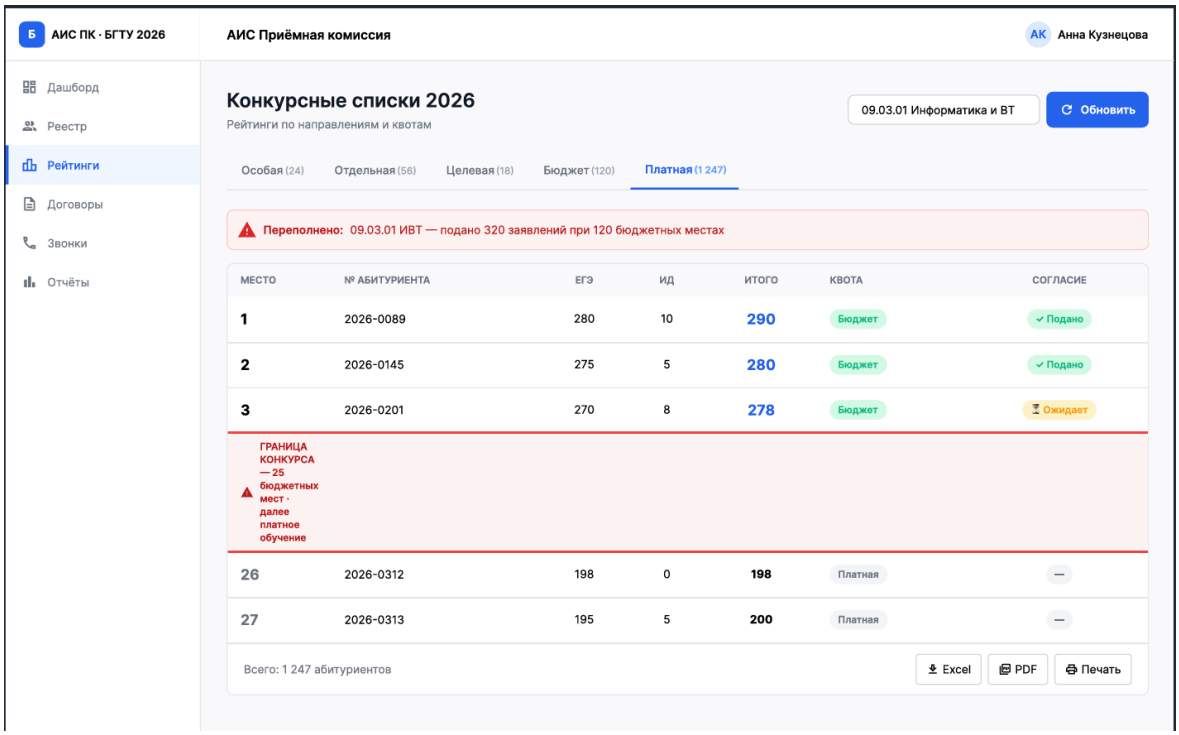


Рисунок 3.6 — Конкурсные списки (исходный прототип)

Вкладки по квотам: Особая, Отдельная, Целевая, Бюджет, Платная (с указанием количества человек в каждой).

Предупреждения: яркий баннер при переполнении направления (например, «Переполнено: 09.03.01 ИВТ — подано 320 заявлений при 120 бюджетных местах»).

Визуализация: горизонтальная линия «ГРАНИЦА КОНКУРСА», отделяющая бюджетные места от платных. Кнопки экспорта: Excel, PDF, Печать.

— Отчёты и статистика

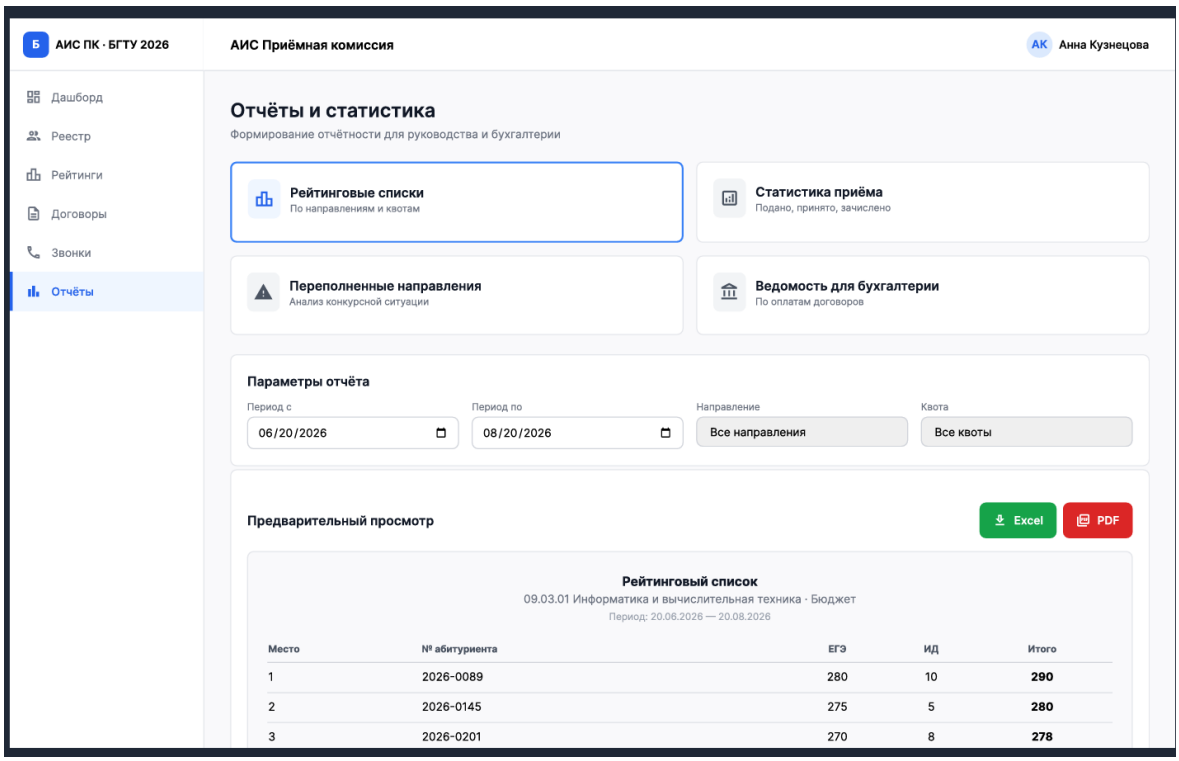


Рисунок 3.7 — Отчёты и статистика (исходный прототип)

Категории: Переполненные направления, Рейтинговые списки, Статистика приёма, Ведомость для бухгалтерии.

Параметры: выбор периода (с/по), направления и квоты.

Превью: визуализация отчёта в виде таблицы с кнопками сохранения в Excel или PDF.

3.2.2 Экраны Сайта 2

— Главная страница

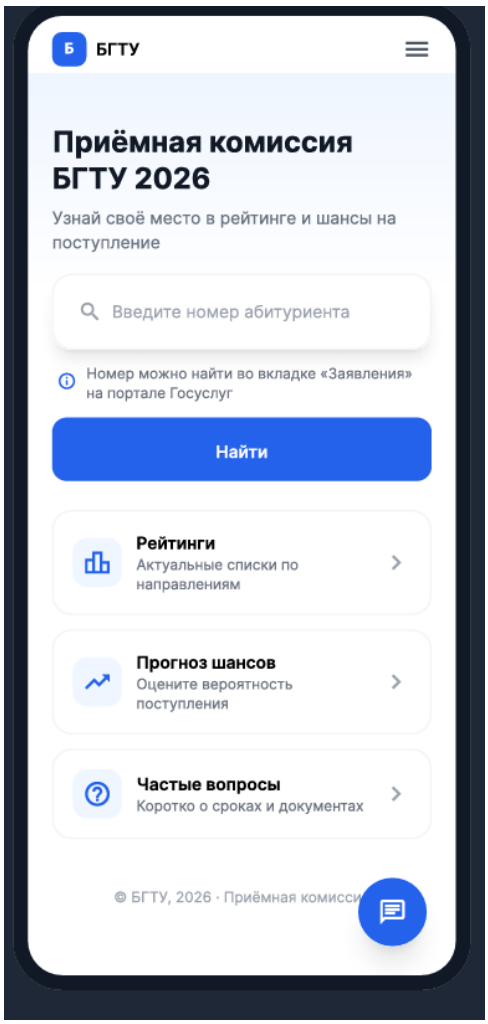


Рисунок 3.8 — Главная страница Сайта 2 (исходный прототип)

Заголовок: «Приёмная комиссия БГТУ 2026. Узнай своё место в рейтинге и шансы на поступление».

Поиск: крупное поле «Введите номер абитуриента» с подсказкой: «Номер можно найти во вкладке "Заявления" на портале Госуслуг». Кнопка «Найти».

Статистика кампании: виджеты «Всего заявлений», «Направлений», обратный отсчёт «До завершения приёма X дней».

Навигация: ссылки на разделы «Рейтинги», «Прогноз шансов», «Частые вопросы».

— Личный кабинет абитуриента (результат поиска)

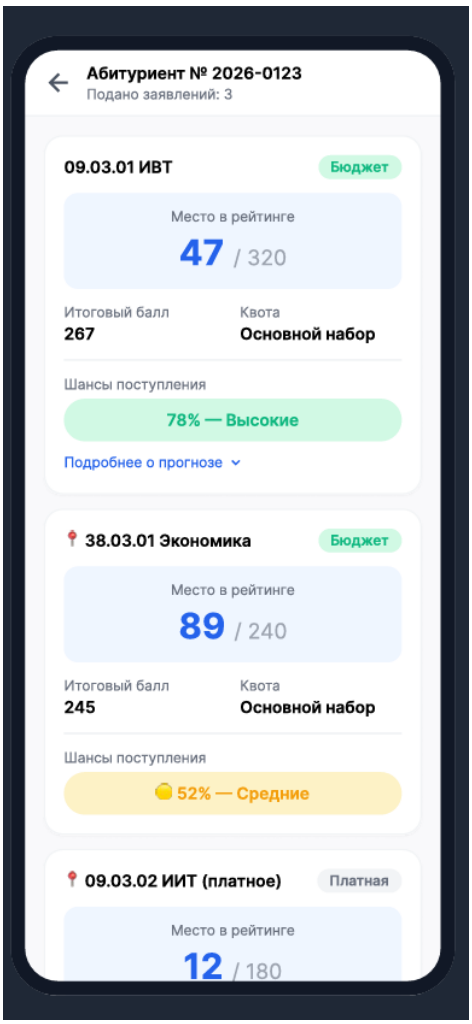


Рисунок 3.9 — Личный кабинет абитуриента Сайта 2 (исходный прототип)

Заголовок: «Абитуриент № [номер]. Подано заявлений: [число]».

Карточки направлений: для каждого поданного направления отображается:

Место в рейтинге (например, «47 / 320»).

Итоговый балл и категория квоты.

Прогноз поступления: цветной бейдж (🟢 78% — Высокие шансы, 🟡 52% — Средние шансы, 🔴 Низкие шансы).

Кнопка «Подробнее о прогнозе».

Поясняющий текст: «Место в рейтинге рассчитывается на основе суммы баллов ЕГЭ и ИД. Прогноз шансов строится с помощью ИИ-модели, учитывающей исторические данные».

— Обезличенный рейтинг

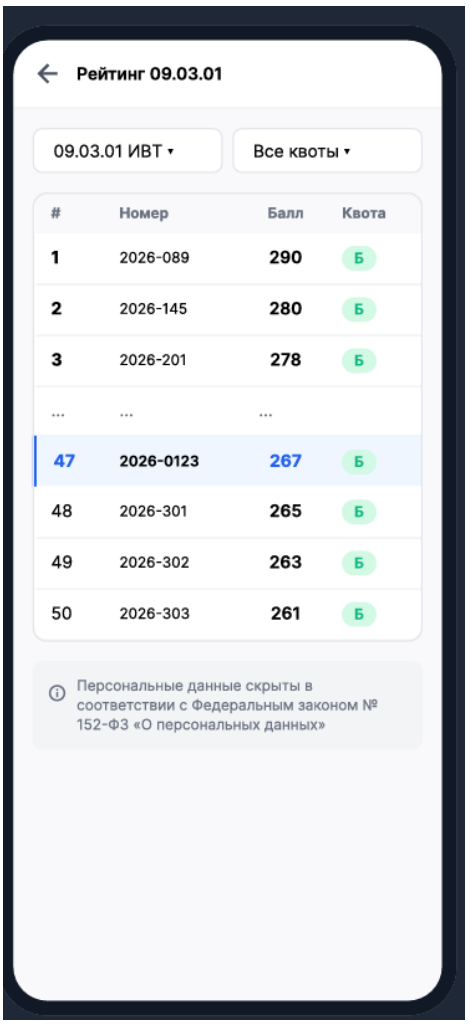


Рисунок 3.10 — Обезличенный рейтинг Сайта 2 (исходный прототип)

Фильтры: выбор направления и квоты.

Выделение: строка с номером текущего абитуриента подсвечена. Выделена «Граница конкурса».

Сноска: «Защита персональных данных: В соответствии с Федеральным законом № 152-ФЗ, ФИО и другие личные данные скрыты. Отображается только уникальный номер заявления».

— ИИ-калькулятор прогноза

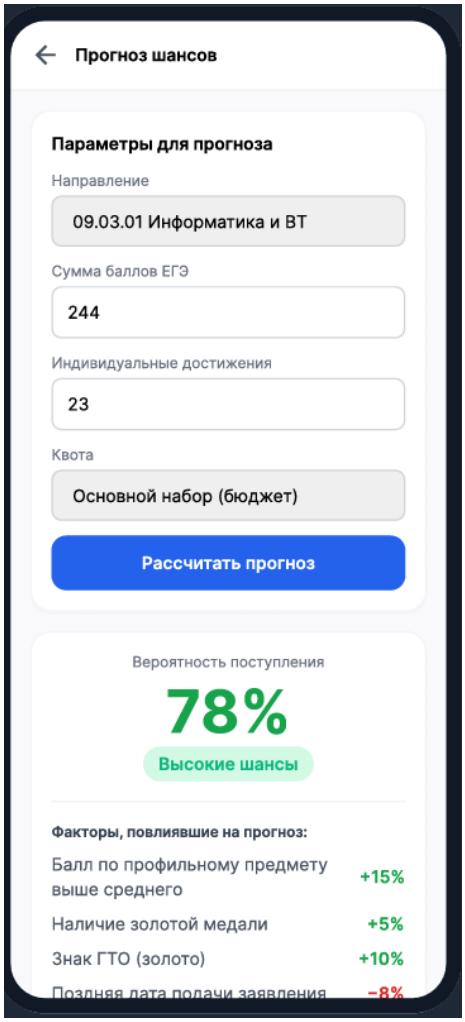


Рисунок 3.11 — ИИ-калькулятор прогноза Сайта 2 (исходный прототип)

Форма ввода: поля «Направление», «Сумма баллов ЕГЭ», «Индивидуальные достижения», «Категория квоты». Кнопка «Рассчитать прогноз».

Результат: крупная цифра вероятности поступления (например, 78%).

Факторы влияния: список с пояснениями (например: «Балл по профильному предмету выше среднего +15%», «Наличие золотой медали +5%», «Поздняя дата подачи заявления −8%»).

График: визуализация «Динамика шансов за 7 дней».

— Справочный центр и чат-бот

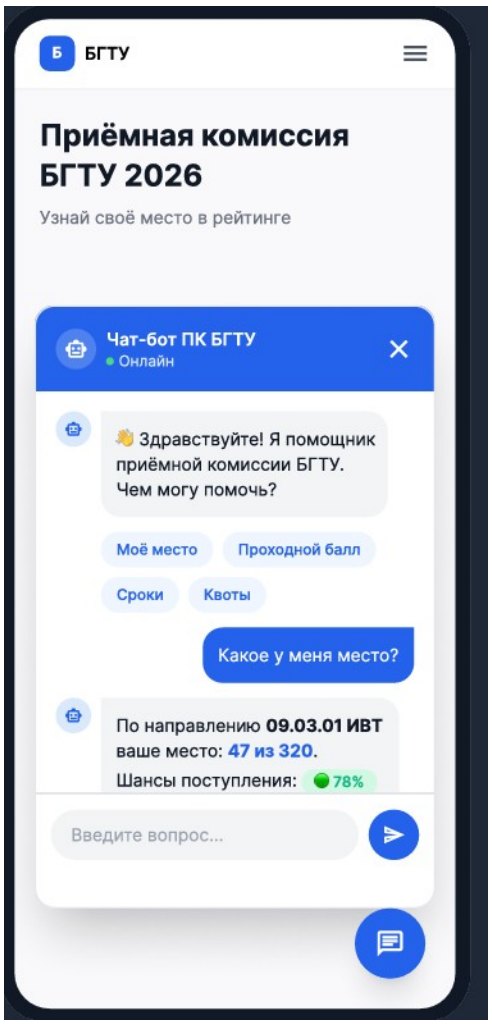


Рисунок 3.12 — Справочный центр и чат-бот Сайта 2 (исходный прототип)

База знаний: список популярных вопросов с развёрнутыми ответами (о сроках, проходных баллах, квотах).

Виджет чат-бота: всплывающее окно «Виртуальный помощник ПК (Онлайн)».

Быстрые кнопки: «Моё место в рейтинге», «Проходной балл», «Сроки подачи», «Общежитие».

Интерфейс диалога: история сообщений и поле ввода «Введите ваш вопрос...» (реализован как rule-based бот по ключевым словам, без использования платных API).

3.3 Дизайн-система

Дизайн-система задаёт цвета, типографику, размеры полей и состояния компонентов обоих сайтов. Таблицы дополнены визуальным примером, позволяющим сопоставить правила оформления с внешним видом элементов.

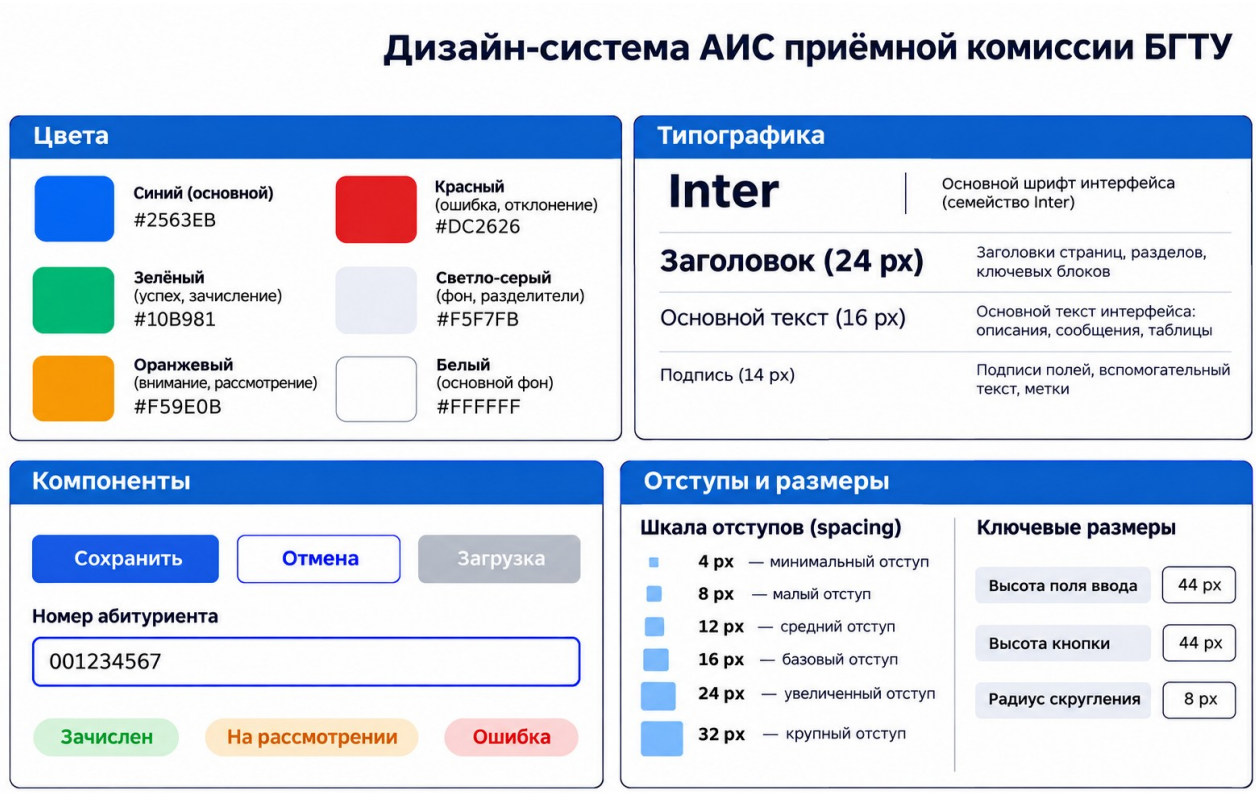


Рисунок 3.13 — Визуальный пример дизайн-системы с увеличенным полем ввода

Проектная иллюстрация компонентов, подготовленная для пояснения правил оформления; не является снимком работающего приложения.

3.3.1 Цветовые токены интерфейса

Для интерфейса АИС Приёмной комиссии используются цветовые токены, сформированные по макетам Figma. Каждому токену соответствует постоянное назначение и конкретный цвет. В макетах используется светлая визуальная схема: светло-серый фон страницы, белые поверхности, синие акценты и пастельные информационные блоки.

Таблица 3.1 — Цветовые токены исходного прототипа

Токен	Назначение	HEX
--bg-page	Основной фон страницы	#F9FAFB
--bg-surface	Карточки, формы, таблицы	#FFFFFF
--bg-sidebar	Боковая панель навигации	#F3F4F6
--text-primary	Основной текст и заголовки	#111827

Токен	Назначение	HEX
--text-muted	Подписи и вспомогательный текст	#6B7280
--border-default	Границы полей и карточек	#E5E7EB
--accent-primary	Основные кнопки и активные элементы	#2563EB
--accent-soft	Мягкое выделение выбранных элементов	#DBEAFE
--status-success	Позитивные статусы («Оплачен», «Высокие шансы»)	#10B981
--status-warning	Предупреждения («Ожидает оплаты», «Средние шансы»)	#F59E0B
--status-error	Критические статусы («Отказ», «Переполнено»)	#DC2626

Обоснование выбора палитры: синий основной цвет (#2563EB) ассоциируется с надёжностью и официальностью государственного вуза. Зелёный (#10B981) используется для позитивных действий и статусов. Оранжевый (#F59E0B) — для предупреждений. Красный (#DC2626) — для критических статусов. Нейтральные оттенки обеспечивают достаточный контраст для работы в условиях приёмной комиссии, где освещение может быть неидеальным.

3.3.2 Типографическая шкала

Для веб-интерфейса АИС Приёмной комиссии закреплён шрифт Inter с системным резервом system-ui, sans-serif. Размер и начертание текста зависят от его роли: названия страниц и ключевые показатели имеют повышенный визуальный приоритет, а подписи, категории и служебный текст отображаются меньшим кеглем.

Таблица 3.2 — Типографическая шкала исходного прототипа

Стиль	Размер	Начертание	Применение
Page Title	24 px	Semibold 600	Название страницы: «Реестр», «Договоры»
Heading	20 px	Semibold 600	Заголовки блоков и модальных окон
Body/L	16 px	Regular 400	Основной текст и пояснения
Body/M	14 px	Regular 400	Поля, подписи, навигация, данные таблицы

Стиль	Размер	Начертание	Применение
Caption	12 px	Regular 400	Бейджи, даты, подсказки, служебный текст
Button	14 px	Semibold 600	Текст кнопок

Интерлиньяж 1.5 обеспечивает хорошую читаемость таблиц и форм при длительной работе секретарей.

3.3.3 Размеры и радиусы компонентов

Для компонентов используется единая шкала отступов 4, 8, 12, 16, 24 и 32 px. Она применяется на всех страницах и обеспечивает одинаковый визуальный ритм.

В уточнённом проектном варианте поля ввода увеличены: минимальная высота — 44 px вместо 40 px, внутренние отступы — 12 px по горизонтали и 10 px по вертикали, основной текст — 16 px. Для многострочного комментария предусмотрена высота от 120 px.

На широком экране рекомендуемая ширина поля поиска составляет не менее 320 px, поля ФИО и номера договора — не менее 280 px. На мобильном экране поле занимает доступную ширину контейнера; длинные подписи переносятся. Сообщение об ошибке размещается под полем. Эти размеры описывают исправленный проект интерфейса, а не изменение исходного кода приложения.

Таблица 3.3 — Уточнённые размеры компонентов

Элемент	Значение	Применение
Радиус малый	4 px	Бейджи статусов, чипы фильтров

Продолжение таблицы 3.3

Элемент	Значение	Применение
Радиус средний	8 px	Кнопки, поля ввода
Радиус большой	8 px	Карточки абитуриентов, договоры
Радиус модального окна	12 px	Окна подтверждения и генерации
Высота кнопки	44 px	Основные действия
Минимальная высота поля	44 px	Однострочные поля поиска и ввода
Отступ карточки	16 px	Содержимое карточек
Размер иконок	16/20/24 px	Навигация, статусы

3.3.4 Состояния интерактивных элементов

Интерактивные элементы АИС Приёмной комиссии имеют набор базовых состояний: Default, Hover, Pressed, Focus, Disabled и Loading. Для выбранного элемента используется состояние Selected. Для ошибок формы применяется состояние Error.

Таблица 3.4 — Состояния интерактивных компонентов

Компонент	Состояния	Оформление
Основная кнопка	Default, Hover, Pressed, Focus, Disabled, Loading	Фон #2563EB, текст #FFFFFF
Второстепенная кнопка	Default, Hover, Pressed, Focus, Disabled	Белый фон, синяя рамка
Опасное действие	Default, Hover, Pressed, Focus, Disabled	Красный фон #DC2626
Поле ввода	Default, Focus, Filled, Error, Disabled	Фон #FFFFFF, граница #E5E7EB; Focus — #2563EB
Пункт навигации	Default, Hover, Active	Active: фон #DBEAFE, синий текст

Продолжение таблицы 3.4

Компонент	Состояния	Оформление
Бейдж статуса	Default, Success, Warning, Error	Пастельные оттенки + иконка
Карточка договора	Default, Hover, Selected	Selected: синяя рамка #2563EB

Состояние Disabled снижает контраст элемента и запрещает действие. Loading блокирует повторное нажатие и показывает ход операции, например при генерации договора или расчёте рейтинга. Focus должен быть заметен при работе с клавиатурой.

3.3.5 Навигация

Навигация выполнена в светлой серой гамме. Слева располагается боковое меню с разделами «Главная», «Реестр», «Договоры», «Обзвон», «Рейтинги», «Отчёты», «Архив». Активный пункт выделяется светло-синей подложкой и синим акцентом. Центральная область использует светло-серый фон, а основные карточки и формы располагаются на белых поверхностях.

Для Сайта 2 используется верхнее меню с гамбургером для мобильных устройств, что обеспечивает удобство работы на смартфонах.

3.4 Интерактивный прототип

Ссылка на прототип в Figma:

https://www.figma.com/design/[ВАША_ССЫЛКА]/AIS-Priemnaya-Komissiya-BGTU

(вставить реальную ссылку или оставить плейсхолдер)

3.4.1 Основные пользовательские сценарии (User Flow)

Основные пользовательские сценарии (User Flow):

Сценарий 1. Загрузка данных и обновление реестра (Сайт 1):

Вход в систему → Главная панель → нажатие «Загрузить Excel» → выбор файла выгрузки из ЕПГУ → автоматическая валидация и сопоставление по номеру абитуриента → просмотр отчёта об успешной загрузке → переход в «Реестр» для проверки новых записей.

Сценарий 2. Работа с договором и подтверждение оплаты (Сайт 1):

Реестр → открытие «Карточки абитуриента» → нажатие «Сгенерировать docx» → печать → последовательное изменение селекторов «ГДЕ?» и «НА КАКОМ ЭТАПЕ?» по мере движения документа → после возврата договора секретарь открывает Excel-выгрузку из бухгалтерии, сверяет номер договора → ставит чекбокс «Оплачен» → статус автоматически меняется на «Оплачен».

Сценарий 3. Приоритизированный обзвон платников (Сайт 1):

Раздел «Звонки» → просмотр списка, отсортированного по колонке «ИИ-прогноз» (от высокого к низкому) → совершение звонка → простановка чекбокса в колонке «Результат звонка» (например, «✅ Дозвон») → переход к следующему абитуриенту в списке.

Сценарий 4. Самопроверка абитуриента (Сайт 2):

Открытие Сайта 2 на смартфоне → ввод номера абитуриента из Госуслуг → просмотр «Личного кабинета» с местом в рейтинге и цветным бейджем прогноза → нажатие «Подробнее о прогнозе» для изучения факторов влияния → при возникновении вопроса нажатие на виджет чат-бота и выбор быстрой кнопки «Проходной балл».

Особенности прототипа:

Полная адаптивность Сайта 2 (корректное отображение на экранах от 320 px до 1920 px).

Реализованы состояния ошибок (например, «Номер абитуриента не найден, проверьте правильность ввода»).

В карточке договора явно разделены логика местоположения («ГДЕ?») и логика этапа («НА КАКОМ ЭТАПЕ?»), что устраняет путаницу при работе с бумажными носителями.

3.4.2 Особенности прототипа

Полная адаптивность Сайта 2 (корректное отображение на экранах от 320 px до 1920 px).

Реализованы состояния ошибок (например, «Номер абитуриента не найден, проверьте правильность ввода»).

В карточке договора явно разделены логика местоположения («ГДЕ?») и логика этапа («НА КАКОМ ЭТАПЕ?»), что устраняет путаницу при работе с бумажными носителями.

Отсутствие функционала загрузки файлов чеков заменено на текстовую подсказку о сверке с Excel, что соответствует реальным бизнес-процессам и снижает нагрузку на сервер.

Обезличенные данные на Сайте 2 — отображается только номер абитуриента, баллы и квота, без ФИО, паспорта и СНИЛС, что соответствует требованиям 152-ФЗ.

ИИ-прогноз с объяснением факторов — абитуриент видит не только вероятность, но и пояснение, какие факторы повлияли на прогноз.

3.4.3 Интерактивные элементы прототипа

Кликабельные пункты боковой навигации.

Работающие переходы между экранами.

Состояния выбранного тарифа.

Пустые и заполненные состояния.

Ошибка авторизации.

Состояния кнопок Default, Hover, Pressed, Focus, Disabled и Loading определены в дизайн-системе.

3.5 Сопоставление проектного прототипа и реализации

Таблица 3.5 — Эволюция интерфейса

Область	Исходный прототип	Реализация 1
Сайт 1	Дашборд и рабочие разделы	React-приложение, поиск, импорт, реестр, договоры, звонки, отчёты.
Сайт 2	Поиск, рейтинг и ИИ-калькулятор	Отдельный HTML-вход /abiturient/, открытый каталог и объяснение положения.

Область	Исходный прототип	Реализация 1
Оценка шансов	Проценты и динамика в макете	Словесная оценка по плану мест; probability=null.
Источник данных	Предполагается единый реестр	Одна SQLite-база и общий FastAPI для двух сайтов.
Демо	Статические примерные значения	Явно обозначенное демо; настоящий режим запрашивает реальные открытые данные.
Стилизация портала	Проектная дизайн-система	Самостоятельная университетская стилизация; точное совпадение с внешним референсом не проверено из-за сетевого ограничения.

Ссылка на Figma сохранена из исходного материала и приведена в списке источников. Наличие скриншотов подтверждает разработку проектных экранов, но не подтверждает работу каждого изображённого элемента в программной реализации.

4 Модель жизненного цикла и паттерны проектирования

4.1 Итерационная модель разработки

Итерационная организация разработки позволяет проверять требования на каждом завершённом приращении и уточнять решение по результатам обратной связи [5].

Для курсового проекта выбран итерационный процесс с небольшими законченными инкрементами. После реализации ядра добавлены внутренний сайт, затем открытый портал. Каждый инкремент проверяется существующими тестами и отдельными сценариями браузера. Такой порядок позволяет уточнять требования на работающем приложении.

Scrum в первоначальном плане используется как модель организации работ: backlog, планирование спринта, обзор результата и ретроспектива. Для индивидуальной учебной работы эти роли могут совмещаться. Отчёт не утверждает, что проведён полный формальный Scrum-процесс с отдельным Scrum Master и регулярными встречами команды.

Каждая итерация завершается проверкой не только отдельной функции, но и связанного пользовательского пути. Например, успешная загрузка файла должна отражаться в реестре, рейтинге и разрешённой публичной выдаче. Если изменяется структура данных, повторно проверяются импорт и ограничения целостности. При изменении интерфейса необходимы сборка и просмотр основных экранов. Обнаруженные ограничения фиксируются вместе с результатом проверки и становятся входом для следующего приращения. Такой порядок позволяет отделять подтверждённое поведение от проектных намерений и не считать завершённой возможность, которая существует только в макете или описании.

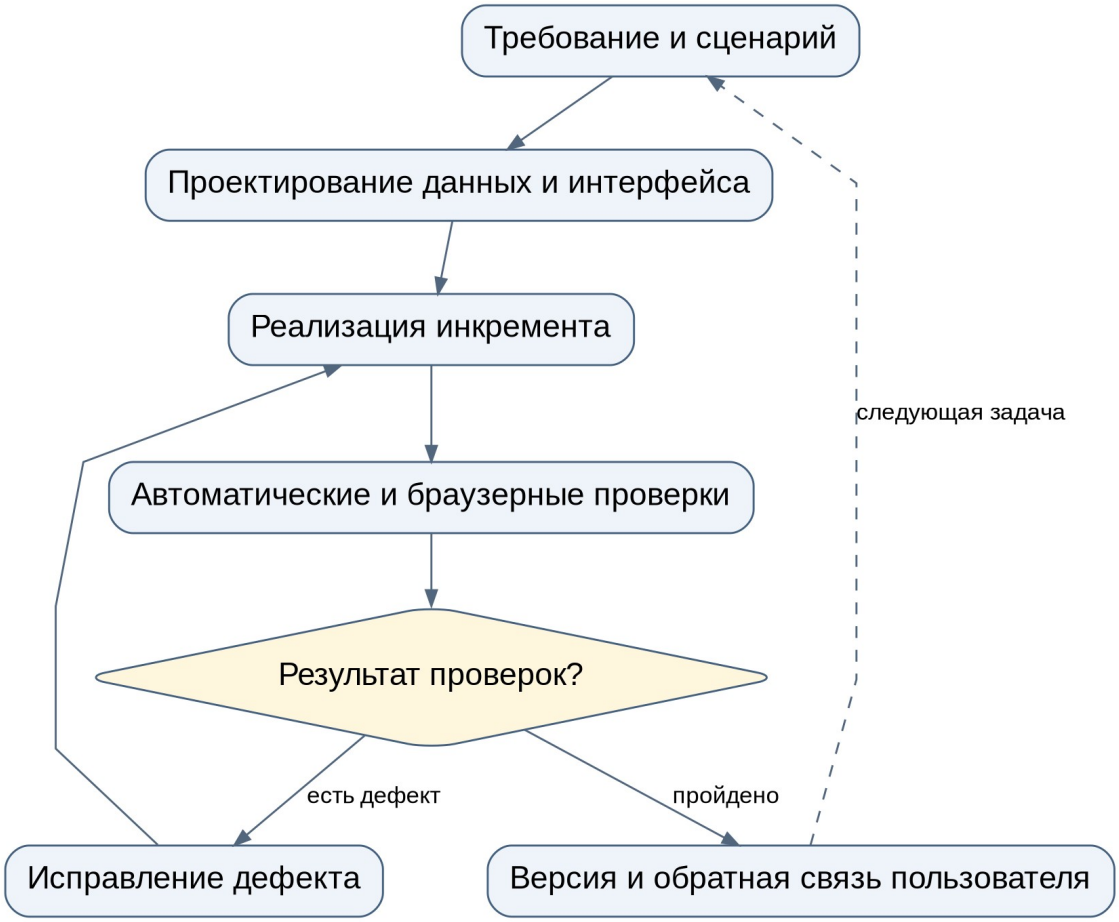


Рисунок 4.1 — Итерационный цикл разработки и проверки

4.2 Паттерны и фактические способы разделения ответственности

Таблица 4.1 — Проектные решения и их применение

Решение	Состояние	Привязка к коду
Слоистая архитектура	Применена	React → API → сервисные функции / импорт → SQLAlchemy → SQLite.
Unit of Work	Применён	Database.uow(): sessions.begin() задаёт общую транзакцию и откат при исключении.

Продолжение таблицы 4.1

Решение	Состояние	Привязка к коду
Adapter / сопоставление форматов	Применено функционально	ALIASES и REVERSE приводят разные названия колонок к единой внутренней модели.
MVC	Концептуальное разделение	Роль представления выполняет React; HTTP-контроллеры находятся в api.py и web.py. Классического серверного MVC-фреймворка нет.
Repository	Отдельные классы не введены	Доступ к данным выполняется параметризованными выражениями SQLAlchemy внутри сервисов и API.
Factory Method	Фабрика приложения	create_app собирает зависимости. Это функция создания приложения, а не заявленная иерархия GoF Factory Method.
Observer	Предоставляется React	State/Context и обновление запросов через revision; отдельной серверной шины событий нет.

Выбор процедурных сервисов сокращает объём инфраструктурного кода MVP. При усложнении прав доступа, транспортов обмена и алгоритмов прогнозирования целесообразно выделить репозитории и интерфейсы стратегий, сохранив существующие контрактные тесты.

4.3 Управление изменениями

Изменение приложения начинается с уточнения пользовательского сценария и модели данных. Проверки повторного импорта, публичной сериализации и ограничений доступа служат регрессионной основой. Выпуск сопровождается обновлением документации, сборкой обоих HTML-входов и упаковкой исходников без рабочих баз и персональной выгрузки.

5 Планирование разработки

Планирование ниже является учебной моделью. Первичные оценки эпиков, детализация MVP и полный план развития имеют разный охват; фактический состав команды и продолжительность внедрения ими не подтверждаются.

5.1 Эпики (Epics)

В рамках проекта выделено пять эпиков, каждый из которых представляет собой крупный блок функциональности. Эпики упорядочены по приоритету и распределены по спринтам с учётом зависимостей между модулями. Общий срок разработки составляет 8 недель (2 месяца), что соответствует 4 спринтам по 2 недели.

Epic 1: Базовый функционал системы

Описание: основные компоненты системы: аутентификация через python-jose (JWT) и passlib (bcrypt), навигация, базовые CRUD-операции через SQLAlchemy 2.0, настройка инфраструктуры (SQLite), загрузка данных из ЕПГУ через openpyxl/pandas.

Пользовательские истории:

— US-1: Аутентификация пользователей Сайта 1 (логин/пароль, JWT, bcrypt).

— US-2: Загрузка Excel-выгрузки из ЕПГУ с семантическим сопоставлением по номеру абитуриента.

— US-3: Ведение реестра абитуриентов (CRUD, поиск, фильтрация, карточка с вкладками).

Story Points: 21.

Приоритет: высокий.

Спринт: 1–2.

Epic 2: Конкурсные списки и рейтинг

Описание: расчёт итогового балла (ЕГЭ + ИД), распределение по пяти квотам, формирование конкурсных списков, учёт внутренних экзаменов СПО, экспорт в Excel/PDF через openpyxl и reportlab.

Пользовательские истории:

— US-4: Расчёт рейтинга (сумма баллов ЕГЭ и индивидуальных достижений).

— US-5: Формирование пяти конкурсных списков по каждому направлению и квоте.

— US-6: Учёт результатов внутренних экзаменов для абитуриентов СПО.

Story Points: 26.

Приоритет: высокий.

Спринт: 2–3.

Epic 3: Договоры и обзвон

Описание: жизненный цикл договора (два вида статусов «ГДЕ?» и «НА КАКОМ ЭТАПЕ?»), генерация договора по шаблону Word через библиотеку python-docx, подтверждение оплаты по Excel-выгрузке из бухгалтерии (галочка «Оплачен»), формирование списков для обзвона, ИИ-прогноз вероятности заключения договора через Яндекс GPT / DeepSeek.

Пользовательские истории:

— US-7: Создание договора по шаблону Word с автозаполнением данными абитуриента.

— US-8: Отслеживание статусов договора («ГДЕ?» и «НА КАКОМ ЭТАПЕ?»), история изменений.

— US-9: Подтверждение оплаты по Excel-выгрузке из бухгалтерии (галочка «Оплачен»).

— US-10: Формирование списков для обзвона платников с ИИ-приоритизацией.

— US-11: ИИ-прогноз вероятности заключения договора (Яндекс GPT / DeepSeek).

Story Points: 47.

Приоритет: высокий.

Спринт: 2–4.

Epic 4: Сайт для абитуриентов + ИИ + чат-бот

Описание: открытый сайт для абитуриентов (поиск по номеру, рейтинг, прогноз шансов), ИИ-модуль прогнозирования шансов поступления (Яндекс GPT /

DeepSeek), скриптовый чат-бот (rule-based) для типовых вопросов без использования платных API.

Пользовательские истории:

— US-12: Поиск абитуриентом своего места в рейтинге по номеру (Сайт 2, без авторизации, 152-ФЗ).

— US-13: Просмотр обезличенного рейтинга по любому направлению.

— US-14: ИИ-прогноз шансов поступления с объяснением факторов.

— US-15: Скриптовый чат-бот для типовых вопросов (rule-based, база знаний FAQ).

Story Points: 31.

Приоритет: средний.

Спринт: 3–4.

Epic 5: Отчётность, архивация и интеграция с 1С

Описание: формирование отчётов (Excel через openpyxl, PDF через reportlab), статистика переполненных направлений, архивация данных на 6 лет, выгрузка в 1С после приказа о зачислении в формате JSON/XML.

Пользовательские истории:

— US-16: Формирование рейтинговых списков и ведомости для бухгалтерии.

— US-17: Статистика приёма и переполненных направлений.

— US-18: Архивация данных за прошлые годы с поиском по ФИО, году, номеру договора.

— US-19: Выгрузка данных в 1С после приказа о зачислении.

— US-20: Продуктовые события и аналитика (таблица analytics_events в SQLite).

Story Points: 34.

Приоритет: средний.

Спринт: 3–4.

Сводная таблица эпиков:

Таблица 5.1 — Первичная оценка эпиков до детализации

ID	Название эпика	Story Points	Приоритет	Спринт
Epic 1	Базовый функционал системы	21	Высокий	1–2
Epic 2	Конкурсные списки и рейтинг	26	Высокий	2–3
Epic 3	Договоры и обзвон	47	Высокий	2–4
Epic 4	Сайт абитуриента + ИИ + чат-бот	31	Средний	3–4
Epic 5	Отчётность, архивация, 1С, аналитика	34	Средний	3–4
	Итого	159		

159 SP — первичная укрупнённая оценка. После детализации задач получено 94 SP. Оценки относятся к разным уровням декомпозиции и не суммируются.

5.2 Декомпозиция эпиков

5.2.1 Декомпозиция Epic 1: Базовый функционал системы

Подзадача 1.1: Реализация аутентификации и инфраструктуры (US-1)

Настройка FastAPI и подключение SQLAlchemy 2.0 к SQLite.

Реализация хеширования паролей через passlib (bcrypt).

Настройка JWT-аутентификации через python-jose.

Создание базовых моделей пользователей.

Написание unit-тестов для AuthService через pytest.

Реализация logout и управления сессией.

Исполнитель: Backend-разработчик.

Подзадача 1.2: Реализация загрузки Excel-выгрузки из ЕПГУ (US-2)

Создать форму загрузки файла на Сайте 1 (React + TypeScript).

Реализовать гибкий парсинг Excel-файла через openpyxl/pandas с семантическим сопоставлением столбцов.

Разработать алгоритм сопоставления записей по номеру абитуриента.

Реализовать обработку ошибок и формирование отчёта о загрузке.

Настроить транзакции через Unit of Work.

Исполнитель: Backend-разработчик.

Подзадача 1.3: Создание модуля реестра абитуриентов (US-3)

Спроектировать сущность Abiturient и связанные таблицы в SQLAlchemy.

Реализовать CRUD-операции через AbiturientRepository.

Разработать форму поиска и фильтрации на React.

Создать карточку абитуриента с вкладками.

Реализовать валидацию данных через Pydantic.

Исполнитель: Fullstack-разработчик.

5.2.2 Декомпозиция Epic 2: Конкурсные списки и рейтинг

Подзадача 2.1: Реализация расчёта рейтинга и формирования списков (US-4, US-5, US-6)

Разработать алгоритм расчёта итогового балла (ЕГЭ + ИД).

Реализовать распределение по пяти квотам.

Разработать алгоритм сортировки и формирования пяти списков.

Создать интерфейс просмотра конкурсных списков на React.

Реализовать экспорт списков в Excel (openpyxl) и PDF (reportlab).

Исполнитель: Backend-разработчик + Frontend-разработчик.

5.2.3 Декомпозиция Epic 3: Договоры и обзвон

Подзадача 3.1: Реализация жизненного цикла договора (US-7, US-8)

Создать шаблон договора в MS Word (юрист).

Реализовать автозаполнение договора данными абитуриента через python-docx.

Разработать систему двух видов статусов («ГДЕ?» и «НА КАКОМ ЭТАПЕ?»).

Создать интерфейс управления договорами на React.

Реализовать историю изменений статусов.

Исполнитель: Fullstack-разработчик.

Подзадача 3.2: Реализация подтверждения оплаты и списков для обзвона (US-9, US-10)

Разработать интерфейс отметки оплаты (галочка «Оплачен»).

Реализовать загрузку Excel-выгрузки из бухгалтерии для сверки через openpyxl.

Разработать алгоритм формирования списка для обзвона (по датам, статусам).

Реализовать интерфейс списка с чекбоксами результатов на React.

Добавить фильтры и сортировку.

Реализовать экспорт в Excel.

Исполнитель: Fullstack-разработчик + Frontend-разработчик.

Подзадача 3.3: Разработка ИИ-модуля прогнозирования (US-11)

Собрать и подготовить исторические данные за 3–5 лет.

Интегрировать API Яндекс GPT / DeepSeek через httpx.

Реализовать API для получения прогноза.

Настроить ежедневный пересчёт прогноза через BackgroundTasks.

Реализовать объяснение прогноза через промпт-инжиниринг.

Исполнитель: Backend-разработчик.

5.2.4 Декомпозиция Epic 4: Сайт для абитуриентов + ИИ + чат-бот

Подзадача 4.1: Разработка Сайта 2 для абитуриентов (US-12, US-13)

Создать главную страницу с поиском по номеру (React, адаптив).

Реализовать страницу результата поиска (личный кабинет).

Разработать страницу обезличенного рейтинга с соблюдением 152-ФЗ.

Реализовать адаптивную вёрстку для смартфонов (375–1920 px).

Исполнитель: Frontend-разработчик.

Подзадача 4.2: Разработка ИИ-прогноза шансов поступления (US-14)

Интегрировать API Яндекс GPT / DeepSeek для расчёта шансов.

Реализовать объяснение факторов влияния через промпт-инжиниринг.

Настроить кэширование прогнозов в SQLite.

Реализовать интерфейс отображения прогноза на Сайте 2.

Исполнитель: Backend-разработчик + Frontend-разработчик.

Подзадача 4.3: Разработка скриптового чат-бота (US-15)

Разработать базу знаний (FAQ приёмной комиссии) в JSON/БД.

Реализовать rule-based логику (поиск ответа по ключевым словам).

Интегрировать чат-бот в Сайт 2 (React-виджет).

Добавить контекстные подсказки и быстрые кнопки.

Исполнитель: Frontend-разработчик.

5.2.5 Декомпозиция Epic 5: Отчётность, архивация и интеграция с 1С

Подзадача 5.1: Реализация отчётности, архивации и интеграции с 1С (US-16, US-17, US-18, US-19)

Разработать генератор отчётов (openpyxl, reportlab).

Реализовать статистику переполненных направлений.

Разработать ведомость для бухгалтерии.

Реализовать архивацию данных на 6 лет.

Согласовать формат выгрузки с администратором 1С.

Реализовать формирование JSON-файла для 1С.

Исполнитель: Backend-разработчик.

Подзадача 5.2: Продуктовая аналитика и события (US-20)

Спроектировать таблицу analytics_events в SQLite.

Реализовать запись событий на каждом шаге воронок.

Настроить SQL-представления для сегментного анализа.

Реализовать дашборд ключевых метрик для Директора ПК.

Исполнитель: Backend-разработчик + Fullstack-разработчик.

5.3 Sprint Backlog

Разработка ведётся итеративно. Общий срок проекта составляет 8 недель (2 месяца), что соответствует 4 спринтам по 2 недели.

Таблица 5.2 — Модельный Sprint Backlog первого спринта

Задача	Story Points	Статус	Исполнитель
US-1: Аутентификация и настройка инфраструктуры	5	В плане	Backend
US-2: Загрузка Excel-выгрузки из ЕПГУ	8	В плане	Backend
US-3: Реестр абитуриентов (CRUD, поиск, карточка)	8	В плане	Fullstack
Итого	21		

Таблица 5.3 — Модельный Sprint Backlog второго спринта

Задача	Story Points	Статус	Исполнитель
US-4, US-5, US-6: Расчёт рейтинга и 5 конкурсных списков	13	В плане	Backend
US-7, US-8: Создание договора (python-docx) и статусы («ГДЕ?», «ЭТАП»)	13	В плане	Fullstack
Итого	26		

Таблица 5.4 — Модельный Sprint Backlog третьего спринта

Задача	Story Points	Статус	Исполнитель
US-12, US-13: Сайт 2 (поиск, личный кабинет, обезличенный рейтинг)	8	В плане	Frontend
US-9, US-10: Подтверждение оплаты и списки для обзвона	8	В плане	Frontend

Задача	Story Points	Статус	Исполнитель
US-11: ИИ-прогноз вероятности заключения договора (Яндекс GPT / DeepSeek)	13	В плане	Backend
Итого	29		

Таблица 5.5 — Модельный Sprint Backlog четвёртого спринта

Задача	Story Points	Статус	Исполнитель
US-14: ИИ-прогноз шансов поступления (Яндекс GPT / DeepSeek)	13	В плане	Backend
US-15: Скриптовый чат-бот (FAQ)	5	В плане	Frontend
US-16, US-17, US-18, US-19: Отчётность, архивация, выгрузка в 1С	13	В плане	Backend
US-20: Продуктовые события и аналитика	8	В плане	Backend + Fullstack
Тестирование (pytest, интеграционное) и документация	8	В плане	Тестировщик
Итого	47		

5.4 Оценка сложности разработки

Для относительной оценки сложности применяется Planning Poker и последовательность Фибоначчи (1, 2, 3, 5, 8, 13, 21). Story Points отражают сложность и неопределённость задачи, а не время её выполнения. Пересчёт в часы не используется.

Таблица 5.6 — Относительная сложность задач

Задача	Сложность (SP)	Комментарии
Аутентификация и инфраструктура (FastAPI, SQLAlchemy, SQLite)	5	Настройка БД, JWT, базовые сервисы
Загрузка Excel-выгрузки из ЕПГУ (openpyxl/pandas)	5	Гибкий парсер, сопоставление, обработка ошибок
Реестр абитуриентов и карточка	8	CRUD, поиск, фильтрация, вкладки
Расчёт рейтинга и 5 списков (openpyxl, reportlab)	8	Логика квот, сортировка, экспорт
Жизненный цикл договора (python-docx, статусы)	13	Два вида статусов, автозаполнение, история
Подтверждение оплаты и списки обзвона	8	Галочка «Оплачен», чекбоксы звонков
Сайт 2 для абитуриентов (адаптив, 152-ФЗ)	8	4 экрана, мобильная версия
ИИ-модуль (Яндекс GPT / DeepSeek)	13	Интеграция API, промпт-инжиниринг, кэш
Скриптовый чат-бот (rule-based)	5	База знаний FAQ, виджет

Продолжение таблицы 5.6

Задача	Сложность (SP)	Комментарии
Отчётность, архивация и интеграция с 1С	8	Excel/PDF, архив, JSON для 1С
Продуктовая аналитика и события	5	Таблица analytics_events, дашборд
Тестирование (pytest) и документация	8	Unit-тесты, руководство пользователя
Итого	94	

Таблица 5.7 — Распределение ответственности

Роль	Задачи
Backend-разработчик	Архитектура, аутентификация, Excel, рейтинг, отчёты, 1С, ИИ, аналитика
Frontend-разработчик	Сайт 2, обзвон, чат-бот, адаптивная вёрстка
Fullstack-разработчик	Реестр, карточки, договоры, экзамены СПО
Тестировщик / QA	pytest, интеграционные тесты, документация

Срок разработки: 8 недель (4 спринта по 2 недели) при команде из 4 человек.

Обоснование выбора длительности спринтов:

Двухнедельные спринты выбраны по следующим причинам:

Ограниченный срок приёмной кампании. Приёмная кампания длится около 3 месяцев (июнь–август), поэтому необходимо успеть внедрить систему до пиковой нагрузки.

Регулярная обратная связь. Двухнедельный цикл позволяет Директору ПК (Product Owner) видеть прогресс и корректировать приоритеты.

Удобство для учебного процесса. Двухнедельные спринты совпадают с учебными модулями, что упрощает контроль со стороны преподавателя.

Оптимальный баланс. Двухнедельный спринт — это баланс между скоростью обратной связи и объёмом выполнимой работы для команды из 4 человек.

5.5 Фактические инкременты и контрольные результаты

Таблица 5.8 — Инкременты репозитория

Инкремент	Результат	Контроль
Ядро, коммит 6ff00a7	Python API, SQLite, импорт и предметные сервисы	Обработка примера: 3 506 абитуриентов, 19 610 заявлений; первоначальные 16 тестов.
Сайт сотрудников, коммит 224d1e0	Внутренний React-интерфейс и служебные представления	21 тест и браузерный сценарий.
Портал, коммит 853721f	Анонимный каталог, поиск и открытые рейтинги	26 тестов и два браузерных сценария.

6 Техническое задание

Техническое задание задаёт целевое состояние системы. Выполнение отдельных пунктов в версии 1 и оставшиеся условия приёмки приведены в конце этой главы.

6.1 Основание для разработки

Разработка программного продукта выполняется в рамках курсового проекта по дисциплине «Технология разработки программного продукта». Основанием для разработки является необходимость создания автоматизированной информационной системы (АИС) приёмной комиссии Брянского государственного технического университета (БГТУ), предназначенной для централизованного учёта абитуриентов, формирования конкурсных списков, сопровождения договоров на платное обучение, обзвона абитуриентов, прогнозирования шансов поступления и формирования отчётности.

Исходными материалами для разработки технического задания являются результаты анализа предметной области, бизнес-требования, функциональные и нефункциональные требования, пользовательские истории, варианты использования, прототип интерфейса в Figma и планирование разработки, представленные в предыдущих разделах курсовой работы.

Заказчик: приёмная комиссия БГТУ. Внедрение осуществляется силами IT-отдела университета в рамках курсового проекта, что исключает затраты на закупку лицензии.

6.2 Назначение разработки

Программный продукт АИС Приёмной комиссии предназначен для автоматизации процессов приёма документов, ведения реестра абитуриентов, формирования конкурсных списков, заключения и сопровождения договоров на платное обучение, обзвона абитуриентов, формирования отчётности и архивации данных.

Система ориентирована на:

— секретарей приёмной комиссии — основных пользователей;

— студентов-практикантов — вспомогательный персонал с тем же уровнем доступа;

— абитуриентов — внешних пользователей Сайта 2 (открытый доступ без авторизации).

Особенность системы: состоит из двух связанных сайтов с общей базой данных:

— Сайт 1 — рабочий инструмент секретарей и практикантов (десктопный интерфейс);

— Сайт 2 — открытый информационный портал для абитуриентов (адаптивный мобильный интерфейс).

6.3 Требования к функциональным характеристикам

В программном обеспечении должны быть реализованы следующие функции:

— Загрузка данных из ЕПГУ:

— загрузка ежедневной Excel-выгрузки из Госуслуг;

— семантическое сопоставление столбцов (гибкий парсер);

— сопоставление записей по номеру абитуриента;

— обновление существующих и добавление новых записей;

— формирование отчёта о загрузке (новые, обновлённые, ошибочные).

— Ведение реестра абитуриентов:

— хранение паспортных данных, СНИЛС, документов об образовании, индивидуальных достижений, льгот, контактных данных, данных родителя;

— добавление, редактирование, поиск и фильтрация записей;

— карточка абитуриента с вкладками.

— Распределение по квотам:

автоматическое отнесение абитуриента к одной из пяти категорий: особая квота (СВО), отдельная квота, целевое направление, основной набор, платное обучение.

— Расчёт рейтинга и формирование списков:

— расчёт итогового балла (сумма ЕГЭ + индивидуальные достижения);

— льготы не влияют на балл, только на квоту;

— формирование пяти конкурсных списков по каждому направлению, сортировка по убыванию балла;

— учёт результатов внутренних экзаменов для СПО.

— Жизненный цикл договора:

— генерация договора по шаблону Word с автозаполнением данными абитуриента;

— отслеживание двух видов статусов: «ГДЕ?» (местоположение) и «НА КАКОМ ЭТАПЕ?» (этап подписания);

— подтверждение оплаты по Excel-выгрузке из бухгалтерии (галочка «Оплачен»);

— история изменений статусов.

— Списки для обзвона:

— автоматическое формирование списков платников в последнюю неделю изменения направления и за неделю до закрытия оплаты;

— приоритизация по вероятности заключения договора (ИИ-прогноз);

— отметка результатов звонка.

— Сайт для абитуриентов (Сайт 2):

— поиск по номеру абитуриента (уникален по РФ, из Госуслуг);

— просмотр своего места в рейтинге;

— просмотр обезличенных рейтингов по любому направлению;

— прогноз шансов поступления (ИИ);

— чат-бот для типовых вопросов;

— персональные данные (ФИО, паспорт, СНИЛС) НЕ отображаются (152-ФЗ).

— Отчётность и статистика:

— рейтинговые списки по направлениям и квотам;

— статистика приёма (подано, принято, зачислено);

— статистика переполненных направлений;

— ведомость для бухгалтерии;

— экспорт в Excel и PDF.

— Архивация:

— хранение данных о приёме в течение 6 лет;

— поиск по ФИО, году поступления, номеру договора.

— Выгрузка в 1С:

— формирование пакета данных (JSON/XML) после приказа о зачислении;

— передача: ФИО, паспорт, СНИЛС, направление, форма обучения, дата зачисления, номер договора.

— Администрирование:

— аутентификация пользователей (логин/пароль, JWT, bcrypt);

— ежедневное резервное копирование БД.

6.4 Требования к организации входных и выходных данных

К входным данным системы относятся:

— Excel-файл выгрузки из ЕПГУ;

— данные абитуриента (ФИО, СНИЛС, паспорт, контакты, документы об образовании, ИД, льготы);

— баллы ЕГЭ и результаты внутренних экзаменов;

— шаблон договора в формате .docx;

— Excel-выгрузка из бухгалтерии для подтверждения оплаты;

— номер абитуриента (для Сайта 2);

— текстовый запрос для чат-бота.

К выходным данным системы относятся:

— реестр абитуриентов с карточками;

— пять конкурсных списков по каждому направлению;

— сгенерированный договор в формате .docx;

— списки для обзвона с приоритизацией;

— ИИ-прогноз шансов поступления с объяснением факторов;

— отчёты в форматах Excel и PDF;

— JSON/XML файл для выгрузки в 1С;

— обезличенный рейтинг на Сайте 2.

6.5 Требования к временным характеристикам

— время отклика на стандартные запросы (поиск, фильтрация) — не более 2 секунд;

— загрузка списка абитуриентов (до 10 000 записей) — не более 3 секунд;

— формирование пяти конкурсных списков по одному направлению — не более 5 секунд;

— формирование стандартных отчётов — не более 30 секунд;

— система должна поддерживать одновременную работу до 10 пользователей Сайта 1;

— Сайт 2 должен выдерживать до 500 одновременных запросов от абитуриентов;

— резервное копирование данных — ежедневно в 02:00;

— пересчёт ИИ-прогноза — ежедневно ночью после загрузки Excel-выгрузки.

6.6 Требования к пользовательскому интерфейсу

Сайт 1 (для секретарей):

— тип интерфейса: веб-приложение (браузеры Chrome, Яндекс.Браузер, Firefox);

— поддержка русского языка;

— адаптация под разрешение от 1280×720;

— интуитивная навигация (не более 3 кликов до нужной функции);

— контекстные подсказки для сложных операций;

— интерфейс рассчитан на пользователей с разным уровнем подготовки (включая практикантов, не владеющих Excel).

Сайт 2 (для абитуриентов):

— тип интерфейса: открытый веб-сайт (без авторизации);

— адаптивный дизайн: смартфоны (от 320 px), планшеты, десктопы;

— минималистичный интерфейс, крупные элементы;

— поиск по номеру абитуриента — главный элемент главной страницы.

6.7 Требования к надёжности и безопасности

— пароли пользователей должны храниться только в хэшированном виде (bcrypt через passlib);

— авторизация API должна выполняться с использованием JWT (python-jose);

— Сайт 2 работает без авторизации, но отображает только обезличенные данные (номер абитуриента, баллы, квота, направление);

— персональные данные (ФИО, паспорт, СНИЛС, контакты) на Сайте 2 НЕ отображаются в соответствии с Федеральным законом № 152-ФЗ «О персональных данных»;

— доступ к Сайту 1 — только из локальной сети БГТУ;

— SQL-запросы должны использовать параметризованный ввод (защита от SQL-инъекций);

— защита от XSS-атак на обоих сайтах;

— операции изменения данных должны сохранять согласованность связанных сущностей (Unit of Work);

— секретные ключи и параметры подключения в рабочей среде должны храниться в переменных окружения (.env);

— должна быть предусмотрена возможность резервного копирования и восстановления данных.

6.8 Условия эксплуатации

Требования к квалификации персонала:

— секретари приёмной комиссии: уверенные пользователи ПК, базовые навыки работы в браузере;

— студенты-практиканты: базовые навыки работы со смартфоном и ПК, минимальное знание офисных программ;

— администратор системы: знание ОС Linux/Windows, основ Python и SQL, навыки настройки веб-сервера.

Режим работы:

— основной режим: 24/7 (с ежедневным резервным копированием);

— техническое обслуживание: еженедельно в воскресенье 02:00–04:00;

— пиковая нагрузка: июнь–август (приёмная кампания).

6.9 Требования к техническим средствам

Клиентские рабочие места (Сайт 1):

— процессор: Intel Core i3 / AMD Ryzen 3 или выше;

— ОЗУ: 4 ГБ (рекомендуется 8 ГБ);

— жёсткий диск: 100 ГБ свободного места;

— ОС: Windows 10/11 или Linux;

— браузер: Chrome, Яндекс.Браузер, Firefox.

Серверная часть:

— процессор: Intel Xeon 4 ядра или выше;

— ОЗУ: 16 ГБ (рекомендуется 32 ГБ);

— жёсткий диск: 500 ГБ (SSD рекомендуется);

— ОС: Ubuntu 22.04 LTS или Windows Server 2019;

— размещение: сервер БГТУ в локальной сети.

Для абитуриентов (Сайт 2):

— любое устройство с браузером (смартфон, планшет, ПК);

— доступ: интернет или локальная сеть БГТУ.

6.10 Требования к программным средствам

СУБД: SQLite (лёгкость развёртывания, отсутствие необходимости в отдельном сервере БД, достаточная производительность для 10 000 записей).

Платформа разработки:

Backend: Python 3.11+ и FastAPI. Автоматические проверки выполнены на Python 3.12.14; пользовательский локальный запуск подтверждён на Windows с Python 3.15.1.

— Frontend: React 18 + TypeScript (SPA для Сайта 1 и Сайта 2);

— ORM: SQLAlchemy 2.0. Для текущей схемы используется create_all; внедрение Alembic-миграций предусмотрено как дальнейшее развитие.

Работа с Excel: openpyxl. pandas указан в исходном проектном варианте, но для текущего импортера не требуется.

— Библиотеки для генерации документов: python-docx (договоры в формате .docx), reportlab (PDF-отчёты);

— ИИ-направление развития: возможное подключение внешнего провайдера после подготовки данных и проверки качества. В версии 1 работают детерминированная оценка положения и эвристика очереди обзвона.

Чат-бот: локальный rule-based помощник с базой типовых ответов в модуле services.py.

— Аутентификация: python-jose (JWT-токены), passlib + bcrypt (хеширование паролей);

— Веб-сервер: Uvicorn (ASGI-сервер для FastAPI) + Nginx как reverse proxy (опционально);

— Тестирование: pytest (unit-тесты, интеграционные тесты).

Средства разработки:

— PyCharm / VS Code;

— Git (GitHub / GitLab);

— Контейнеризация Docker — возможный вариант дальнейшего развёртывания; в текущем репозитории рабочий Docker-сценарий не подготовлен.

— Postman / Swagger (тестирование API);

— Figma (прототипирование);

— DB Browser for SQLite — администрирование БД.

Протоколы: TCP/IP, HTTP/HTTPS, REST.

6.11 Требования к программной документации

В состав программной документации должны входить:

— руководство пользователя (пошаговая инструкция для секретарей и практикантов);

— руководство программиста (описание архитектуры, API, структуры БД);

— описание структуры базы данных (ER-диаграмма, описание таблиц);

— описание REST API (автоматически генерируется через Swagger/OpenAPI);

— результаты тестирования и список основных сценариев проверки.

6.12 Этапы разработки

Разработка выполняется итерационно в соответствии с планом спринтов:

Таблица 6.1 — Этапы полного целевого проекта

Этап	Срок	Результат
Планирование и анализ	1 неделя	ТЗ, бизнес-требования
Проектирование архитектуры	1 неделя	Схема архитектуры, ER-диаграмма БД (SQLite)
Разработка базовых модулей	2 недели	Аутентификация, реестр, загрузка Excel
Разработка модулей рейтинга и договоров	2 недели	5 списков, жизненный цикл договора
Разработка Сайта 2 и ИИ-модуля	2 недели	Сайт абитуриента, прогноз, чат-бот
Разработка отчётности и интеграции с 1С	2 недели	Отчёты, архивация, выгрузка в 1С
Тестирование и отладка	1 неделя	Unit (pytest), интеграционные, нагрузочные тесты
Документирование	1 неделя	Руководство пользователя, Swagger API
Внедрение и обучение	1 неделя	Опытная эксплуатация

Продолжение таблицы 6.1

Этап	Срок	Результат
Общий срок разработки	13 недель	

13 недель — последовательный план полного целевого проекта, включая будущие функции. План четырёх спринтов по две недели в главе 5 относится к сокращённому MVP. Календарные сроки фактического внедрения в вуз не подтверждены.

6.13 Критерии приёмки

Система считается готовой к внедрению при выполнении следующих условий:

Все пять конкурсных списков формируются автоматически и корректно.

Договор генерируется по шаблону за < 10 секунд.

Статусы договора обновляются без потери данных.

Списки для обзвона формируются по расписанию.

Загрузка Excel-выгрузки из ЕПГУ работает без ошибок (гибкий парсер распознаёт столбцы по семантическим признакам).

Выгрузка в 1С проходит без ошибок.

Отчёты формируются корректно в форматах Excel и PDF.

Производительность соответствует заявленной (10 000 записей загружаются за < 3 секунд).

Сайт 2 отображает только обезличенные данные (152-ФЗ).

Все данные защищены и архивируются.

ИИ-прогноз шансов поступления пересчитывается ежедневно ночью.

Чат-бот отвечает на типовые вопросы из базы знаний FAQ.

6.14 Проверяемые условия приёмки MVP

Таблица 6.2 — Условия приёмки и доказательства

Условие	Доказательство	Статус
Повторный импорт не создаёт дубли	Проверки upsert и повторный прогон полного Excel	Подтверждено

Условие	Доказательство	Статус
Рейтинги разделены по ключу конкурса	Тесты года, направления, формы и пяти категорий	Подтверждено
Публичный ответ не содержит ПДн	Проверки запрещённых полей и контрольных маркеров	Подтверждено
Договор сохраняет две оси статуса	Тесты переходов и истории	Подтверждено
Оплата повторно не подтверждается как новая	Тест повторной бухгалтерской выгрузки	Подтверждено
Оба сайта читают одну базу	Браузерные сценарии реального API без демо-подмены	Подтверждено
500 одновременных публичных запросов	Нужен отдельный нагрузочный стенд	Не проверено
Ночная обученная ИИ-модель	Нужны исторические данные, метрики качества и согласование	Дальнейшее развитие
Ежедневный backup в 02:00	CLI работает; планировщик устанавливается администратором	Организационная настройка
Прямой обмен с конкретной 1С	Нужны спецификация и приёмочный контур 1С	Дальнейшее развитие

7 Архитектура реализованной системы

7.1 Общая архитектура и границы доверия

Реализация состоит из общего Python API и двух независимых React-приложений. Внутренний сайт использует JWT и служебные HTTP-маршруты. Публичный портал обращается только к /public/* без авторизации и не получает объект карточки абитуриента. Оба интерфейса используют один файл SQLite, поэтому повторно переносить данные между сайтами не требуется.

Архитектурная граница приватности проходит в API, а не только в интерфейсе. Удаление колонок из HTML не обеспечивает защиту, если сервер возвращает исходную запись целиком. Поэтому публичные ответы создаются как отдельные словари с номером, баллами, конкурсом и оценкой положения.

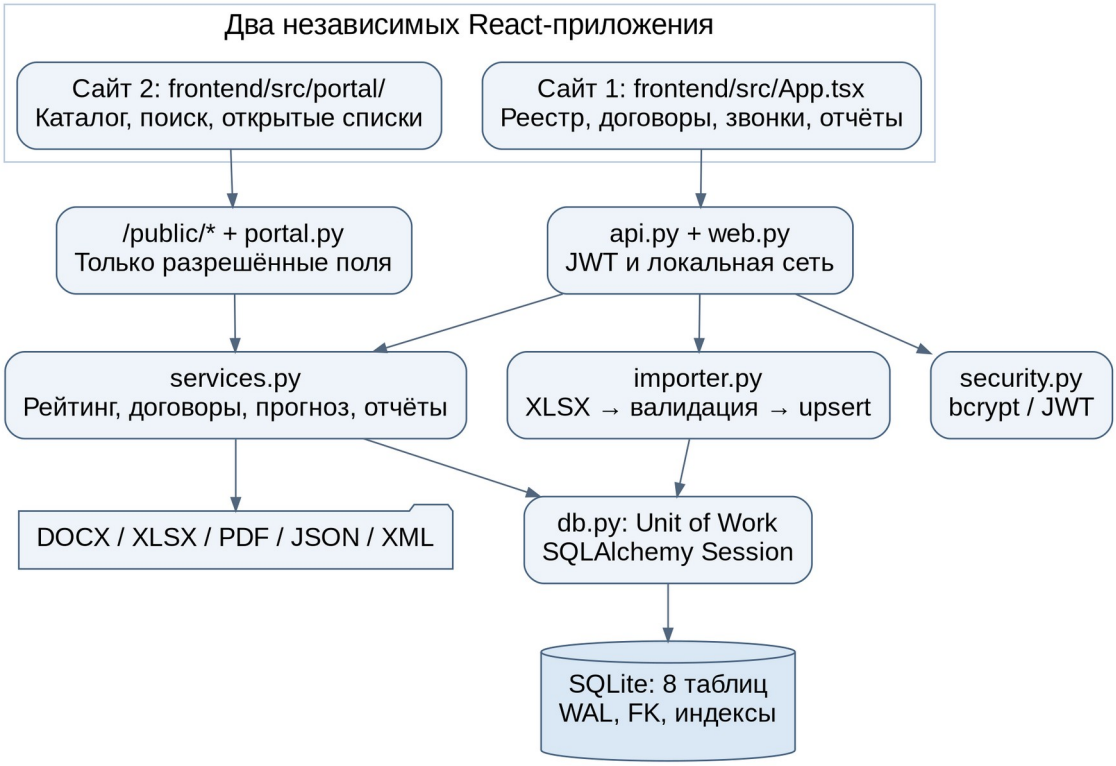


Рисунок 7.1 — Компоненты реализованной версии 1

7.2 Функциональные модули

Таблица 7.1 — Исходная концептуальная функциональная декомпозиция

№	Модуль	Назначение	Основные сервисы	Библиотеки
1	AbiturientModule	Реестр абитуриентов, загрузка Excel	AbiturientService, ExcelParserService	openpyxl, pandas, Pydantic
2	DogovorModule	Жизненный цикл договора	DogovorService, DogovorGeneratorService	python-docx
3	RatingModule	Конкурсные списки и рейтинг	RatingCalculator, CompetitiveListService	SQLAlchemy
4	ObzvonModule + AIModule	Обзвон, ИИ-прогноз, чат-бот	CallListService, AIForecastService, ChatBotService	httpx, Яндекс GPT API
5	IntegrationModule	Отчёты, 1С, архив, аналитика	ReportGenerator, OneCExporter, ArchiveService	openpyxl, reportlab

Имена AbiturientModule, DogovorModule и AIModule являются концептуальными названиями из черновика. Конкретные модули реализации и состояние ИИ-функций приведены далее; внешние GPT API и pandas в текущем ядре не используются.

Таблица 7.2 — Реальные файлы и ответственность

Файл / каталог	Назначение	Основные элементы
admissions/db.py	Соединение и транзакции	Database, create_schema, uow; SQLite WAL, foreign_keys, busy_timeout.

Продолжение таблицы 7.2

Файл / каталог	Назначение	Основные элементы
admissions/models.py	Физическая модель	Восемь ORM-таблиц, составной ключ заявления, индексы.
admissions/importer.py	Импорт ЕПГУ	Алиасы заголовков, валидация, read_epgu, import_epgu.
admissions/services.py	Предметные операции	Рейтинг, прогноз, договоры, оплаты, звонки, экспорт и backup.
admissions/security.py	Доступ сотрудника	bcrypt, аутентификация, выдача и проверка JWT.
admissions/api.py	REST и входные схемы	Pydantic, закрытые и открытые методы, обработка ошибок.
admissions/web.py	Внутренние представления	Дашборд, каталог групп, фильтры реестра/договоров, журнал.
admissions/portal.py	Публичный каталог	Группы, категории, код программы и агрегированные количества.
admissions/__main__.py	Командная строка	init-db, create-user, import-epgu, backup.
frontend/src	Сайт 1	App, StoreProvider и рабочие страницы.
frontend/src/portal	Сайт 2	PortalProvider, поиск, списки, каталог, помощник.

7.3 Развёртывание

Для локальной проверки Uvicorn слушает 127.0.0.1:8000, а браузер обращается к двум HTML-входам этого же сервера. Сайт сотрудников доступен на /, портал — на /abiturient/. Текущая версия не размещена на публичном сервере университета. Промышленное размещение планируется через Nginx и HTTPS: внешние запросы проходят reverse proxy, Uvicorn работает во внутреннем контуре, резервное копирование выполняет планировщик. Это проект дальнейшего внедрения.

За reverse proxy ограничение служебных путей должно применяться на самом прокси: backend обычно видит адрес прокси. Публично разрешаются /public/*, /abiturient/, /assets/ и favicon; остальные пути допускаются из согласованной сети вуза. Секрет JWT и путь БД задаются окружением службы.

7.4 Диаграмма классов системы

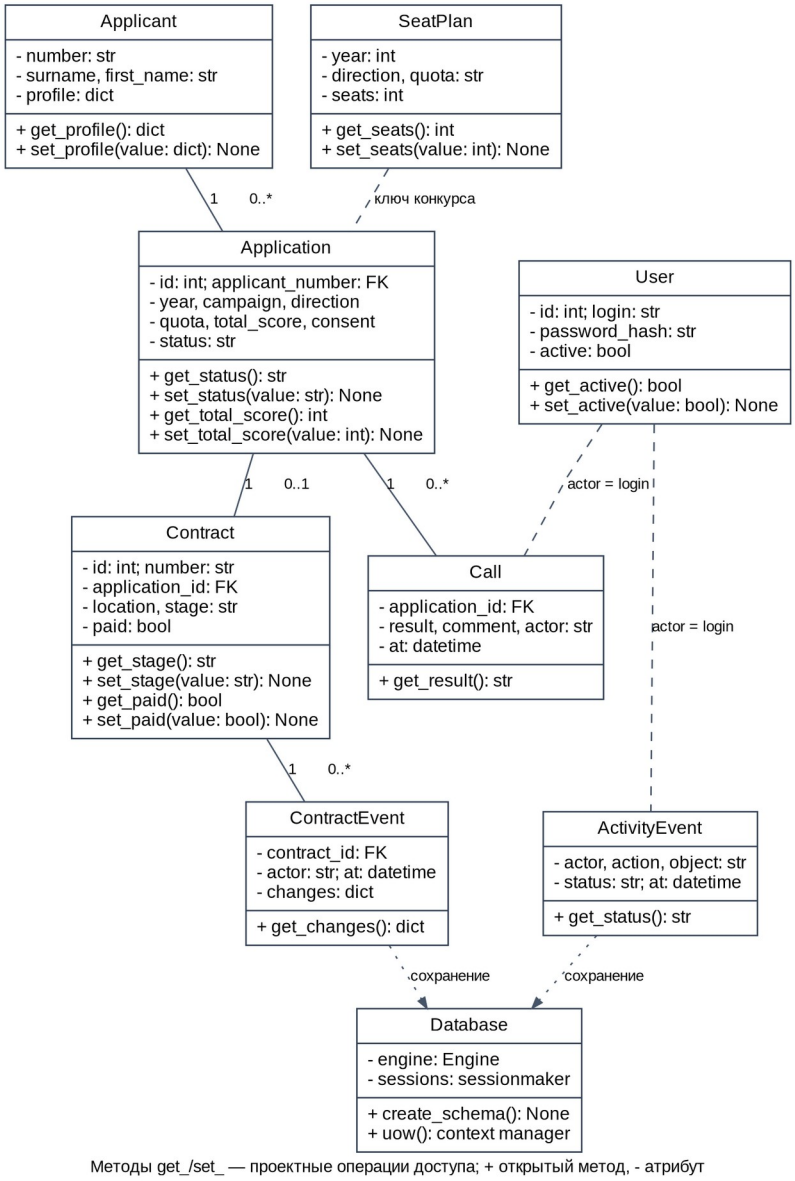


Рисунок 7.2 — Диаграмма классов системы

Методы get_/set_ относятся к проектной инкапсуляции. В Python-реализации используются ORM-атрибуты и сервисные проверки. Неизменяемые события аудита и записи звонков имеют только операции чтения.

Applicant хранит общие данные человека, Application — участие в конкурсе. Договор и звонок связаны с заявлением. Database.uow обеспечивает атомарное сохранение связанных изменений.

7.5 Основные взаимодействия компонентов

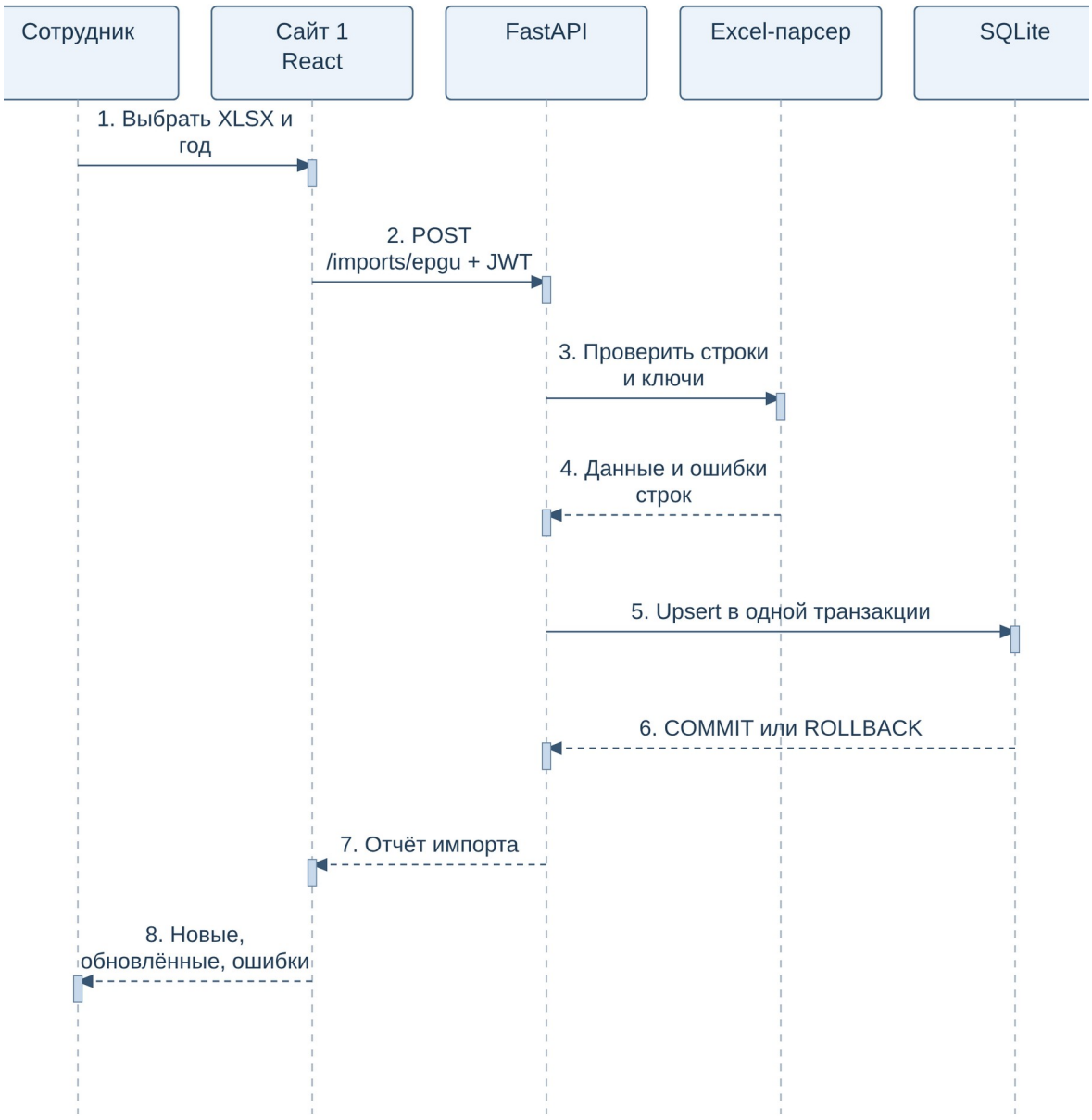


Рисунок 7.3 — Последовательность загрузки и фиксации Excel

Импорт сначала формирует валидные строки и список ошибок, затем сохраняет согласованное состояние базы. Ошибки отдельных входных строк отражаются в отчёте; исключение уровня БД откатывает всю транзакцию сохранения.

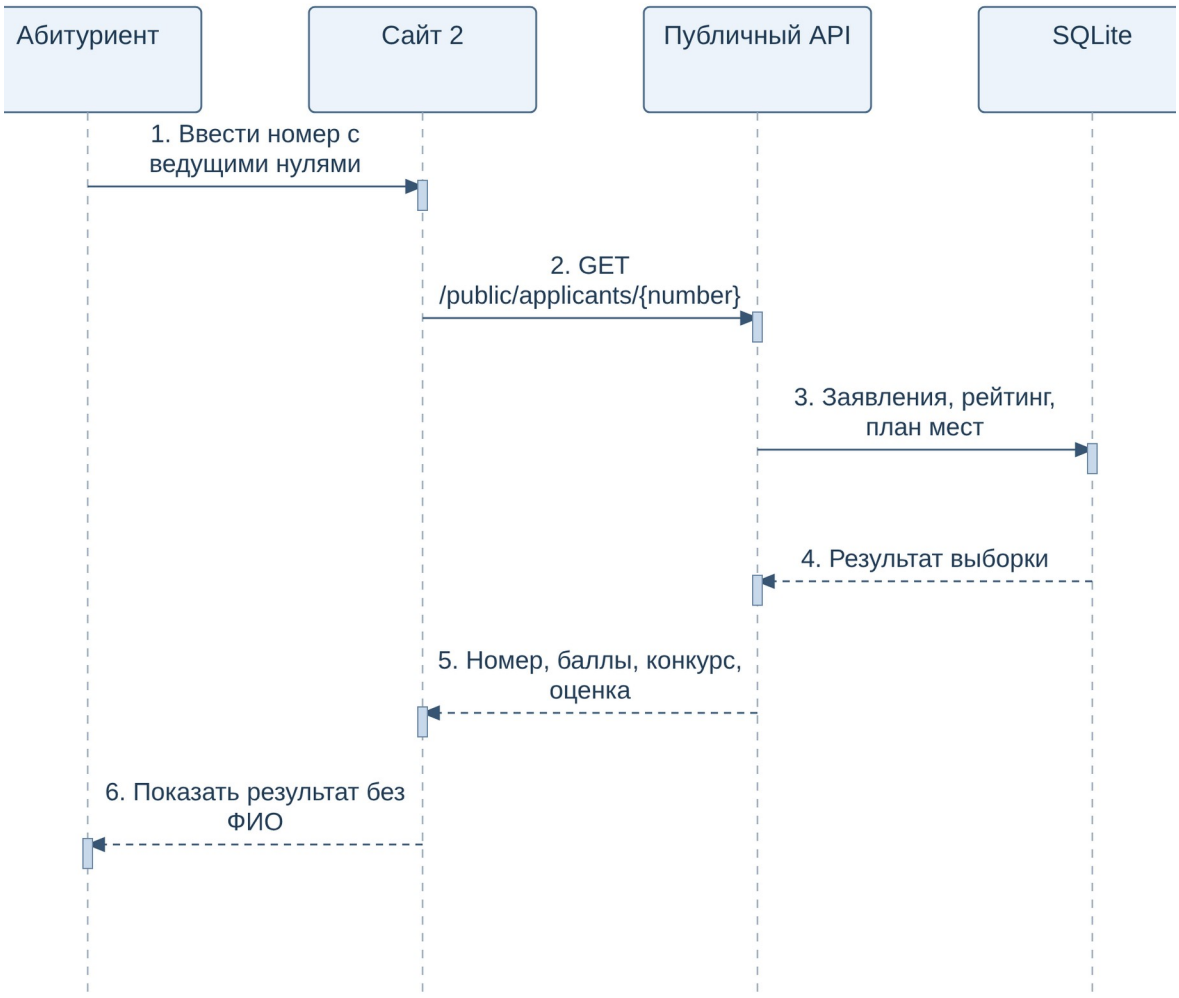


Рисунок 7.4 — Публичный поиск: общая база без раскрытия персональных данных

Запрос открытого портала не использует токен сотрудника. Результат поиска формируется из разрешённых полей; неизвестный номер или отсутствие заявлений в выбранном году приводят к отдельному ответу, а не к подстановке демонстрационных значений.

7.6 Диаграммы состояний сущностей

Состояние договора задаётся тремя независимыми атрибутами: stage, location и paid. У заявления отдельно учитываются status и consent; у пользователя — active. Поэтому представлены шесть диаграмм по шести наборам состояний (рисунки 7.5–7.10). Они не сводятся к одной линейной цепочке. ContractEvent и ActivityEvent — неизменяемые записи аудита, а Call фиксирует результат отдельного разговора; их

значения не образуют самостоятельный жизненный цикл одной изменяемой записи. Applicant и SeatPlan не имеют атрибута жизненного цикла.

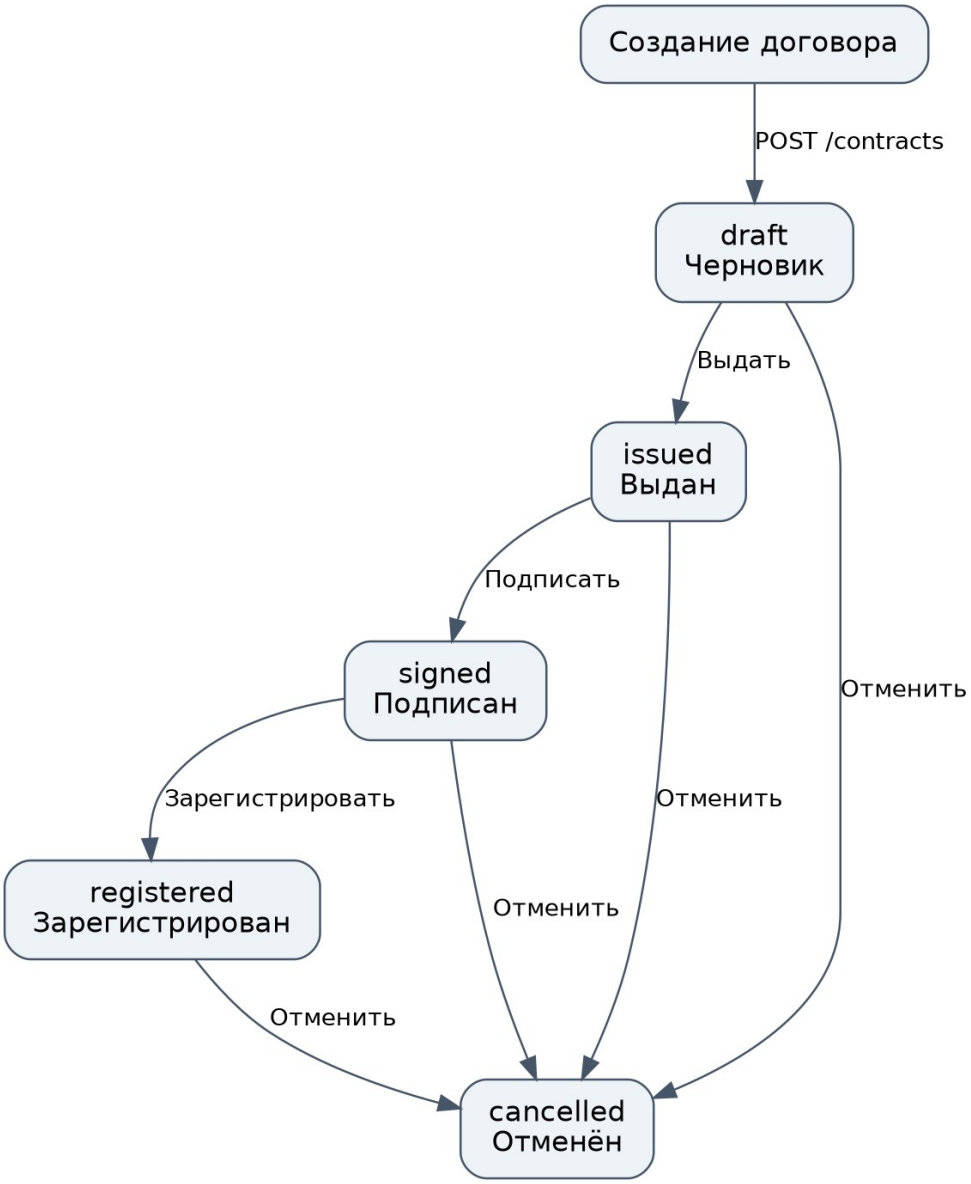


Рисунок 7.5 — Этап договора (Contract.stage)

Переходы соответствуют TRANSITIONS в services.py. Обратный переход и выход из cancelled запрещены; изменение записывается в ContractEvent.



Рисунок 7.6 — Местоположение договора (Contract.location)

При передаче бумажного договора сотрудник фиксирует его фактическое местоположение, а не предполагаемый следующий этап. Передача в бухгалтерию сама по себе не означает подписание, регистрацию или поступление денег. Поэтому интерфейс должен показывать место хранения рядом с этапом оформления и признаком оплаты. В журнале сохраняются автор и время изменения. Такое представление помогает искать документ и разбирать задержки, не смешивая физическое движение экземпляра с юридическим результатом его оформления.

Местоположение не зависит от stage. Реализация проверяет принадлежность LOCATIONS, но не навязывает порядок передачи между подразделениями.



Рисунок 7.7 — Признак оплаты договора (Contract.paid)

Признак оплаты отражает подтверждённое состояние расчётов и не заменяет банковский документ. При повторной загрузке бухгалтерского файла система сопоставляет сведения по номеру договора, поэтому повтор одного и того же подтверждения не должен создавать новое обязательство. Отрицательное значение или отсутствие строки в очередной выгрузке не трактуются как автоматический возврат денег. Исправление ранее установленного признака является отдельным явным действием сотрудника и должно оставлять запись в истории изменения договора.

В бухгалтерском XLSX отсутствие договора или отрицательное значение не отменяет ранее подтверждённую оплату. Общий сервис PATCH допускает явное paid=false. Отменённому договору впервые подтвердить оплату нельзя.

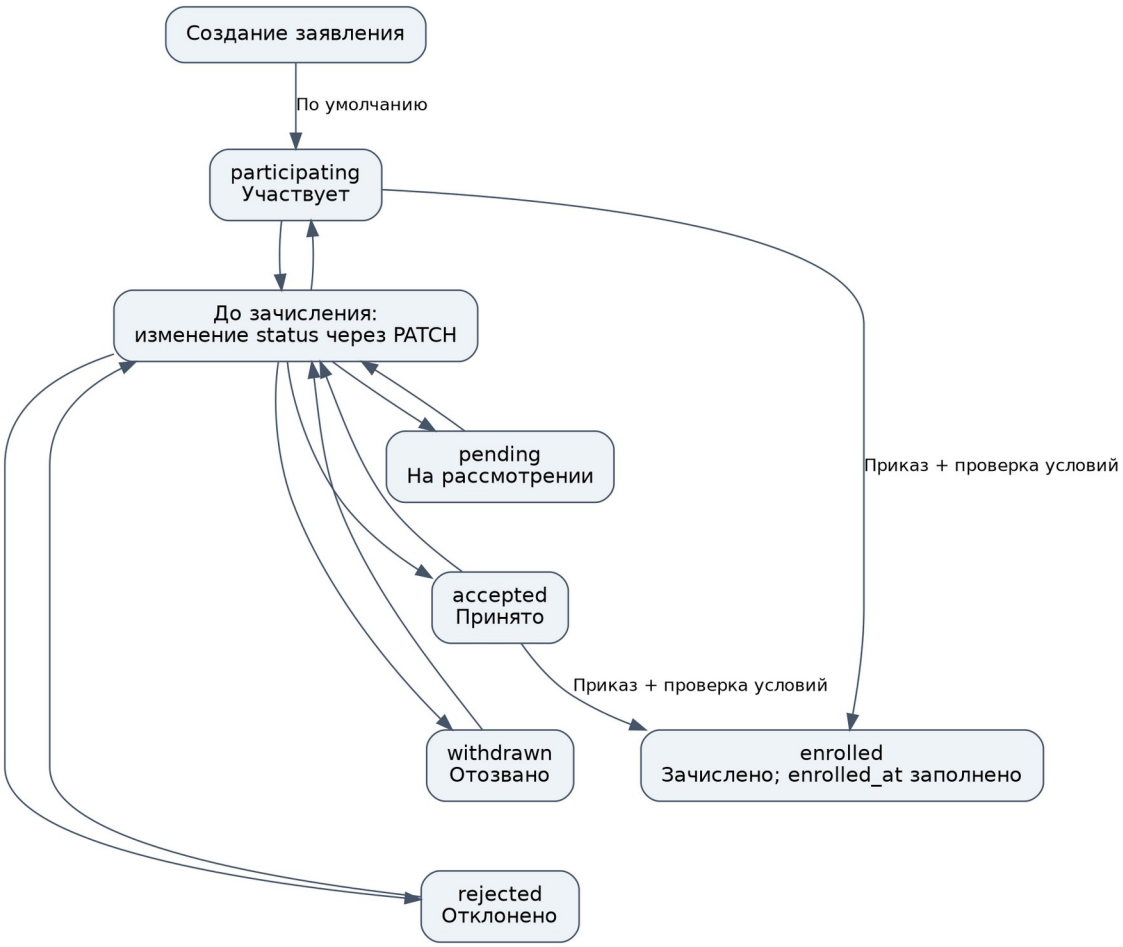


Рисунок 7.8 — Статус заявления (Application.status)

До зачисления API допускает выбор любого из пяти редактируемых статусов. Для бюджетного зачисления нужно согласие; для платного — зарегистрированный оплаченный договор; повторное зачисление в кампании запрещено. После фиксации приказа обычное редактирование блокируется. Импорт может принести исходный статус enrolled, но не заменяет фиксацию приказа.

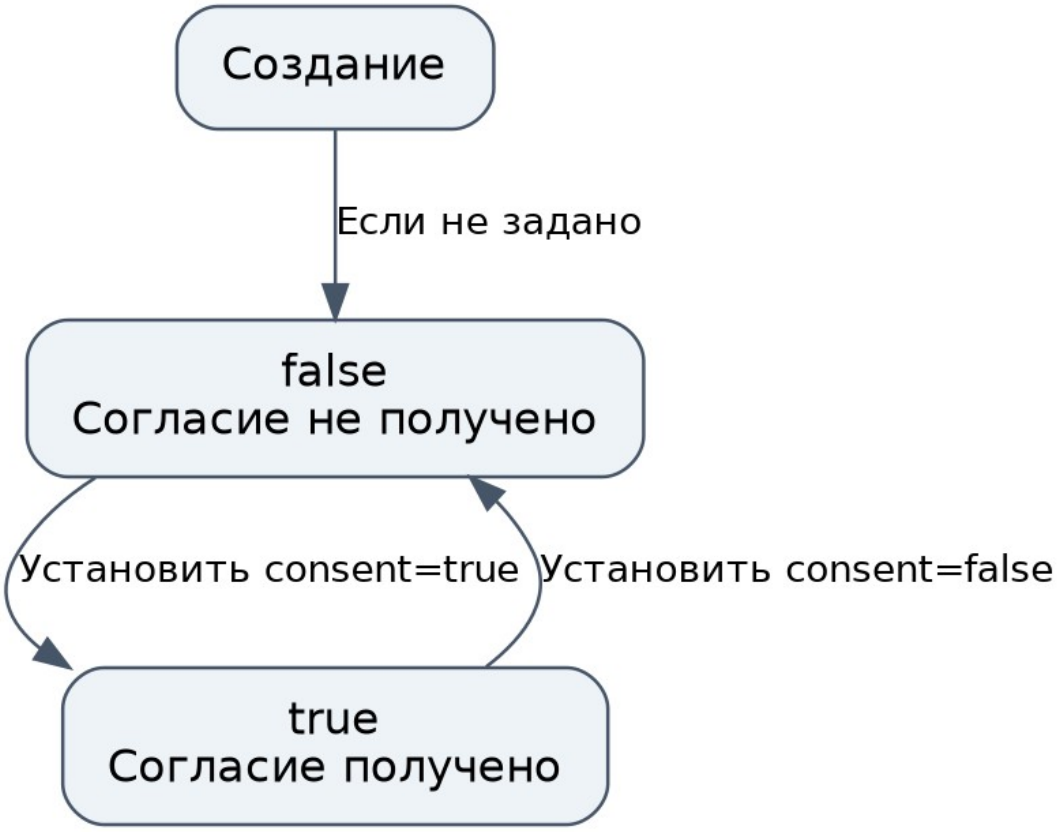


Рисунок 7.9 — Согласие на зачисление (Application.consent)

Согласие рассматривается отдельно от баллов и позиции в списке. Его изменение может повлиять на возможность бюджетного зачисления, но само по себе не меняет результат ранжирования. После фиксации приказа обычное редактирование заявления ограничено: исправления должны выполняться по отдельной согласованной процедуре.

Признак поступает из импорта или PATCH заявления. Переходы через PATCH допускаются только до enrolled_at. Согласие учитывается отдельно от статуса заявления и от оплаты договора.



Рисунок 7.10 — Доступ пользователя (User.active)

authenticate и проверка JWT отклоняют неактивного пользователя. В версии 1 отдельный HTTP-интерфейс переключения active отсутствует; административные переходы показаны как модель сопровождения, а не как реализованные кнопки.

7.7 Полнота диаграмм последовательностей

Для требований FR-1–FR-15 приведены диаграммы последовательностей. Проектные взаимодействия описаны в главе 7; схемы, поясняющие импорт, рейтинг и договоры, перенесены в руководство программиста. В главе 9 приведены соответствующие фрагменты кода.

Таблица 7.3 — Соответствие требований диаграммам последовательностей

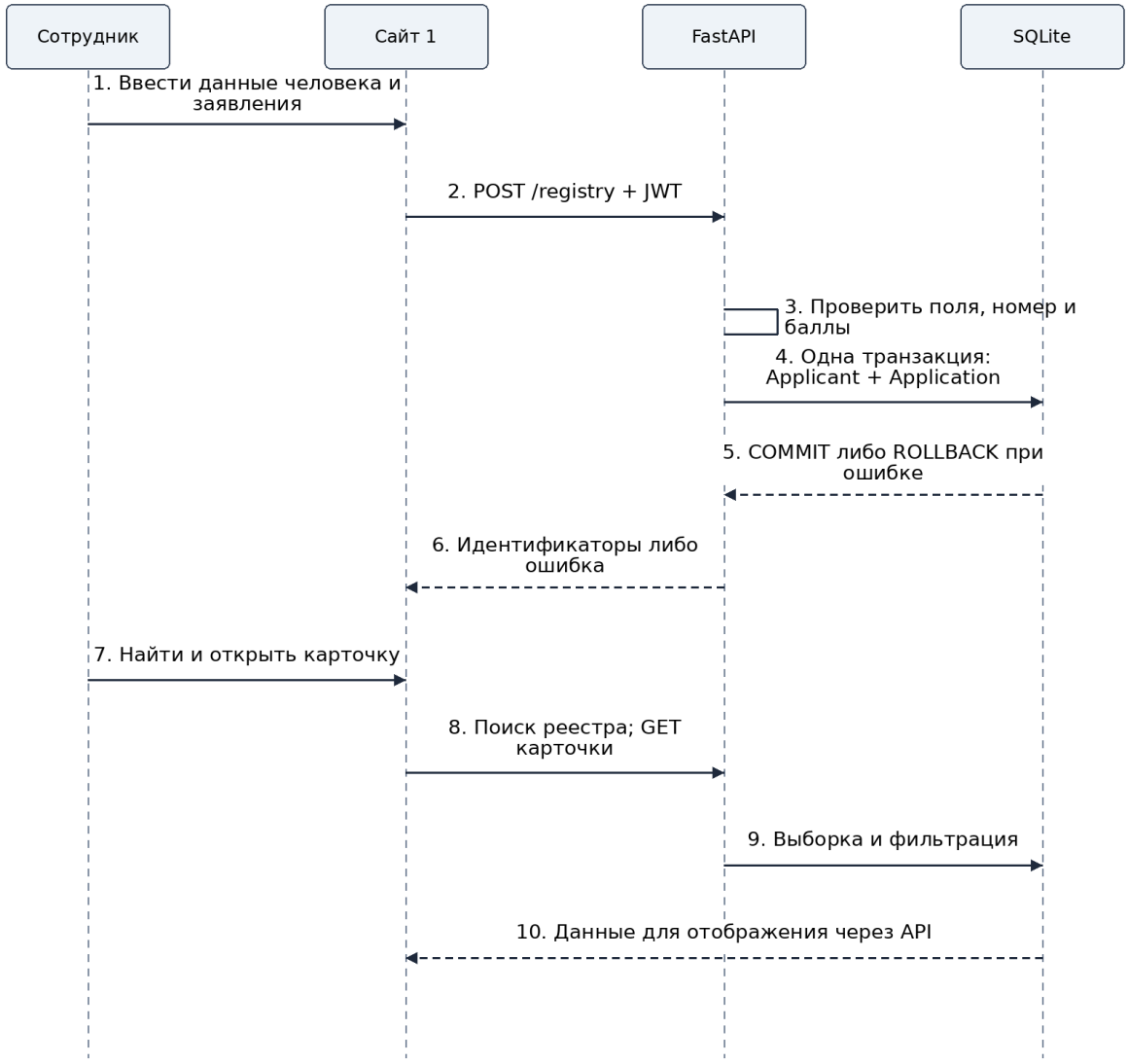
Требование	Сценарий	Рисунок
FR-1	Реестр и карточка	7.11
FR-2	Импорт Excel	7.3

Продолжение таблицы 7.3

Требование	Сценарий	Рисунок
FR-3	Классификация квот	7.12
FR-4	Расчёт рейтинга	7.13
FR-5	Внутренние экзамены	7.14
FR-6	Договор и аудит	9.3
FR-7	Подтверждение оплаты	9.4
FR-8	Очередь и результат обзвона	9.5
FR-9	Оценка положения и целевой ML-прогноз	7.15
FR-10	Обезличенный поиск	7.4
FR-11	Справочный чат-помощник	7.16
FR-12	Статистика и отчёты	7.17
FR-13	Исторические данные и резервная копия	7.18
FR-14	Пакет для 1С	9.6
FR-15	Вход и проверка JWT	7.19

Матрица трассировки связывает каждое требование с отдельным сценарием взаимодействия. При проверке сначала выбирается требование FR, затем рассматриваются участники, входные данные и ожидаемый результат соответствующей диаграммы. Для операций изменения данных важны граница транзакции и поведение при ошибке, для публичных запросов — состав разрешённых полей. Требования к обученному прогнозированию и длительному архивному хранению нельзя считать выполненными только потому, что для них нарисована последовательность. В этих случаях диаграмма разделяет работающую часть прототипа и проектное расширение. Такое сопоставление помогает планировать дальнейшие работы: видно, какие возможности проверяются существующими тестами, а какие требуют модели, регламента, интеграционного контура или дополнительных пользовательских форм. Поэтому матрица используется как навигация по доказательствам реализации, а не как формальный перечень иллюстраций.

7.7.1 Ведение реестра и карточки

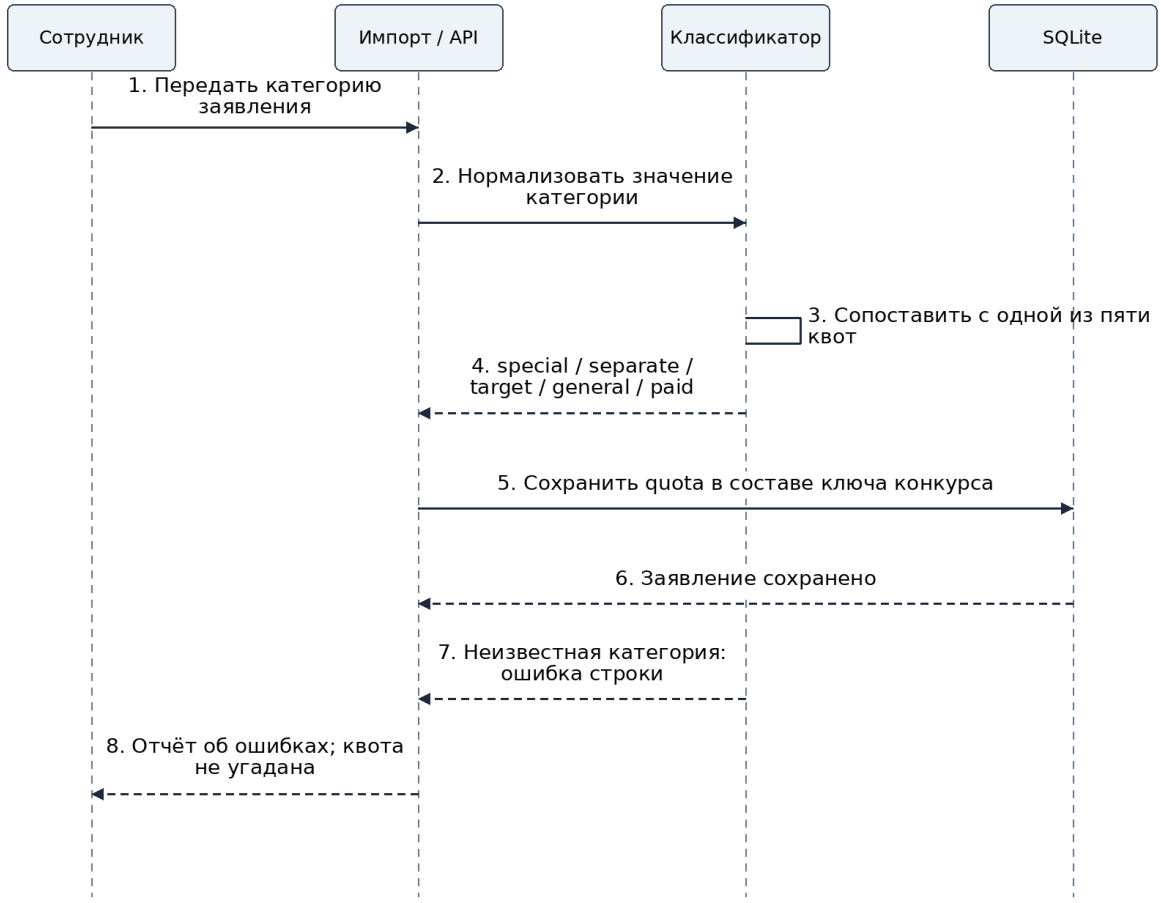


Сплошная стрелка — вызов; пунктирная — ответ.

Рисунок 7.11 — FR-1: ведение реестра и карточки

Основной сценарий MVP: атомарная регистрация, поиск и карточка. Расширенные вкладки и отдельная форма родителя остаются развитием FR-1.

7.7.2 Распределение по квотам

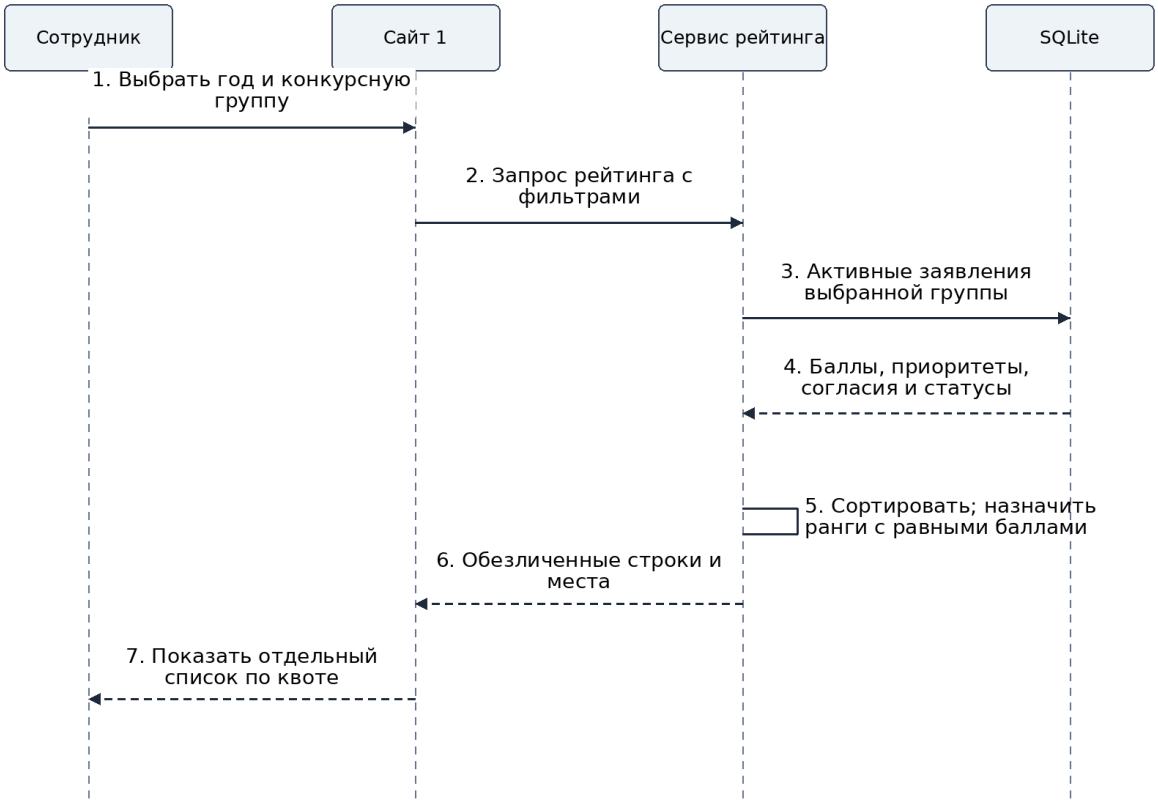


Сплошная стрелка — вызов; пунктирная — ответ.

Рисунок 7.12 — FR-3: распределение по квотам

Альтернативная ветвь неизвестной категории выполняется вместо сохранения. Основание льготы задаётся исходными данными; автоматической юридической проверки права на квоту в MVP нет.

7.7.3 Рейтинг и конкурсные списки



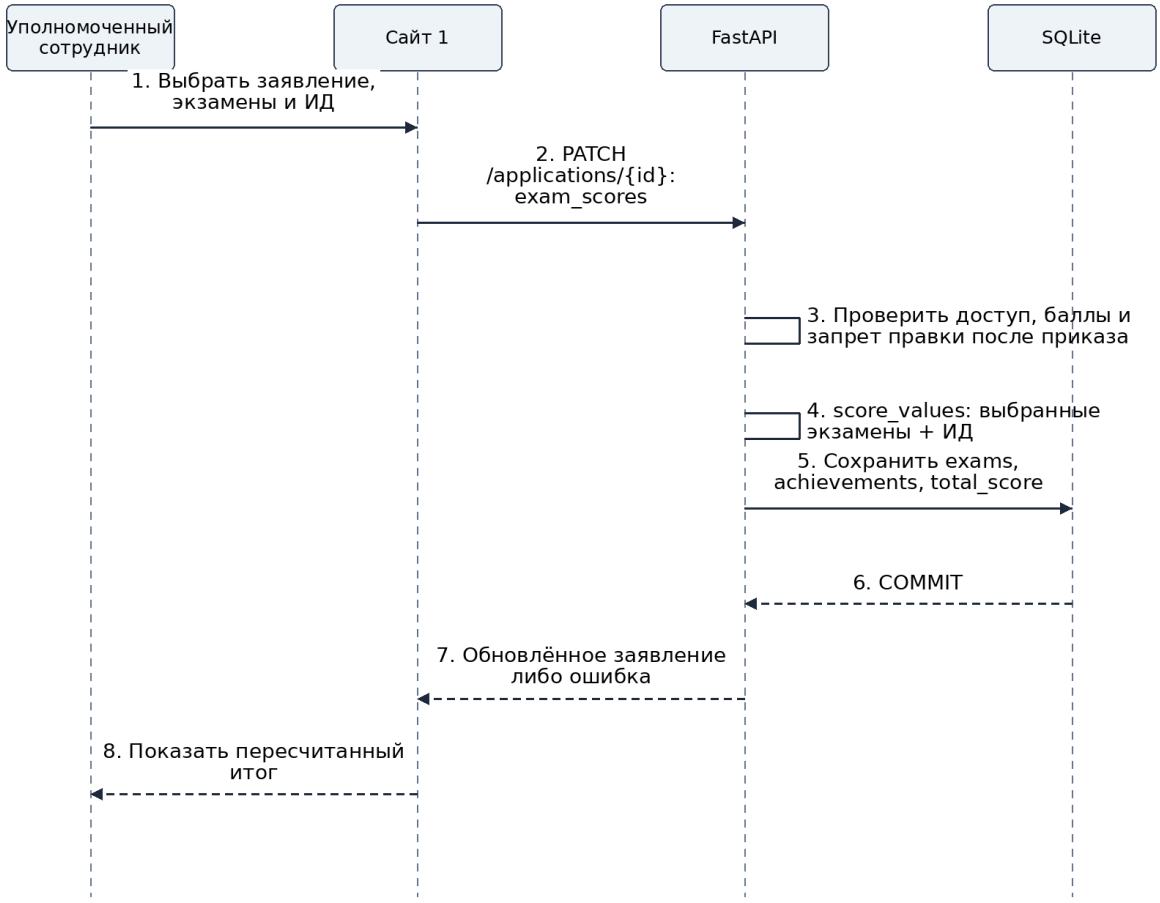
Сплошная стрелка — вызов; пунктирная — ответ.

Рисунок 7.13 — FR-4: рейтинг и конкурсные списки

Результат расчёта описывает текущую выборку на момент запроса. Он не заменяет приказ о зачислении и не предполагает автоматического разрешения всех ситуаций с равными баллами.

Группа определяется годом, кампанией, направлением, формой и квотой. Равенство баллов сохраняется; эта диаграмма описывает ранжирование, а не юридическое решение о зачислении.

7.7.4 Учёт внутренних экзаменов



Сплошная стрелка — вызов; пунктирная — ответ.

Рисунок 7.14 — FR-5: учёт внутренних экзаменов

В MVP нет отдельной роли преподавателя СПО: действует общий доступ сотрудников. Выбор альтернативных экзаменов явный; итог из Excel не складывается повторно с ИД.

7.7.5 Оценка положения и целевой прогноз

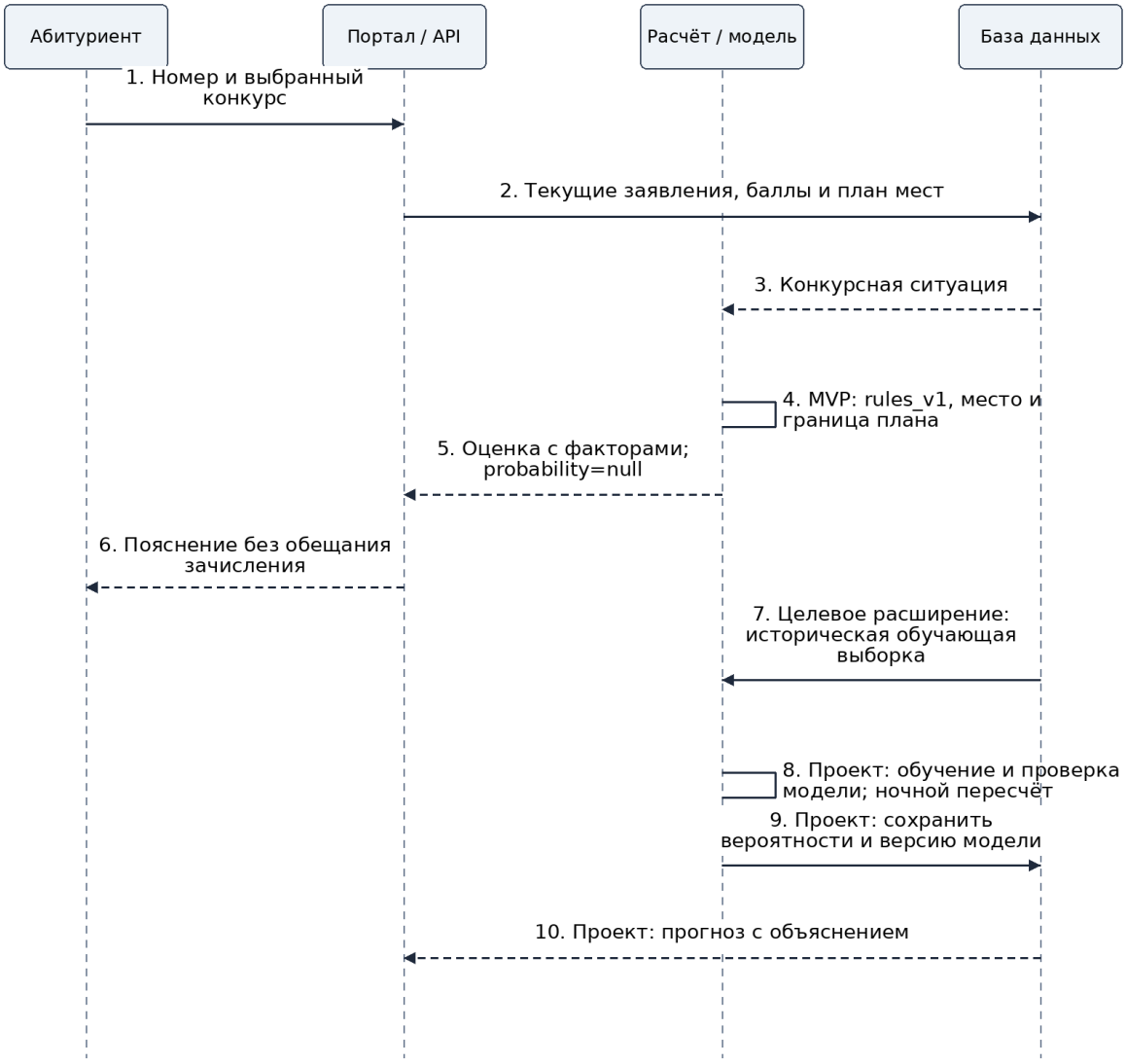
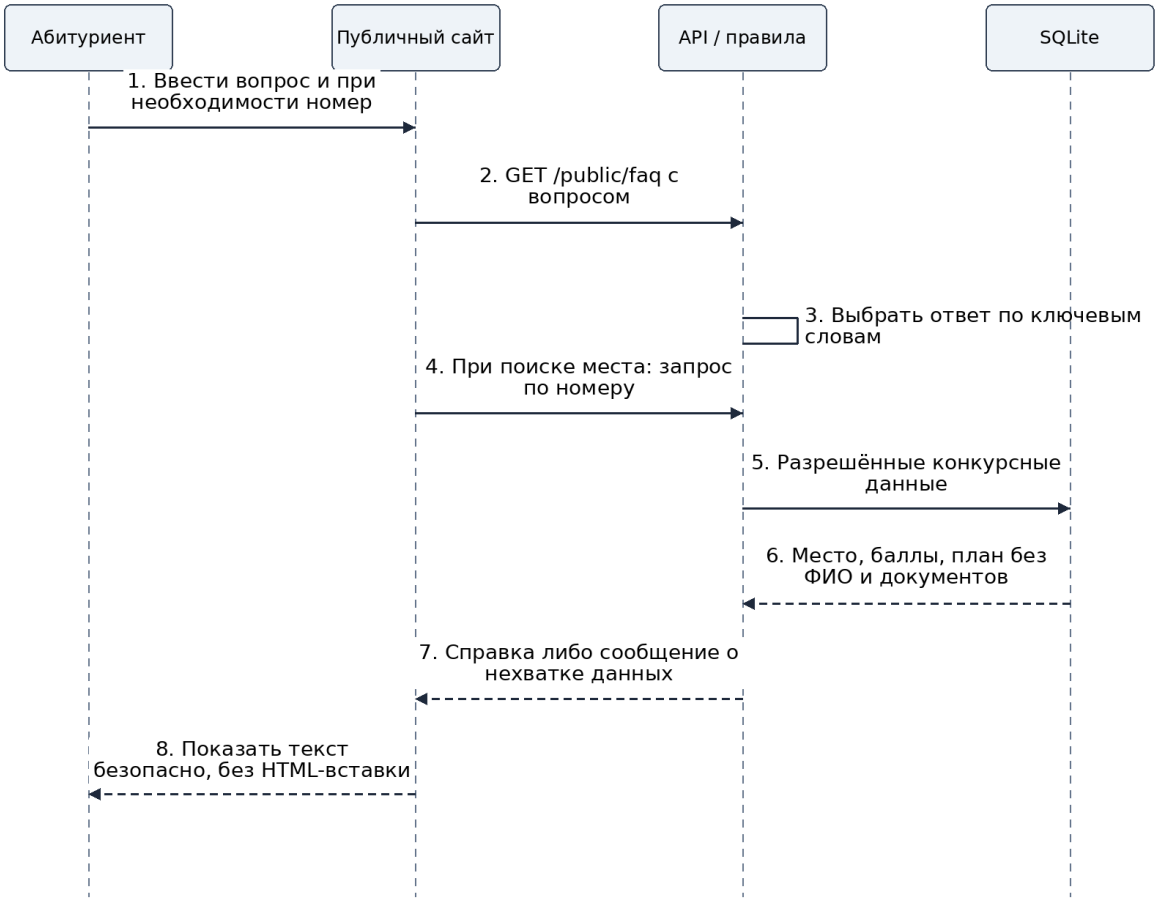


Рисунок 7.15 — FR-9: оценка положения и целевой прогноз

Шаги 1–6 отражают реализованную оценку. Шаги 7–10 — целевая часть FR-9: обученная модель, исторические данные и ночной пересчёт пока не реализованы.

7.7.6 Ответ справочного чат-помощника

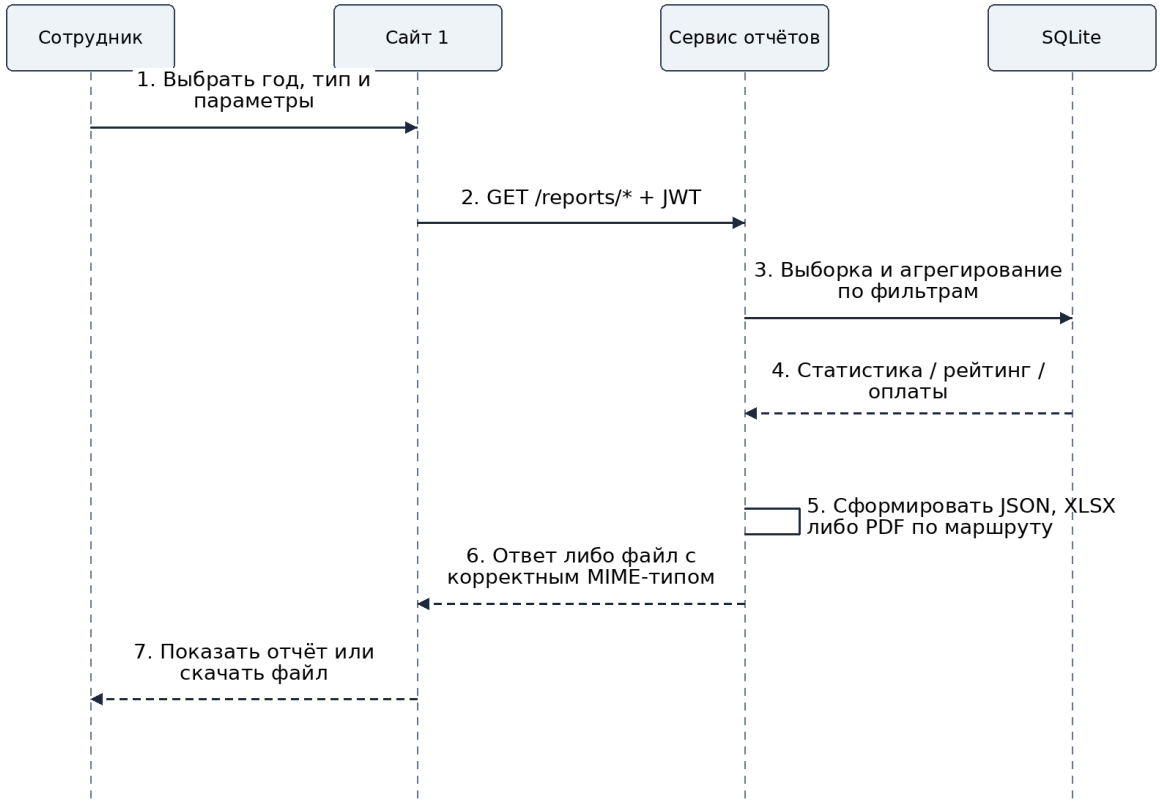


Сплошная стрелка — вызов; пунктирная — ответ.

Рисунок 7.16 — FR-11: ответ справочного чат-помощника

Помощник работает по правилам, не обращается к внешнему GPT и не выдумывает сроки или вероятность. Поиск места использует публичный маршрут, а не служебную карточку.

7.7.7 Формирование отчётов



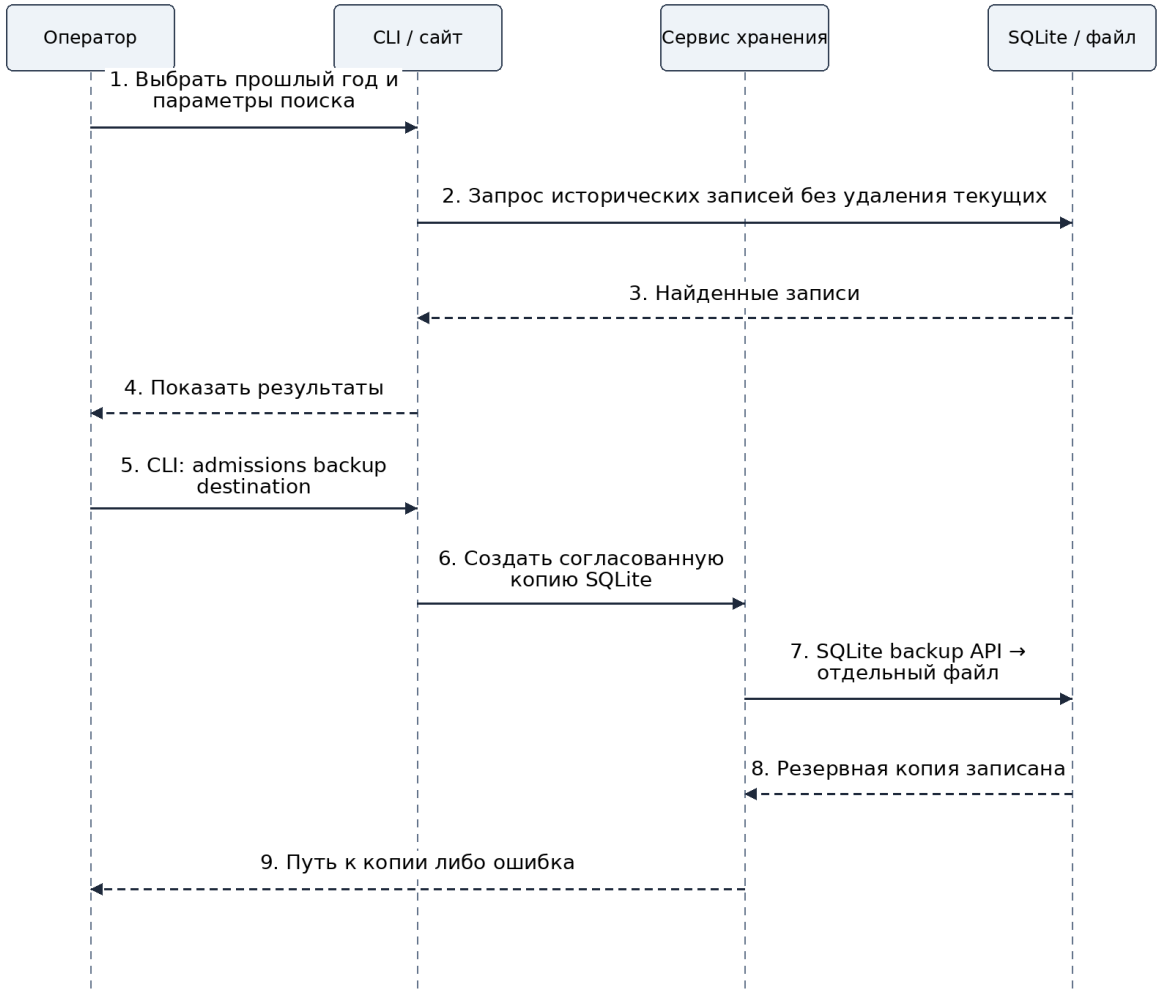
Сплошная стрелка — вызов; пунктирная — ответ.

Рисунок 7.17 — FR-12: формирование отчётов

Параметры отчёта должны сохранять одинаковый смысл при просмотре на экране и выгрузке файла. Перед использованием результата сотрудник проверяет год кампании, конкурсную группу и состав применённых фильтров.

MVP формирует статистику, рейтинг и бухгалтерскую ведомость. Специальный регламентированный отчёт для Министерства образования не подтверждён реализацией и остаётся расширением FR-12.

7.7.8 Исторические записи и резервная копия

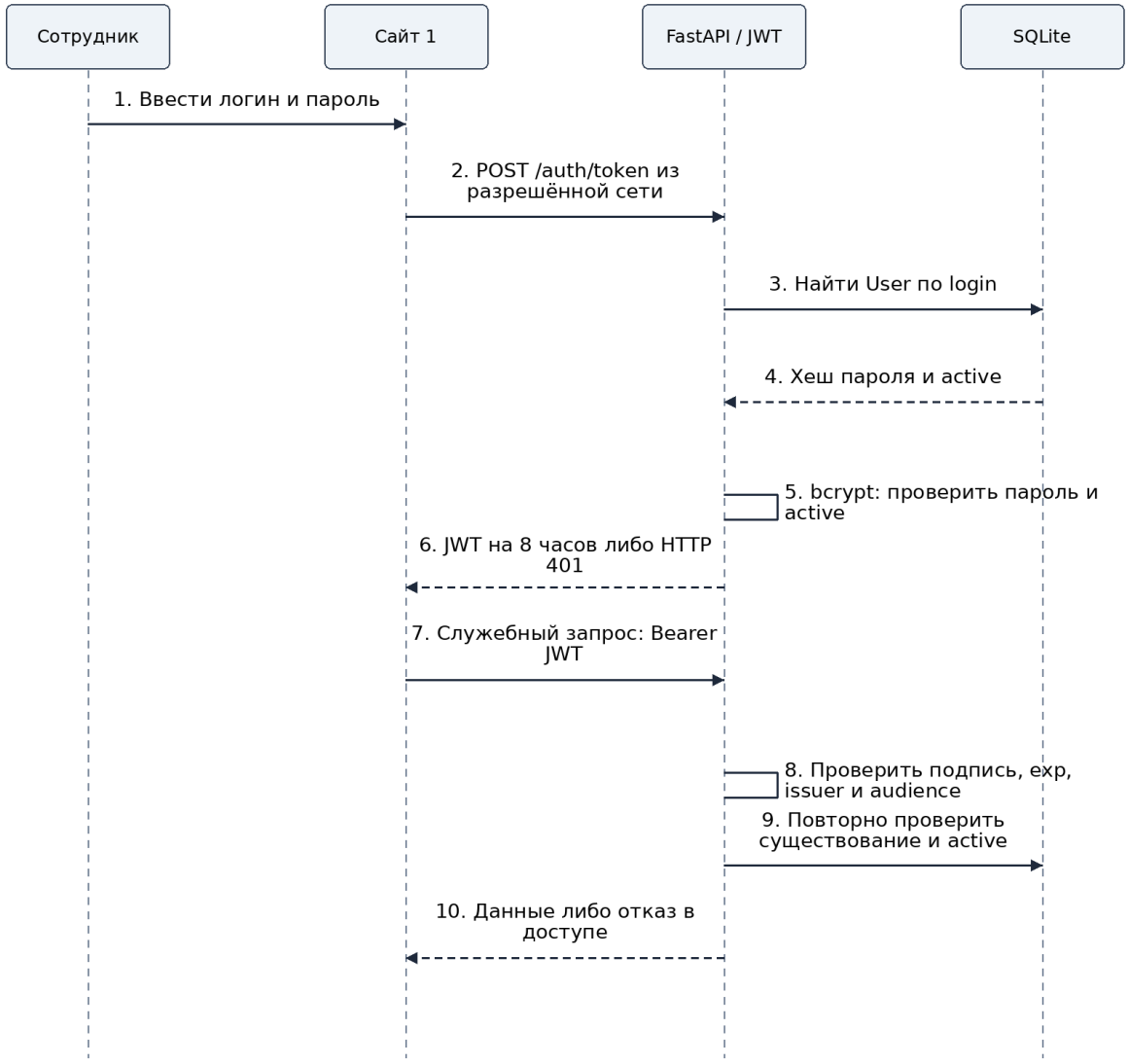


Сплошная стрелка — вызов; пунктирная — ответ.

Рисунок 7.18 — FR-13: исторические записи и резервная копия

Разделение по году и backup поддержаны. Автоматическое шестилетнее хранение, архивный контур, поиск архива по всем заданным полям и проверка восстановления требуют отдельного регламента; диаграмма не утверждает их готовность.

7.7.9 Аутентификация и доступ



Сплошная стрелка — вызов; пунктирная — ответ.

Рисунок 7.19 — FR-15: аутентификация и доступ

Успешный вход не отменяет проверку пользователя при последующих запросах. Публичный портал JWT не требует; сотрудники имеют общий уровень доступа.

8 Проектирование базы данных

8.1 Инфологическая и физическая модель MVP

Физическая модель версии 1 содержит восемь таблиц: applicants, applications, seat_plans, contracts, contract_events, calls, users и activity_events. В исходном проекте рассматривалась более детализированная нормализованная модель из 23 сущностей. Она сохранена в архиве справочных материалов как проект расширения и не выдаётся за реально созданную ORM-схему.

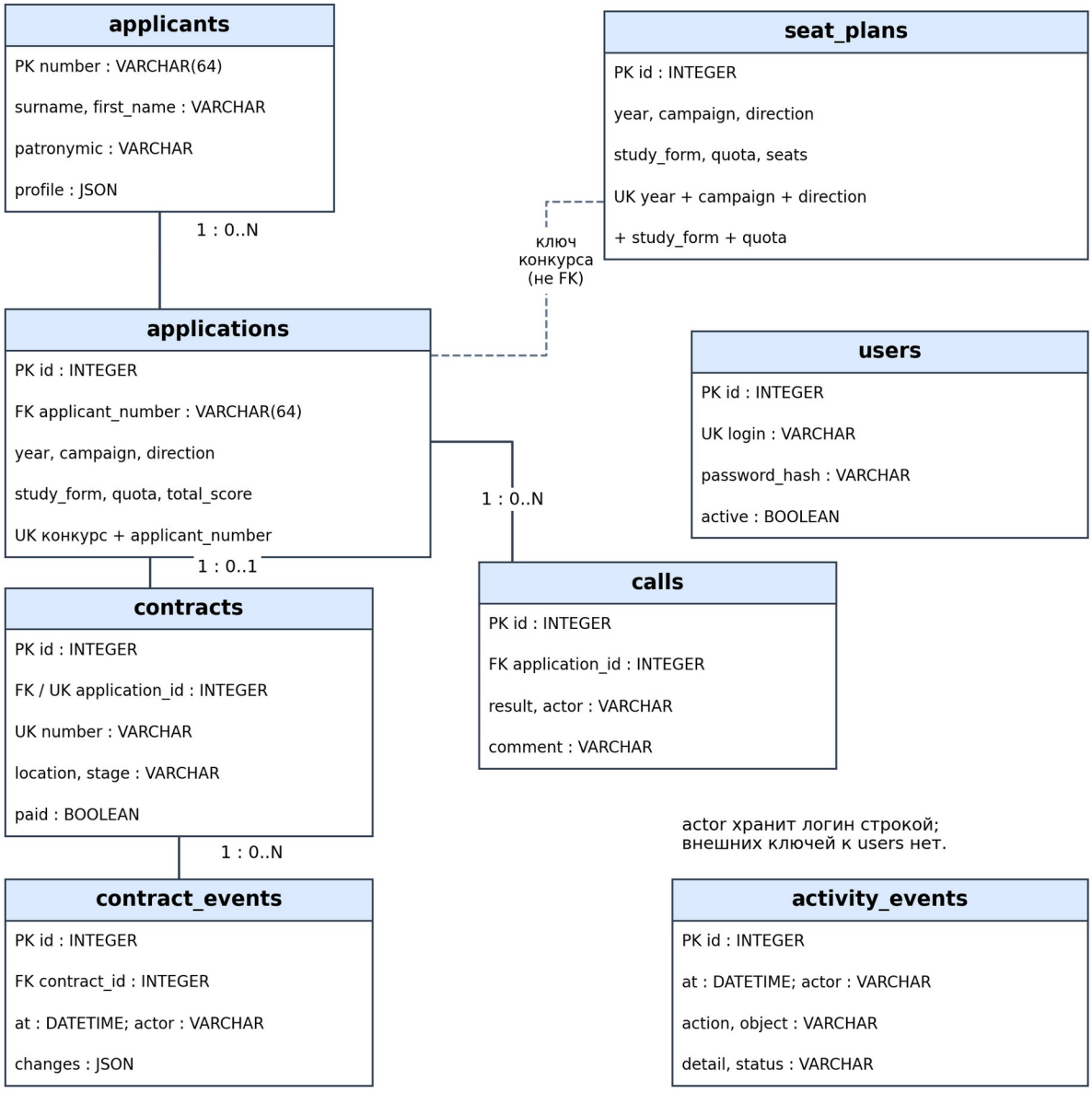


Рисунок 8.1 — ER-модель данных: ключи и кратности связей

Сплошные линии обозначают внешние ключи и кратности: один абитуриент может иметь несколько заявлений; заявление — не более одного договора и любое число звонков; договор — историю событий. Пунктир отражает сопоставление seat_plans с applications по составному ключу конкурса, без физического внешнего ключа. Поля actor хранят логин строкой и не связаны с users внешним ключом. На схеме показаны ключевые атрибуты; полный состав полей приведён в словаре таблиц.

8.2 Словарь физических таблиц

Таблица 8.1 — applicants — реализованная таблица

Атрибут	Тип данных	Ограничения	Описание
number	VARCHAR(64)	PK; NOT NULL	Строковый номер ЕПГУ; ведущие нули сохраняются.
surname	VARCHAR	NOT NULL	Фамилия, индекс для поиска.
first_name	VARCHAR	NOT NULL	Имя.
patronymic	VARCHAR	NOT NULL	Отчество.
profile	JSON	NOT NULL	Персональные, контактные и образовательные сведения в JSON.
updated_at	DATETIME	NOT NULL	Время последнего обновления карточки.

Таблица 8.2 — applications — реализованная таблица

Атрибут	Тип данных	Ограничения	Описание
id	INTEGER	PK; NOT NULL	Внутренний идентификатор заявления.
applicant_number	VARCHAR(64)	NOT NULL; FK → applicants.number	Ссылка на applicants.number.
year	INTEGER	NOT NULL	Год приёмной кампании.
campaign	VARCHAR	NOT NULL	Уровень/кампания: бак, маг, асп и т.п.

Продолжение таблицы 8.2

Атрибут	Тип данных	Ограничения	Описание
direction	VARCHAR	NOT NULL	Идентификатор направления из источника.
study_form	VARCHAR	NOT NULL	Форма обучения.
quota	VARCHAR	NOT NULL	Явная категория конкурса.
priority	INTEGER	NOT NULL	Приоритет заявления.
status	VARCHAR	NOT NULL	Участие в конкурсе / рассмотрение / зачисление / отзыв.
total_score	INTEGER	NOT NULL	Готовый итоговый балл конкурса.
achievements	INTEGER	NOT NULL	Баллы индивидуальных достижений.
exams	JSON	NOT NULL	Предметы и их результаты; могут включать альтернативы.
consent	BOOLEAN	NOT NULL	Наличие согласия по данным заявления.
enrolled_at	DATETIME	Допускается NULL	Дата внесённого зачисления, может отсутствовать.
source	JSON	NOT NULL	Исходные колонки строки и номер приказа в локальном поле.

Таблица 8.3 — seat_plans — реализованная таблица

Атрибут	Тип данных	Ограничения	Описание
id	INTEGER	PK; NOT NULL	Первичный ключ.
year	INTEGER	NOT NULL	Год кампании.
campaign	VARCHAR	NOT NULL	Уровень образования.
direction	VARCHAR	NOT NULL	Направление.

Продолжение таблицы 8.3

Атрибут	Тип данных	Ограничения	Описание
study_form	VARCHAR	NOT NULL	Форма обучения.
quota	VARCHAR	NOT NULL	Категория конкурса.
seats	INTEGER	NOT NULL	Количество мест по данному ключу конкурса.

Таблица 8.4 — contracts — реализованная таблица

Атрибут	Тип данных	Ограничения	Описание
id	INTEGER	PK; NOT NULL	Первичный ключ.
number	VARCHAR	NOT NULL; UNIQUE	Уникальный номер договора.
application_id	INTEGER	NOT NULL; UNIQUE; FK → applications.id	Уникальная ссылка на платное заявление.
location	VARCHAR	NOT NULL	Местоположение договора.
stage	VARCHAR	NOT NULL	Этап signing stage.
paid	BOOLEAN	NOT NULL	Признак подтверждённой полной оплаты.
created_at	DATETIME	NOT NULL	Время создания договора.

Таблица 8.5 — contract_events — реализованная таблица

Атрибут	Тип данных	Ограничения	Описание
id	INTEGER	PK; NOT NULL	Первичный ключ события.
contract_id	INTEGER	NOT NULL; FK → contracts.id	Ссылка на договор.

Продолжение таблицы 8.5

Атрибут	Тип данных	Ограничения	Описание
at	DATETIME	NOT NULL	Время события.
actor	VARCHAR	NOT NULL	Логин сотрудника, строка без FK.
changes	JSON	NOT NULL	Изменения полей и их old/new значения в JSON.

Таблица 8.6 — calls — реализованная таблица

Атрибут	Тип данных	Ограничения	Описание
id	INTEGER	PK; NOT NULL	Первичный ключ.
application_id	INTEGER	NOT NULL; FK → applications.id	Ссылка на заявление.
result	VARCHAR	NOT NULL	no_answer, interested, callback, declined, confirmed.
comment	VARCHAR	NOT NULL	Комментарий сотрудника.
actor	VARCHAR	NOT NULL	Логин сотрудника.
at	DATETIME	NOT NULL	Время записи результата.

Таблица 8.7 — users — реализованная таблица

Атрибут	Тип данных	Ограничения	Описание
id	INTEGER	PK; NOT NULL	Первичный ключ.
login	VARCHAR	NOT NULL; UNIQUE	Уникальный логин.
password_hash	VARCHAR	NOT NULL	Хеш bcrypt; открытый пароль не хранится.

Продолжение таблицы 8.7

Атрибут	Тип данных	Ограничения	Описание
active	BOOLEAN	NOT NULL	Возможность использования учётной записи.

Таблица 8.8 — activity_events — реализованная таблица

Атрибут	Тип данных	Ограничения	Описание
id	INTEGER	PK; NOT NULL	Первичный ключ.
at	DATETIME	NOT NULL	Время успешной операции.
actor	VARCHAR	NOT NULL	Логин сотрудника.
action	VARCHAR	NOT NULL	Краткое описание операции.
object	VARCHAR	NOT NULL	Тип объекта, например реестр или договоры.
detail	VARCHAR	NOT NULL	Дополнительная строка описания.
status	VARCHAR	NOT NULL	Категория отображения события.

8.3 Ключи, ограничения и индексы

Таблица 8.9 — Правила целостности

Правило	Реализация	Пояснение
Уникальность человека	applicants.number — PK	Номер является строкой. Числовая ячейка идентификатора в Excel отклоняется, чтобы не угадывать утраченную длину.

Продолжение таблицы 8.9

Правило	Реализация	Пояснение
Уникальность заявления	UNIQUE(applicant_number, year, campaign, direction, study_form, quota)	Составной ключ разделяет направления, формы, годы и категории.
Один договор на заявление	contracts.application_id — UNIQUE + FK	Ограничение paid проверяется в сервисе, а не SQL CHECK.
Конкурсный индекс	ix_ranking	year, direction, quota, status, total_score.
Связанные записи	PRAGMA foreign_keys=ON	Не допускаются ссылки на отсутствующие заявления/договоры.
Параллельный доступ	WAL + busy_timeout=30000	Поддержка чтения и коротких транзакций; не является гарантией любой нагрузки.
Доменные диапазоны	Pydantic и сервисы	Не все бизнес-ограничения продублированы SQL CHECK.

8.4 Хранение JSON и развитие схемы

JSON-профиль и исходная строка позволяют сохранить неизвестные дополнительные поля выгрузки без изменения схемы при каждом появлении новой колонки. Цена такого решения — меньшая нормализация и необходимость выделить устойчивые справочники при расширении продукта. В рабочей версии нет отдельных физических таблиц паспортов, льгот, ИД, платёжных подтверждений или обученных прогнозов.

create_all создаёт отсутствующие таблицы, но не мигрирует существующую структуру. Расширение требует версионируемых миграций, проверки данных, резервной копии и сценария отката. SQL старого черновика для рабочей базы неприменим.

9 Реализация программного продукта

9.1 Технологический стек

Таблица 9.1 — Используемые технологии

Компонент	Версия / технология	Назначение
Python	3.12.14 — автоматическая проверка	Основной серверный код; ручной Windows-запуск подтверждён на 3.15.1.
FastAPI / Uvicorn	0.141.1 / 0.52.1	REST API, валидация, OpenAPI и HTTP-сервер.
SQLAlchemy / SQLite	2.0.44 / встроенная SQLite	ORM, транзакции, локальное хранение.
React / TypeScript / Vite	18.3.1 / 5.7 / 6.4.5	Два интерфейса и сборка независимых HTML-входов.
openpyxl	3.1.5	Чтение и формирование XLSX.
python-docx / ReportLab	1.2.0 / 4.4.9	Шаблонные DOCX и PDF рейтинга.
python-jose / passlib / bcrypt	3.5.0 / 1.7.4 / 4.0.1	Токены доступа и хеширование паролей.
pytest / Playwright	8.4.2 / Chromium	Автоматические и браузерные проверки.
GitHub	zarunast/r, версия 1	Версионирование исходников, артефакты и документация.

Выбранный стек разделяет интерфейс, прикладные проверки и хранение данных. Версии зависимостей фиксируются в файлах проекта, а работоспособность определяется совместным запуском сборки, тестов и пользовательских сценариев.

9.2 Импорт и идентификация записей

Для чтения Excel используется openpyxl [16]. Импортер ищет строку заголовков среди первых 20 строк и использует нормализованные алиасы названий. Строка абитуриента сопоставляется по номеру, а заявление — по составному ключу конкурса. Идентификатор не преобразуется в число, что сохраняет ведущие нули.

Листинг 9.1 — Транзакция импорта ЕПГУ

```
def import_epgu(db, path, year, sheet=None):
   parsed, report = read_epgu(path, year, sheet)
   with db.uow() as session:
      applicants = {a.number: a for a in session.scalars(select(Applicant))}
      applications = {application_key(a): a for a in
   session.scalars(select(Application).where(Application.year == year))}
      updated = set()
      fresh = set()
      desired = {}
      conflicts = set()
```

Источник: admissions/importer.py, функция import_epgu; показан фрагмент реализации. Полный контекст приведён в приложении с исходным кодом.

В исходной выгрузке присутствует 60 столбцов, включая альтернативные предметы. Поэтому total_score принимается из готового столбца «Сумма баллов». Суммирование всех заполненных предметных колонок привело бы к двойному учёту альтернативных экзаменов. Для ручного пересчёта API принимает только выбранные exam_scores и отдельно achievements.

Листинг 9.2 — Составной ключ заявления (фрагмент фактического кода)

```
def application_key(obj):
    get = obj.get if isinstance(obj, dict) else lambda k:
getattr(obj, k)
    return tuple(get(k) for k in ("applicant_number", "year",
"campaign", "direction", "study_form", "quota"))
```

Листинг 9.3 — Пересчёт по выбранным экзаменам

```
def score_values(exams, achievements):
    if not exams or any(type(v) is not int or not 0 <= v <=
100 for v in exams.values()):
        raise ValueError("Укажите выбранные для конкурса
экзамены с баллами от 0 до 100")
    return sum(exams.values()) + achievements
```

В ходе анализа файла обнаружено расхождение персональных полей одного человека между строками. Импортер собирает итоговую карточку до сравнения с БД и последовательно использует последнюю строку файла, выдавая предупреждение. Локальные дополнительные поля профиля, например сведения о родителе, не удаляются.

9.3 Рейтинг и оценка положения

Доступ к связанным данным реализован средствами SQLAlchemy и SQLite [12, 13]. Рейтинг сортируется по убыванию итогового балла. Участникам с одинаковым баллом присваивается одинаковое место; номер используется только для устойчивого порядка строк, а не как официальное преимущественное право. Pending, withdrawn и rejected не включаются в действующий конкурс.

Листинг 9.4 — Оценка положения с учётом ничьих

```
  else:
    better = sum(r["score"] > a.total_score for r in rows)
    tied = sum(r["score"] == a.total_score for r in rows)
    if better >= seats.seats:
      result["assessment"] = "outside_seats"
    elif better + tied <= seats.seats:
      result["assessment"] = "within_seats"
    else:
      result["assessment"] = "tie_at_boundary"
    result["factors"].append("Не учитывает будущие заявления, отзыв согласий и
  правила разрешения равных баллов.")
```

Источник: admissions/services.py, функция forecast; показан фрагмент реализации. Полный контекст приведён в приложении с исходным кодом.

Для оценки положения используются B — количество участников с большим баллом, T — количество участников с равным баллом и S — план мест. Если $B \geq S$, заявление находится за пределами мест. Если $B + T \leq S$, оно находится внутри текущей границы. В остальных случаях возникает ничья на границе. Без плана или места в рейтинге возвращается unknown. Поле probability равно null.

9.4 Договоры, оплаты, звонки и пакет 1С

Листинг 9.5 — Изменение договора и запись аудита

```
def update_contract(session, contract, actor, *, location=None, stage=None,
    paid=None):
    changes = {}
    if location is not None and location not in LOCATIONS:
        raise ValueError("Неизвестное местоположение договора")
    if stage is not None and stage != contract.stage and stage not in
    TRANSITIONS[contract.stage]:
        raise ValueError("Недопустимый переход этапа договора")
    if paid is True and (stage or contract.stage) == "cancelled":
        raise ValueError("Нельзя подтвердить оплату отменённого договора")
    for field, value in {"location": location, "stage": stage, "paid": paid}.items():
        if value is not None and value != getattr(contract, field):
```

```
            changes[field] = {"from": getattr(contract, field), "to": value}
            setattr(contract, field, value)
    if changes:
        session.add(ContractEvent(contract_id=contract.id, actor=actor,
    changes=changes))
    return contract
```

Источник: admissions/services.py, функция update_contract; показан фрагмент реализации. Полный контекст приведён в приложении с исходным кодом.

Подстановка DOCX поддерживает абзацы, таблицы и колонтитулы, в том числе маркер, разбитый Word на несколько runs. Изменённый абзац получает текст в первом run; сложные текстовые поля и фигуры Word не поддерживаются. Юридический шаблон предоставляет университет.

Листинг 9.6 — Подтверждение оплаты

```
            continue
        try:
            contract = contracts.get(text(row[n]))
            if contract is None:
                raise ValueError("Договор не найден")
            if not boolean(row[p]) or contract.paid:
                report["unchanged"] += 1
                continue
            update_contract(session, contract, actor, paid=True)
            report["confirmed"] += 1
        except ValueError as error:
            report["errors"].append({"row": line, "error": str(error)})
    return report
finally:
    workbook.close()
```

Источник: admissions/services.py, функция reconcile_payments; показан фрагмент реализации. Полный контекст приведён в приложении с исходным кодом.

Бухгалтерский файл должен содержать «Номер договора» и «Оплачен». Отсутствие строки или значение «НЕТ» не отменяет ранее подтверждённую оплату. Частичные платежи и сверка сумм в MVP не моделируются.

Листинг 9.7 — Формирование очереди обзвона

```
def call_queue(session, year, change_deadline, payment_deadline, today=None):
    today = today or datetime.now(ZoneInfo("Europe/Moscow")).date()
    reasons = []
    if change_deadline - timedelta(days=7) <= today <= change_deadline:
        reasons.append("direction_deadline")
    if payment_deadline - timedelta(days=7) <= today <= payment_deadline:
        reasons.append("payment_deadline")
    if not reasons:
        return []
    apps = session.scalars(group_query(year,
```

```
    quota=Quota.PAID.value).where(Application.status.in_(ACTIVE)))
    contracts = {c.application_id: c for c in
    session.scalars(select(Contract).join(Application).where(Application.year ==
    year))}
    last_calls = {}
    for call in session.scalars(select(Call).join(Application).where(Application.year
     == year).order_by(Call.at, Call.id)):
        last_calls[call.application_id] = call
    result = []
    for app in apps:
        contract = contracts.get(app.id)
```

Источник: admissions/services.py, функция call_queue; показан фрагмент реализации. Полный контекст приведён в приложении с исходным кодом.

Очередь формируется в интервале от семи дней до указанного срока включительно. Приоритет интереса: 40 за согласие, 25 за первый приоритет и 30 за подписанный/зарегистрированный договор. Оплаченные и отменённые договоры, а также последний результат declined исключаются. Это эвристика, а не обученная вероятность заключения договора.

Листинг 9.8 — Отбор заявлений для экспорта в 1С

```
def export_1c(session, year, format="json"):
    records = []
    contracts = {c.application_id: c for c in session.scalars(select(Contract))}
    for app, person in session.execute(select(Application, Applicant).join(Applicant)
                                       .where(Application.year == year,
    Application.status == "enrolled",
                                              Application.enrolled_at.is_not(None))):
        contract = contracts.get(app.id)
        records.append({"applicant_number": person.number,
                        "full_name": " ".join(filter(None, [person.surname,
    person.first_name, person.patronymic])),
                        "passport_series": person.profile.get("Серия", ""),
                        "passport_number": person.profile.get("Номер", ""), "snils":
    person.profile.get("СНИЛС", ""),
                        "direction": app.direction, "study_form": app.study_form,
                        "enrolled_at": app.enrolled_at.date().isoformat(),
    "contract_number": contract.number if contract else None})
    if format == "json":
        return json.dumps({"schema_version": 1, "students": records},
    ensure_ascii=False).encode()
```

Источник: admissions/services.py, функция export_1c; показан фрагмент реализации. Полный контекст приведён в приложении с исходным кодом.

В пакет JSON/XML включаются только заявления с зачислением и внесённой датой приказа. Для платного зачисления проверяется зарегистрированный оплаченный договор, для бюджетного — согласие. Создание файла версии схемы 1 не означает интеграцию с конкретной информационной базой 1С.

9.5 Два сайта и общий источник данных

Клиентская часть реализована на React и собирается Vite [14, 15]. Vite собирает index.html для сотрудников и abiturient.html для поступающих. В режиме разработки запросы /api/* проксируются на Uvicorn; после сборки FastAPI отдаёт оба сайта и локальные assets. Сайт 1 хранит токен только в памяти вкладки. Сайт 2 использует credentials=omit и маршруты /public/*, не обращаясь к закрытому реестру.

Демонстрация сайта абитуриента включается параметром ?demo=1 и видимой полосой предупреждения. Настоящий режим не подставляет демо при отсутствии данных или сетевой ошибке. Чтобы увидеть импорт на втором сайте, нужно открыть настоящий режим и обновить страницу или конкурсный список.

9.6 Связь реализации с интерфейсом

Экранные формы и пошаговые сценарии перенесены в руководства раздела 12. Служебный и публичный сайты используют один API; проверка полномочий и доступа к персональным данным выполняется на сервере.

10 Тестирование

10.1 Стратегия и среда

Проверки реализованы средствами pytest и Playwright [18, 19]. Проверки строятся вокруг требований к достоверности данных и закрытости персональных сведений. Тесты используют временные SQLite-базы и синтетические записи. На момент подготовки отчёта команда python -m pytest -q завершилась результатом 26 passed, 2 warnings, 8.60 s на Python 3.12.14 в Linux. Предупреждения относятся к переходу тестового клиента Starlette и устаревающему модулю crypt в passlib; ошибок проверки нет.

Таблица 10.1 — Состав автоматических проверок

Файл	Количество	Что проверяется
tests/test_core.py	16	Импорт, пять категорий, рейтинг, договор, оплата, документы, backup, rollback и доступ.
tests/test_web.py	5	Внутренние представления, атомарное создание, журнал, фильтры и UTC.
tests/test_portal.py	5	Анонимный каталог, отсутствие ПДн, ничьи, ведущие нули, пустые данные и FAQ.
Итого	26	Все проверки завершились успешно.

10.2 Основные контрольные случаи

Таблица 10.2 — Контрольные сценарии (группировка проверок)

ID	Сценарий	Зафиксированный результат
TC-01	Один человек, несколько заявлений	Создан один Applicant и несколько Application.
TC-02	Повтор неизменённого Excel	Новые и обновлённые записи отсутствуют.
TC-03	Переименование и перестановка колонок	Используются семантические алиасы.

Продолжение таблицы 10.2

ID	Сценарий	Зафиксированный результат
TC-04	Ошибочная строка и дубль в файле	Возвращаются номера строк и причины; корректные строки сохраняются.
TC-05	Неизвестная категория	Ошибка, без автоматической подмены на общий конкурс.
TC-06	Пять категорий	Категория не меняет итоговый балл.
TC-07	Конфликт профилей и локальный родитель	Повторный импорт стабилен; локальные сведения сохраняются.
TC-08	Равные баллы и граница мест	Одинаковое место и tie_at_boundary.
TC-09	Запрещённый переход договора	Переход отклоняется; история остаётся согласованной.
TC-10	Повторное подтверждение оплаты	Второй импорт не создаёт новое подтверждение.
TC-11	DOCX с разорванным маркером	Значение корректно подставляется.
TC-12	XLSX с формулой в строковом поле	Значение экранируется при экспорте.
TC-13	Резервная копия и откат	Копия читается; ошибочная транзакция не сохраняется.
TC-14	Запрос реестра без токена	Отказ в доступе.
TC-15	Публичный ответ с контрольными ПДн в БД	Запрещённые поля и маркеры не появляются.
TC-16	Атомарное создание с неверным баллом	Не остаётся отдельной карточки без заявления.
TC-17	Зачисление и повторный импорт	Сохраняются внесённое зачисление и номер приказа.
TC-18	Платная группа с совпадающим названием	Публичный выбор сохраняет правильную категорию и форму.

10.3 Интеграционные и браузерные сценарии

Скрипт tests/browser_smoke.py запускает отдельный Uvicorn и проверяет реальную связку браузер → API → SQLite: навигацию демо, поиск и редактирование, план мест, звонки, CSV, мобильное меню, вход, создание заявления, импорт ЕПГУ, договор, оплату, скачивание XLSX и выход. Скрипт tests/browser_portal.py выполняет анонимный поиск и проверяет приватность, объяснение оценки, CSV, категории, отсутствующий год, FAQ, экранирование HTML и мобильную навигацию.

Оба сценария ранее завершились успешно и повторно выполнены при подготовке документации. В публичном сценарии нет запросов к служебным API и ошибок JavaScript. Контрольные персональные маркеры в тестовой базе не отображаются в ответах и DOM. Размеры проверенных экранов — 1440 × 1000 и 390 × 844; широкие таблицы прокручиваются внутри своего блока.

10.4 Проверка предоставленной выгрузки

Таблица 10.3 — Результаты обработки реального формата XLSX

Показатель	Наблюдение	Интерпретация
Строк данных / колонок	19 610 / 60	Файл пользователя, один лист.
Уникальные абитуриенты	3 506	Номер — строковый первичный ключ.
Заявления	19 610	Несколько заявлений одного абитуриента допустимы.
Конкурсные группы	301	Направление, кампания и форма обучения.
Ошибки первичного импорта	0	Для предоставленного файла.
Повторный импорт	19 610 неизменённых заявлений	Новые и обновлённые записи — 0.
Конфликт персональных полей	1 абитуриент	Выдано предупреждение, используется последняя строка.

Продолжение таблицы 10.3

Показатель	Наблюдение	Интерпретация
Время полного импорта	Около 7–9 секунд	Наблюдение текущей среды, не нагрузочная гарантия.

10.5 Производительность и непроверенные условия

Время импорта XLSX, время получения страницы реестра и время формирования рейтинга являются разными метриками. Измерение полного файла не подтверждает норматив отклика для 10 000 записей. Для проверки 500 одновременных запросов требуются профиль нагрузки, изолированный сервер, уровни перцентилей, число ошибок и мониторинг памяти/CPU. Такие результаты в данном проекте не получены и не подменяются расчётными таблицами.

На компьютере пользователя подтверждены установка зависимостей, создание пользователя, запуск сервера и открытие обоих сайтов на Windows с Python 3.15.1. Полный автоматический прогон на этой конфигурации не выполнялся. Для PDF текущий сервис ожидает шрифт DejaVu Sans по Linux-пути; Windows-экспорт PDF требует настройки переносимого пути к шрифту. Это известное ограничение платформенной проверки, а не подтверждённая совместимость всех функций с Windows.

11 Сопровождение и внедрение

11.1 План внедрения

Развёртывание в учебной локальной среде является первым этапом. Для использования вузом необходимо согласовать юридические основания обработки данных, правила квот и ничьих, шаблон договора, форматы бухгалтерии и спецификацию конкретной 1С. Доступ сотрудников и открытого портала разделяется на reverse proxy.

Таблица 11.1 — План перехода к опытной эксплуатации

Этап	Действие	Результат
Подготовка	Согласовать ответственных, инфраструктуру, категории и источники	Утверждённый регламент.
Контур проверки	Отдельная база и обезличенные сценарии	Проверка правил конкурса и ошибок.
Пилот	Ограниченное число сотрудников и выбранные направления	Замечания и протокол пилота.
Техническая настройка	HTTPS, LAN, постоянный JWT_SECRET, backup и мониторинг	Управляемый серверный запуск.
Приёмка	Проверить требования и ограничения по матрице	Протокол, перечень доработок.
Эксплуатация	Обучить сотрудников, организовать поддержку	Назначенные ответственные и журнал изменений.

11.2 Резервное копирование и восстановление

Команда backup использует штатный SQLite Backup API и проверяет integrity_check. Копирование одного файла при активном WAL не заменяет согласованный backup. Существующая копия не перезаписывается, а новый файл получает ограниченные права в поддерживающей их ОС. Ежедневный запуск в 02:00 по Москве назначается внешним планировщиком; программа не устанавливает cron автоматически.

Пример команды локальной резервной копии в Windows CMD

```
.venv\Scripts\python.exe -m admissions backup
backups\campaign-2026-01.sqlite3
```

Для восстановления останавливают службу, сохраняют текущую БД вместе с относящимися WAL/SHM, проверяют копию и заменяют файл согласованной резервной копией. Старые WAL/SHM заменённой базы не должны применяться к восстановленному файлу. После запуска проверяются /health, вход, число записей и публичный поиск.

11.3 Риски и развитие продукта

Таблица 11.2 — Риски и меры

Риск	Текущее состояние	Следующий шаг
Юридическая трактовка квот	Используется явная категория источника	Согласовать справочник и основания с правилами приёма.
Равные баллы	Одинаковое место, порядок строк по номеру	Реализовать официальное преимущественное право отдельно от технической сортировки.
Обученная модель	Истории зачислений и оценки качества нет	Подготовить данные, базовую модель, валидацию и мониторинг.
Миграции	create_all для первой схемы	Ввести Alembic и проверяемый откат.
Хранение 6 лет	Автоматического удаления нет	Утвердить сроки, основания, архив и доступ.
Интеграция 1С	Файлы JSON/XML	Согласовать схему, транспорт и подтверждение приёма.

Продолжение таблицы 11.2

Риск	Текущее состояние	Следующий шаг
Нагрузка	Функциональные тесты пройдены	Проверить целевые 10 сотрудников и 500 публичных запросов.
Переносимость PDF	В Linux шрифт найден, путь фиксирован	Настроить путь или поставку лицензированного шрифта для Windows.

11.4 Управление версиями

Состояние реализации зафиксировано коммитом 853721f856ed214c1dc5e5e06df877ec8eebfc4b. Документация и артефакты версий хранятся отдельно от рабочих баз. Обновление приложения должно сопровождаться резервной копией, проверкой зависимостей, пересборкой двух сайтов и регрессионными тестами. Персональные выгрузки и секретные значения не включаются в публичный репозиторий.

12 Документация и руководства

12.1 Руководство администратора: первый запуск

Ниже описан локальный запуск из распакованного полного архива. Корневой считается папка с admissions, frontend и requirements.txt. В готовом архиве уже есть frontend/dist/index.html и frontend/dist/abiturient.html. Если используются только исходники, предварительно выполняются npm ci и npm run build в frontend.

Таблица 12.1 — Первичная настройка Windows CMD

Шаг	Команда / действие	Ожидаемый результат
1	Открыть CMD в корневой папке проекта	Команда dir показывает requirements.txt.
2	py -3.15 -m venv .venv	Создано отдельное окружение; эта версия подтверждена пользовательским запуском.
3	.venv\Scripts\python.exe -m pip install -r requirements.txt	Зависимости установлены.
4	.venv\Scripts\python.exe -m admissions init-db	Создана data/admissions.sqlite3.
5	.venv\Scripts\python.exe -m admissions create-user admin	Пароль введён дважды; минимум 10 символов, максимум 72 байта UTF-8.
6	Задать JWT_SECRET из случайного значения, не менее 32 байт	Секрет хранится в окружении процесса или защищённом файле службы.
7	Запустить Uvicorn по команде ниже	Сервер отвечает на 127.0.0.1:8000.
8	Открыть /#/login и /abiturient/	Доступны оба сайта.

Команды локального запуска. Во второй строке требуется заменить обозначение настоящим случайным значением; это не готовый пароль.

```
.venv\Scripts\python.exe -c "import secrets;
print(secrets.token_urlsafe(48))"
set "JWT_SECRET=ВСТАВЬТЕ_СГЕНЕРИРОВАННОЕ_ЗНАЧЕНИЕ"
.venv\Scripts\python.exe -m uvicorn admissions.api:create_app
^
  --factory --host 127.0.0.1 --port 8000 --no-proxy-headers
```

Окно сервера остаётся открытым; остановка — Ctrl+C. Для следующего запуска не нужно повторно создавать БД и пользователя. Переменные DATABASE_URL и INTERNAL_NETWORKS задаются при необходимости; .env сам по себе автоматически не загружается. Для длительной эксплуатации

используется постоянный секрет окружения, чтобы перезапуск не аннулировал все действующие JWT.

12.2 Руководство сотрудника

1. Открыть http://127.0.0.1:8000/#/login, ввести созданный логин и пароль. Главная демонстрация не является рабочей базой; переход выполняется кнопкой «Подключить рабочую базу».

Введите логин и пароль созданной учётной записи и подтвердите вход. При ошибке проверьте раскладку клавиатуры и данные; повторный импорт или создание новой базы не требуется.

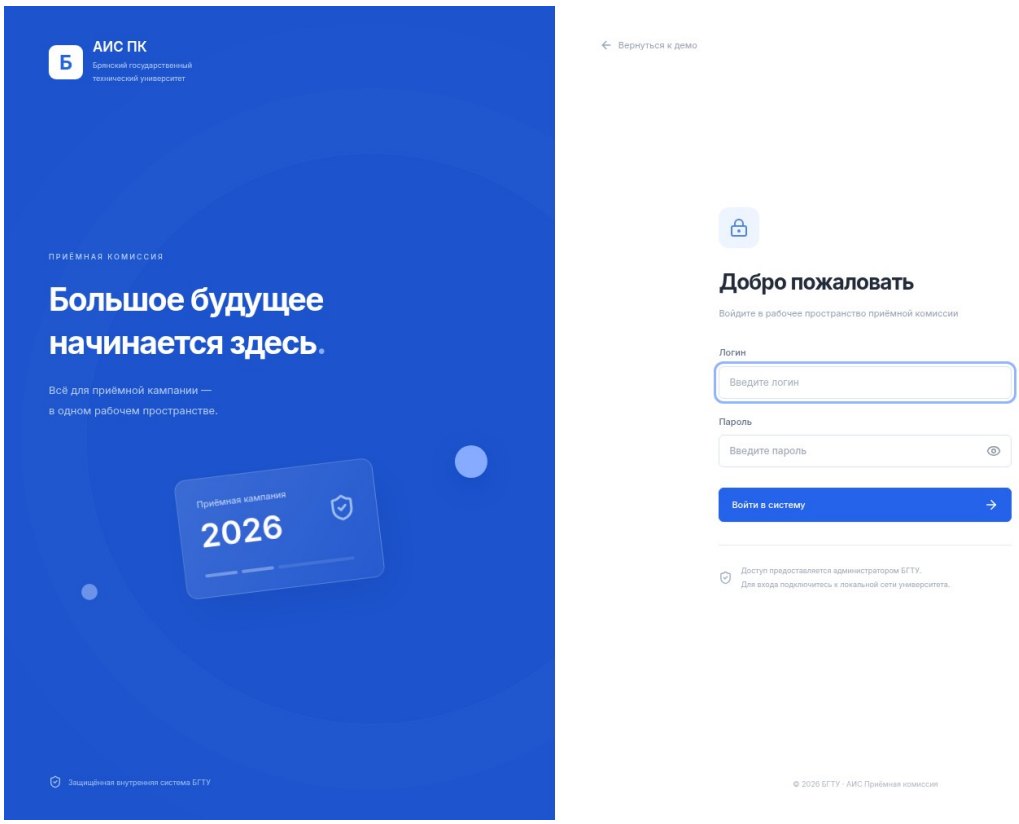


Рисунок 12.1 — Вход сотрудника в рабочую базу

Изображение из исходной курсовой; данные на экране являются демонстрационными или синтетическими.

После входа выберите год кампании. Демонстрационные показатели не являются сведениями из рабочей базы: перед обработкой реальных заявлений проверьте активный режим.

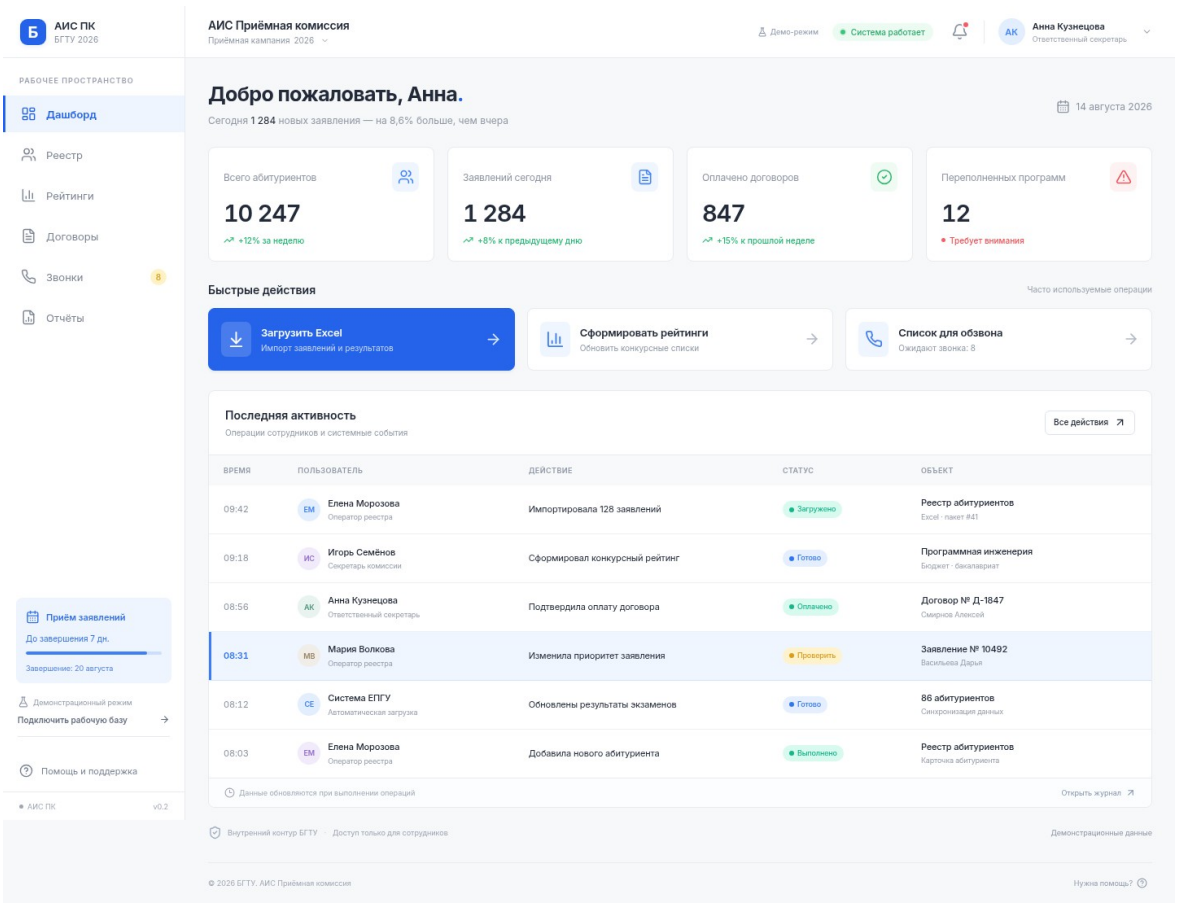


Рисунок 12.2 — Внутренний сайт: дашборд в обозначенном демо-режиме

Изображение из исходной курсовой; данные на экране являются демонстрационными или синтетическими.

2. Выбрать год кампании, нажать «Загрузить Excel», выбрать XLSX и при необходимости лист книги. После обработки проверить новые, обновлённые и ошибочные строки. Импорт может сохранить корректные строки при наличии ошибок других строк; отчёт нужно прочитать.

Выберите файл XLSX и нужный лист. До загрузки убедитесь, что номер абитуриента хранится строкой, а обязательные колонки присутствуют.

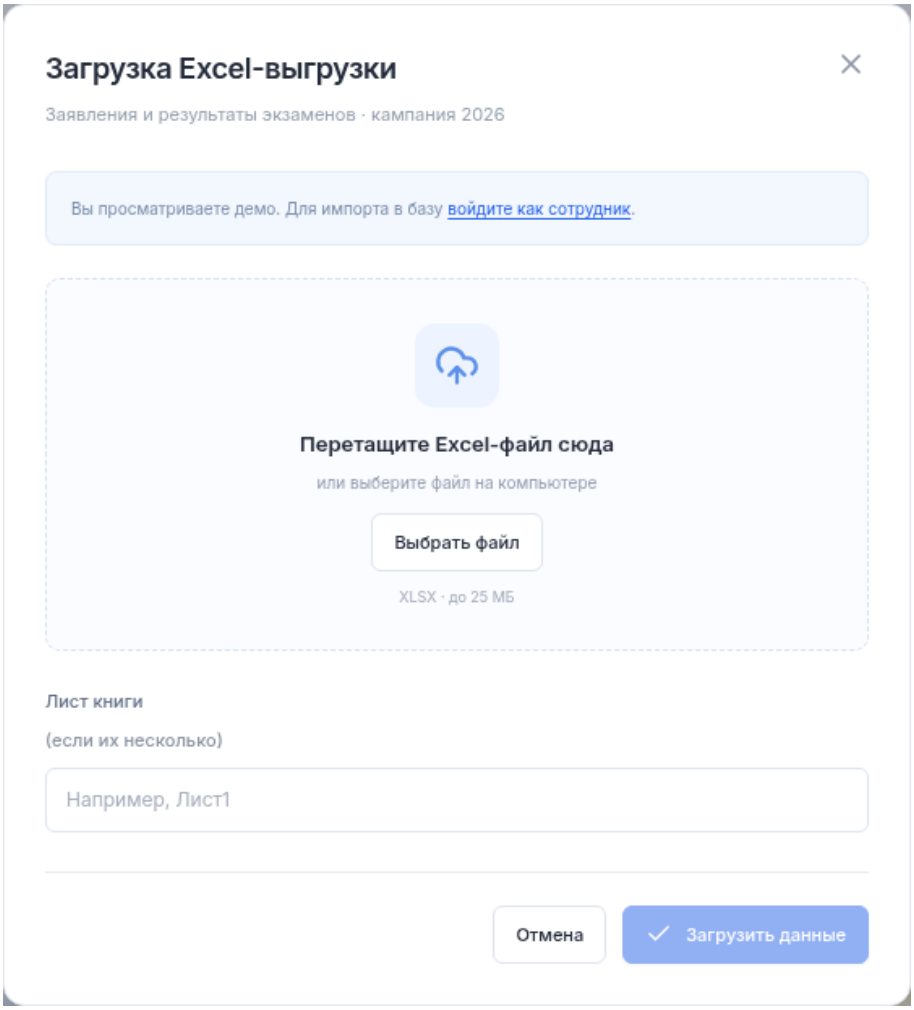


Рисунок 12.3 — Выбор файла Excel перед импортом

Изображение из исходной курсовой; данные на экране являются демонстрационными или синтетическими.

Прочитайте отчёт: количество новых, изменённых и ошибочных строк. Исправьте указанные строки исходного файла и повторите импорт; неизменённые заявления не должны дублироваться.

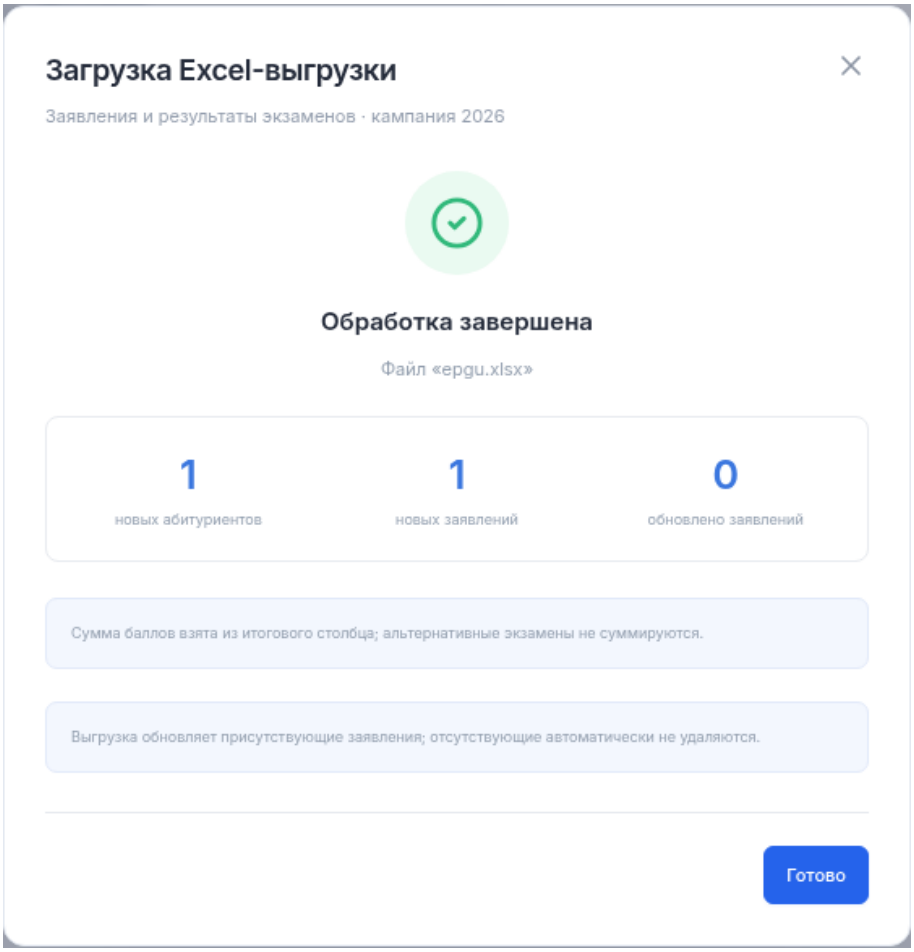


Рисунок 12.4 — Отчёт об импорте в рабочем режиме на синтетической базе

Изображение из исходной курсовой; данные на экране являются демонстрационными или синтетическими.

3. В «Реестре» найти номер или ФИО, применить фильтры направления, категории и статуса. Карточка показывает личные данные, документы и заявления. Редактирование контактов выполняется внутри карточки.

Введите ФИО или полный номер и примените фильтры. Если запись не найдена, проверьте выбранный год и снимите лишние фильтры.

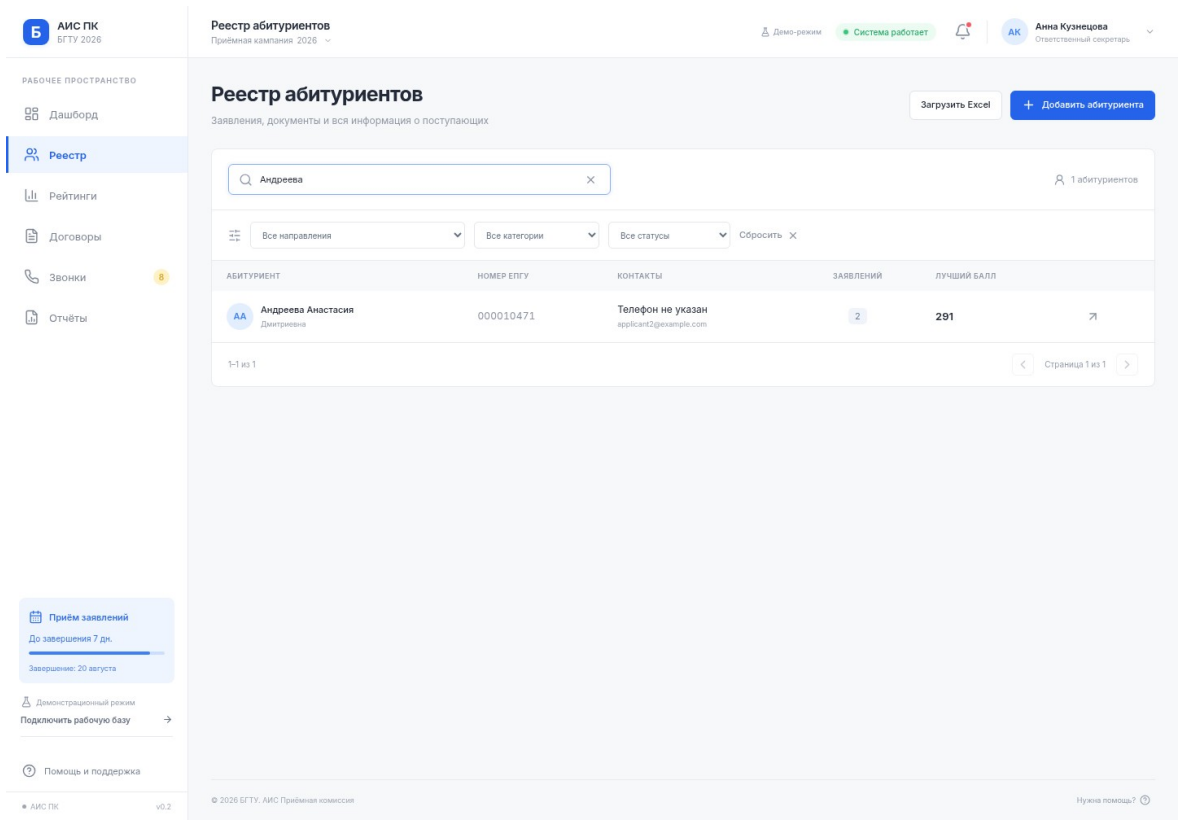


Рисунок 12.5 — Реестр: фильтрация и поиск абитуриента

Изображение из исходной курсовой; данные на экране являются демонстрационными или синтетическими.

Откройте карточку найденного человека, внесите разрешённые изменения и сохраните. Повторно откройте карточку, чтобы проверить сохранение контактов и связанных заявлений.

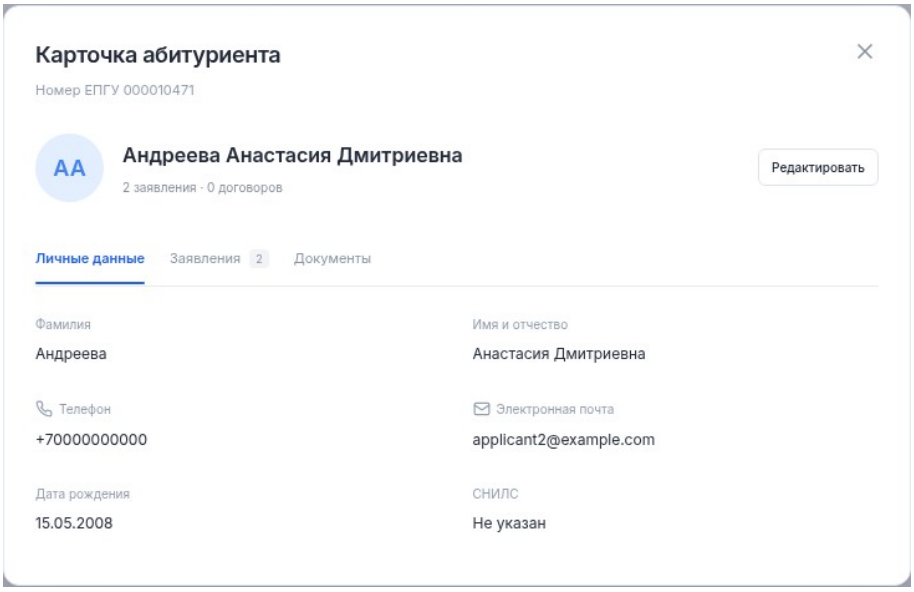


Рисунок 12.6 — Карточка абитуриента: сохранённые контактные сведения

Изображение из исходной курсовой; данные на экране являются демонстрационными или синтетическими.

4. В «Рейтингах» выбрать полную конкурсную группу и категорию, задать фактический план мест. Заполнение плана позволяет публичному порталу объяснять положение абитуриента.

Выберите год, кампанию, направление, форму обучения и категорию. Укажите утверждённый план мест. При равных баллах проверьте пояснение пограничной ничьей.

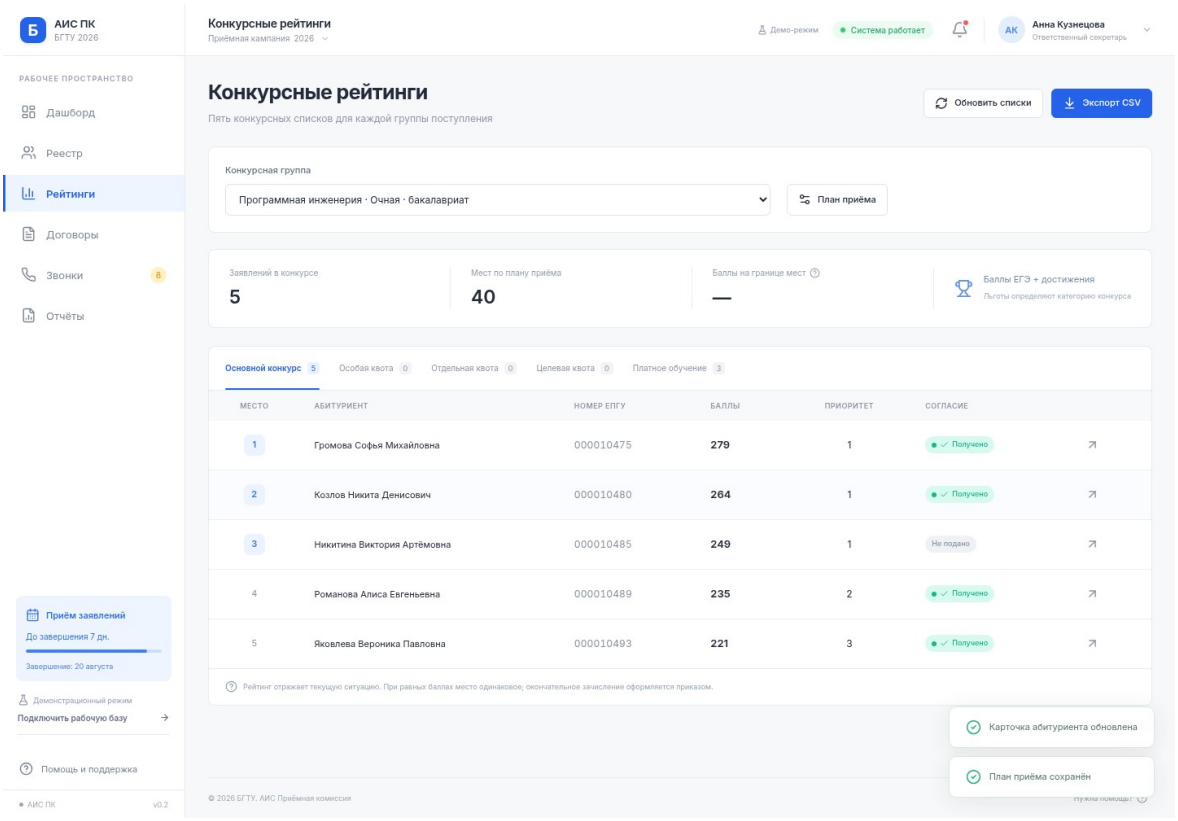


Рисунок 12.7 — Конкурсные рейтинги внутреннего сайта

Изображение из исходной курсовой; данные на экране являются демонстрационными или синтетическими.

5. В «Договорах» найти платное заявление, создать уникальный номер, управлять местоположением и допустимым этапом. Для Word загрузить согласованный DOCX-шаблон с маркерами. Для оплаты загрузить бухгалтерский Excel.

Найдите платное заявление, проверьте номер договора и допустимый этап. Местоположение и этап меняются независимо; история показывает автора и время изменения.

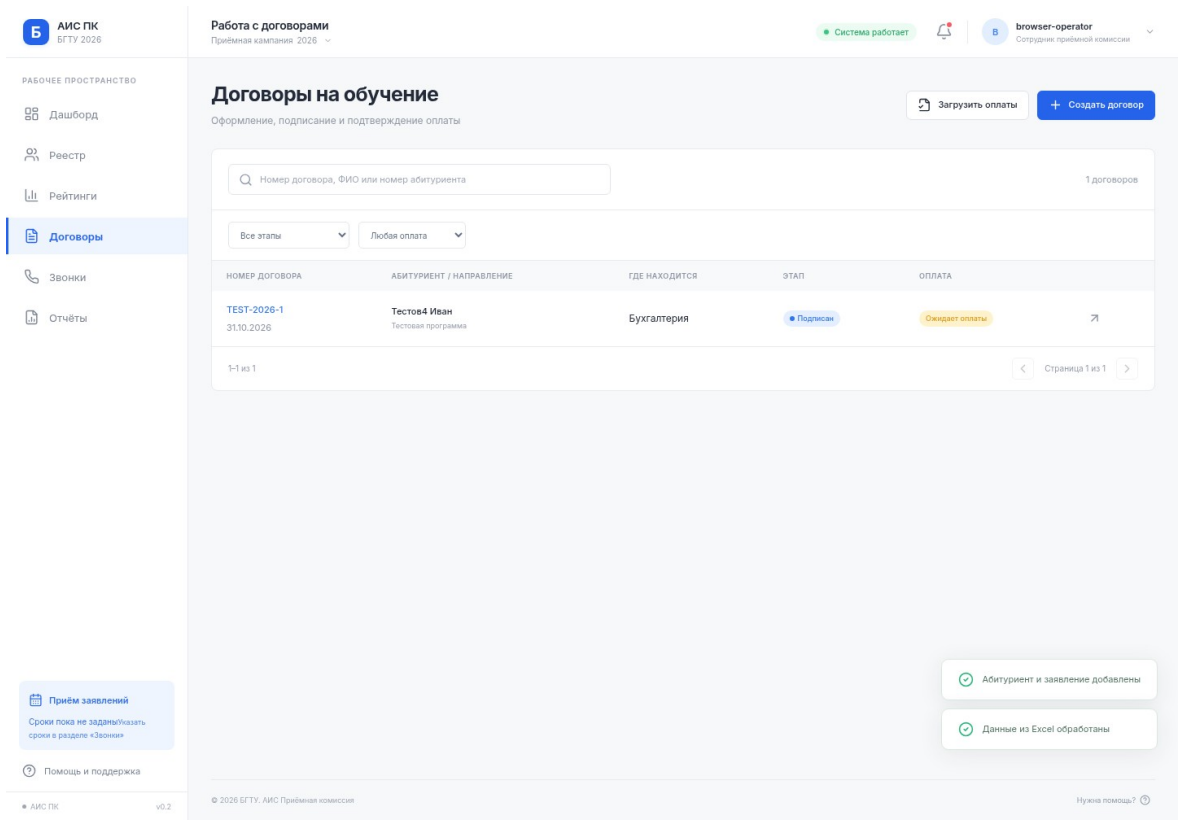


Рисунок 12.8 — Договор: этап «Подписан» после изменения местоположения

Изображение из исходной курсовой; данные на экране являются демонстрационными или синтетическими.

Загрузите бухгалтерский файл с подтверждениями полной оплаты. Проверьте отчёт сверки и обновлённый признак оплаты. Отсутствие договора в файле не отменяет ранее подтверждённый платёж.

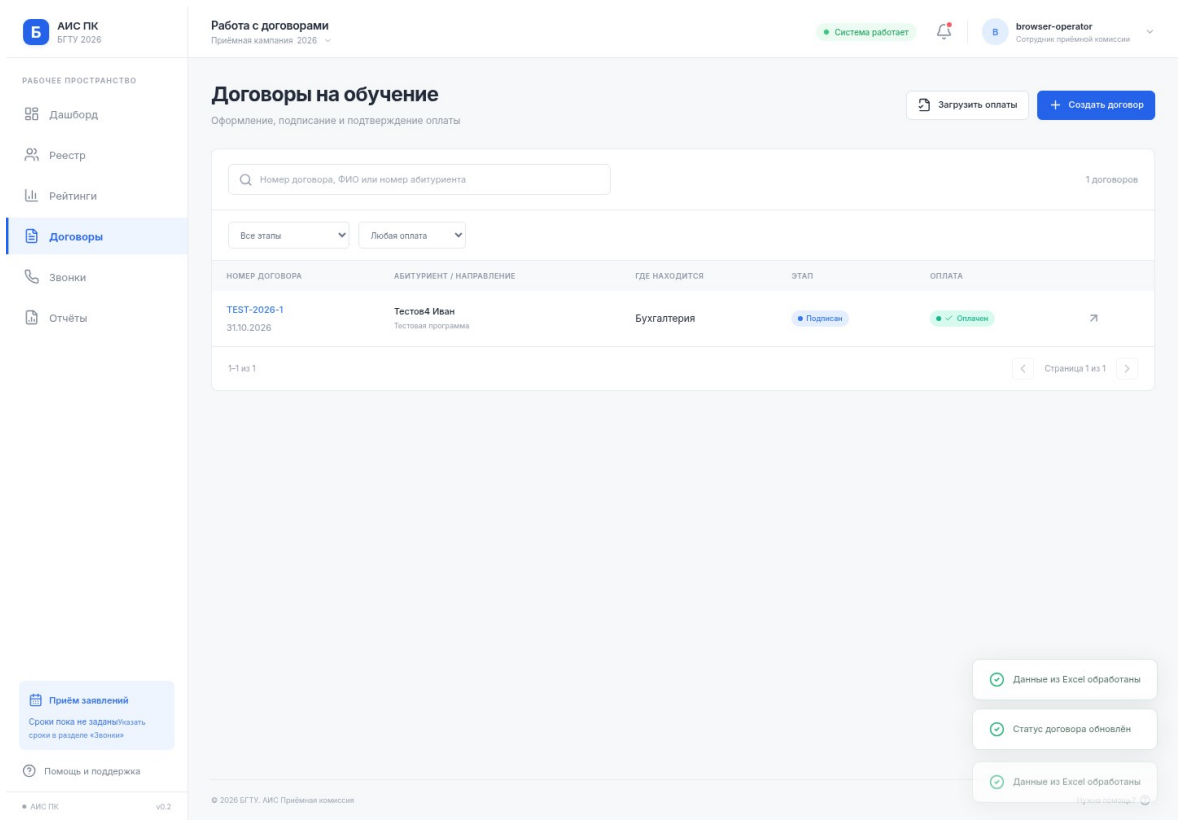


Рисунок 12.9 — Договор: результат загрузки сведений об оплате

Изображение из исходной курсовой; данные на экране являются демонстрационными или синтетическими.

6. В «Звонках» указать реальные сроки кампании. Очередь появляется в пределах семи дней до срока. Сотрудник выполняет звонок самостоятельно и сохраняет результат и комментарий.

Задайте сроки изменения направления и оплаты, откройте очередь и сохраните результат разговора. После обновления страницы проверьте, что результат сохранился.

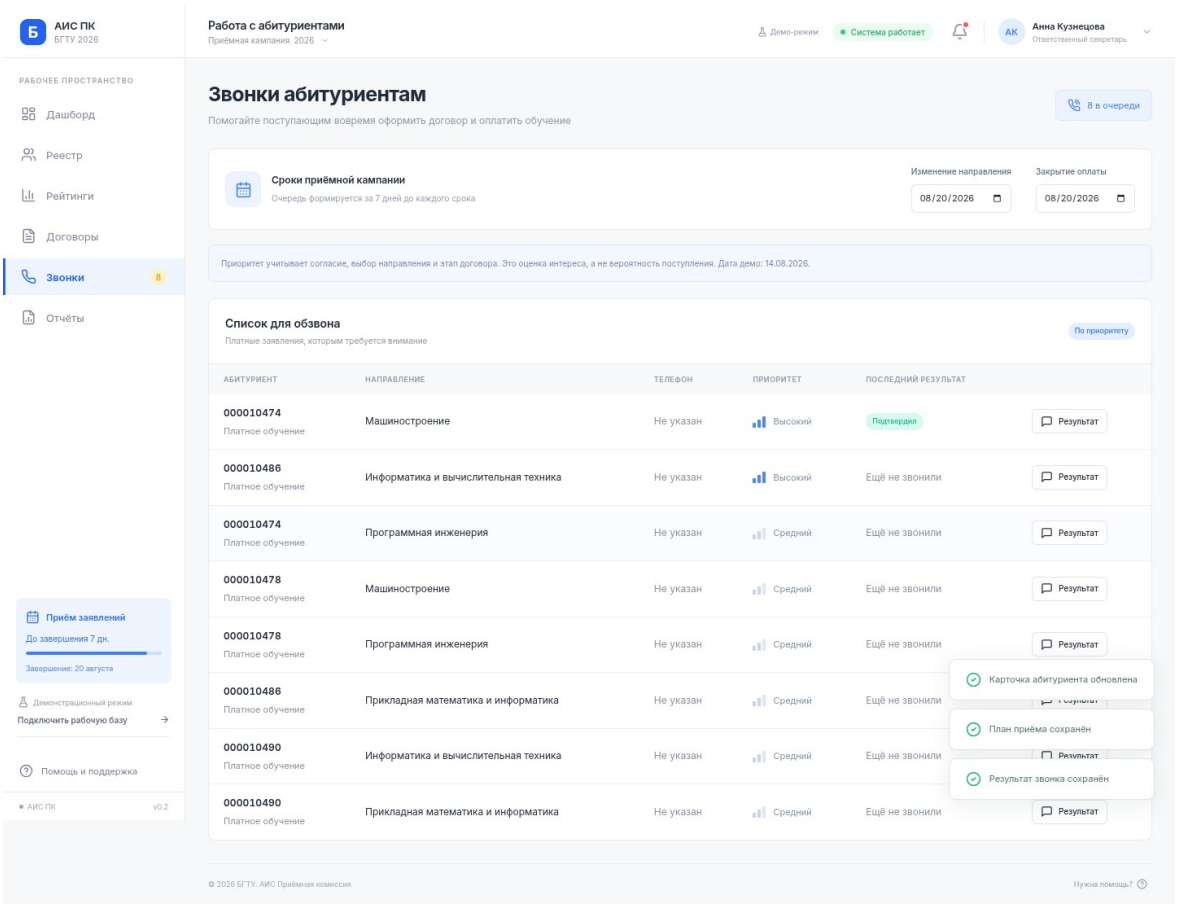


Рисунок 12.10 — Список звонков: сохранённый результат разговора

Изображение из исходной курсовой; данные на экране являются демонстрационными или синтетическими.

7. В «Отчётах» выбрать конкурс, категорию и формат. Выгрузка 1С содержит персональные данные и предназначена для внутренней передачи после внесения приказа. Не следует публиковать её вместе с открытыми рейтингами.

Выберите отчёт и необходимые параметры конкурса. Проверьте открытие загруженного файла. Выгрузка 1С предназначена для внутренней передачи и содержит персональные сведения.

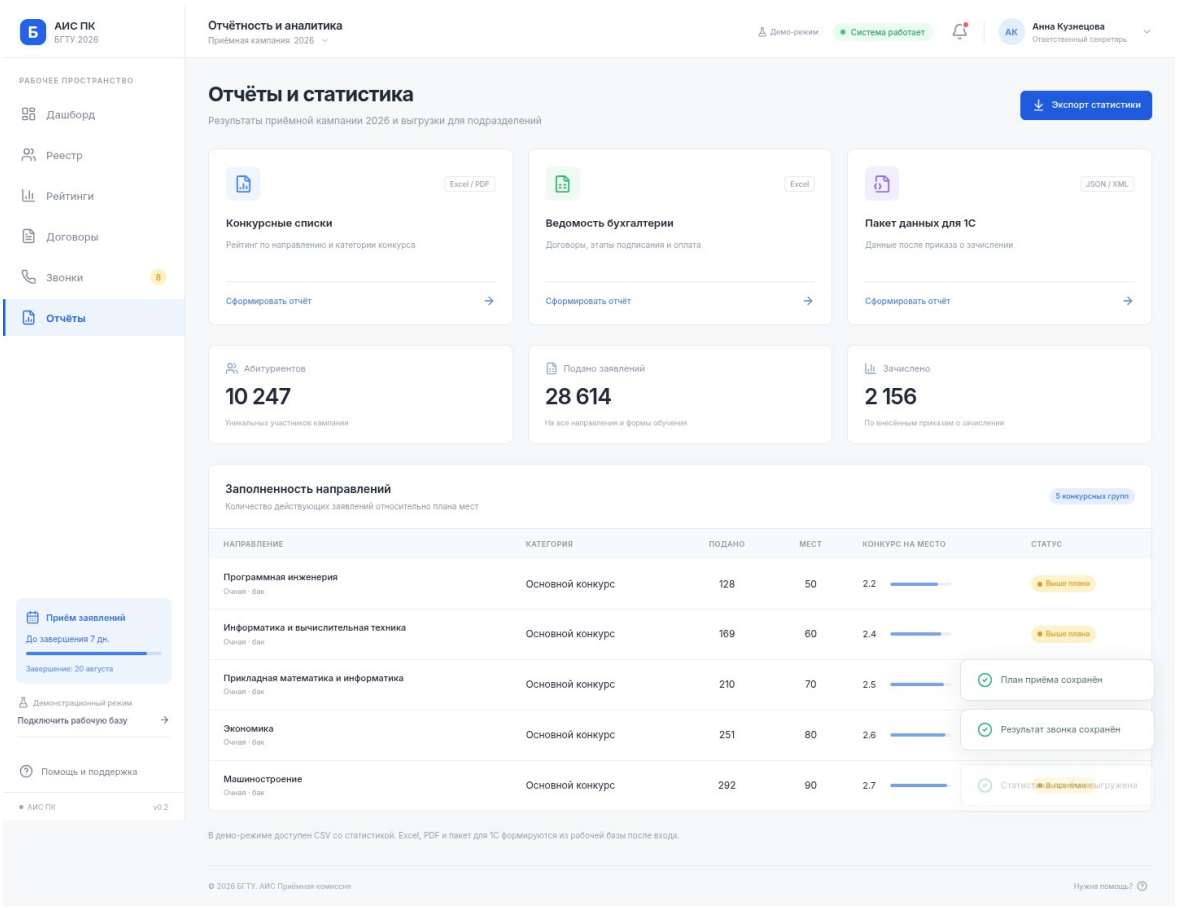


Рисунок 12.11 — Раздел отчётов и выгрузки

Изображение из исходной курсовой; данные на экране являются демонстрационными или синтетическими.

12.3 Руководство абитуриента

Открыть /abiturient/ без параметра demo. Для работы не требуется логин. Выбрать год и ввести номер из заявления ЕПГУ полностью, сохраняя ведущие нули.

Откройте публичный портал без входа. Для рабочих данных отключите демонстрационный режим и выберите соответствующий год приёма.

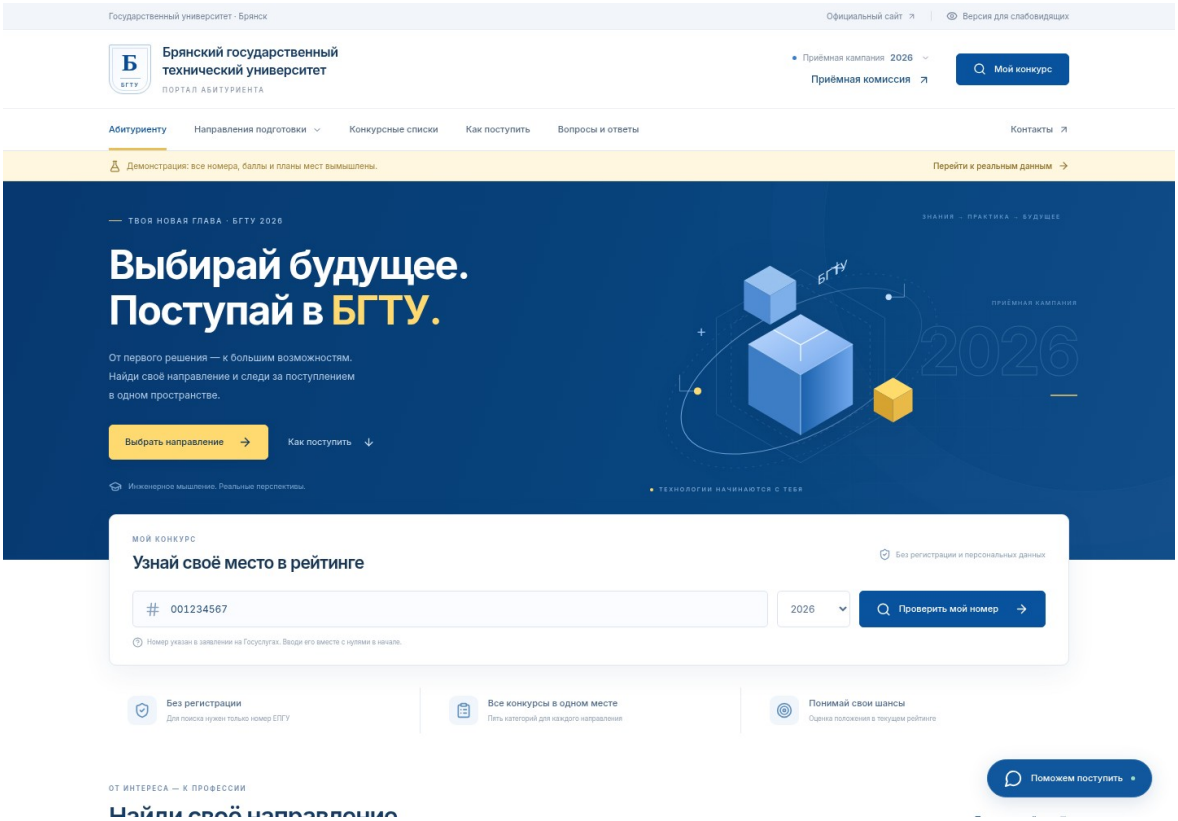


Рисунок 12.12 — Главная публичного портала, демо-режим

Изображение из исходной курсовой; данные на экране являются демонстрационными или синтетическими.

На узком экране используйте адаптивное меню; поля ввода должны оставаться доступными без горизонтальной прокрутки.

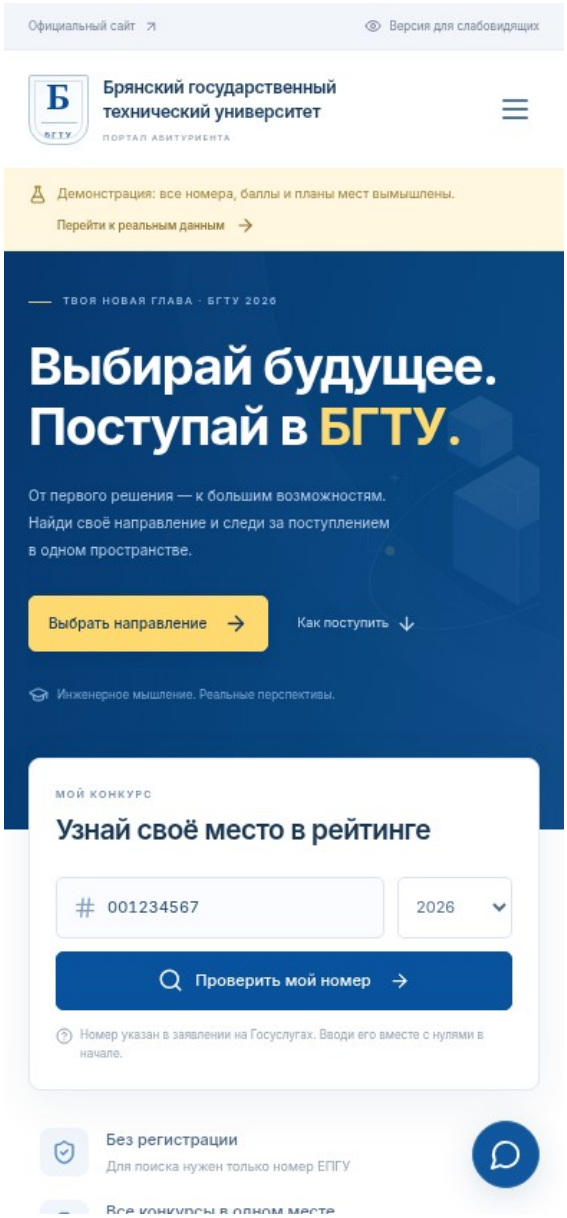


Рисунок 12.13 — Публичный портал на мобильном экране

Изображение из исходной курсовой; данные на экране являются демонстрационными или синтетическими.

Нажать «Проверить мой номер». В результате отображаются заявления, итоговый балл, место и план мест. Кнопка открытия списка переводит к соответствующей группе; номер выделяется в таблице.

Введите номер ЕПГУ полностью, включая ведущие нули. После поиска сопоставьте направление, баллы, место и число мест. Оценка не заменяет официальный приказ.

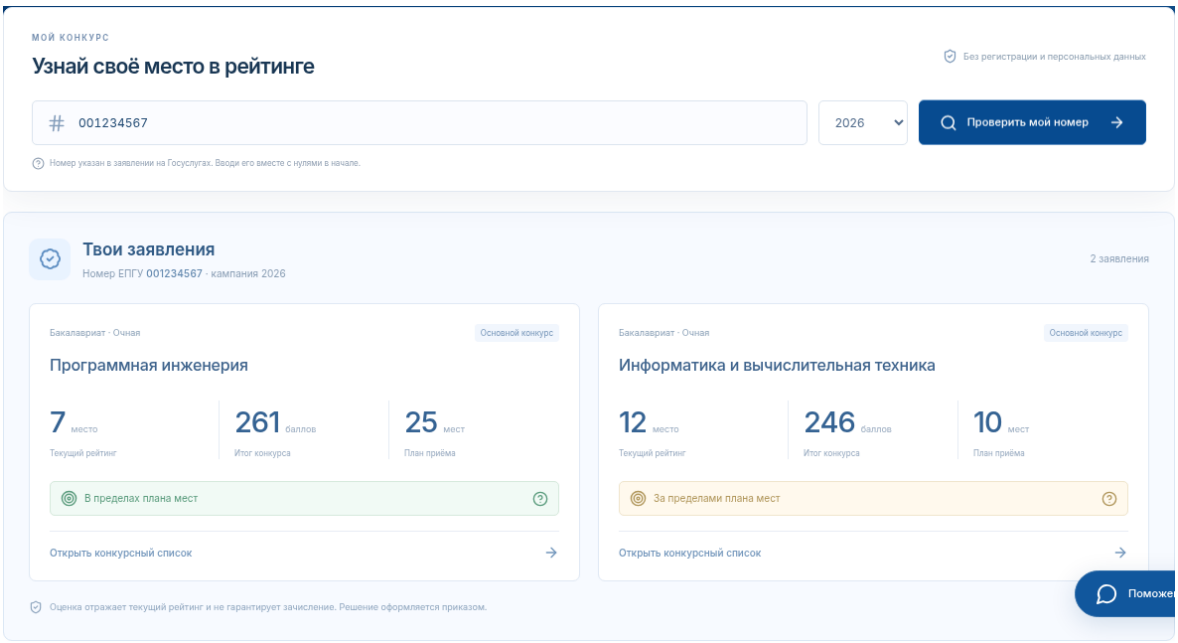


Рисунок 12.14 — Поиск по номеру и объяснимая оценка положения, демо

Изображение из исходной курсовой; данные на экране являются демонстрационными или синтетическими.

Откройте конкурсный список и найдите свой номер. Публичная таблица не должна показывать ФИО, контакты или документы других абитуриентов.

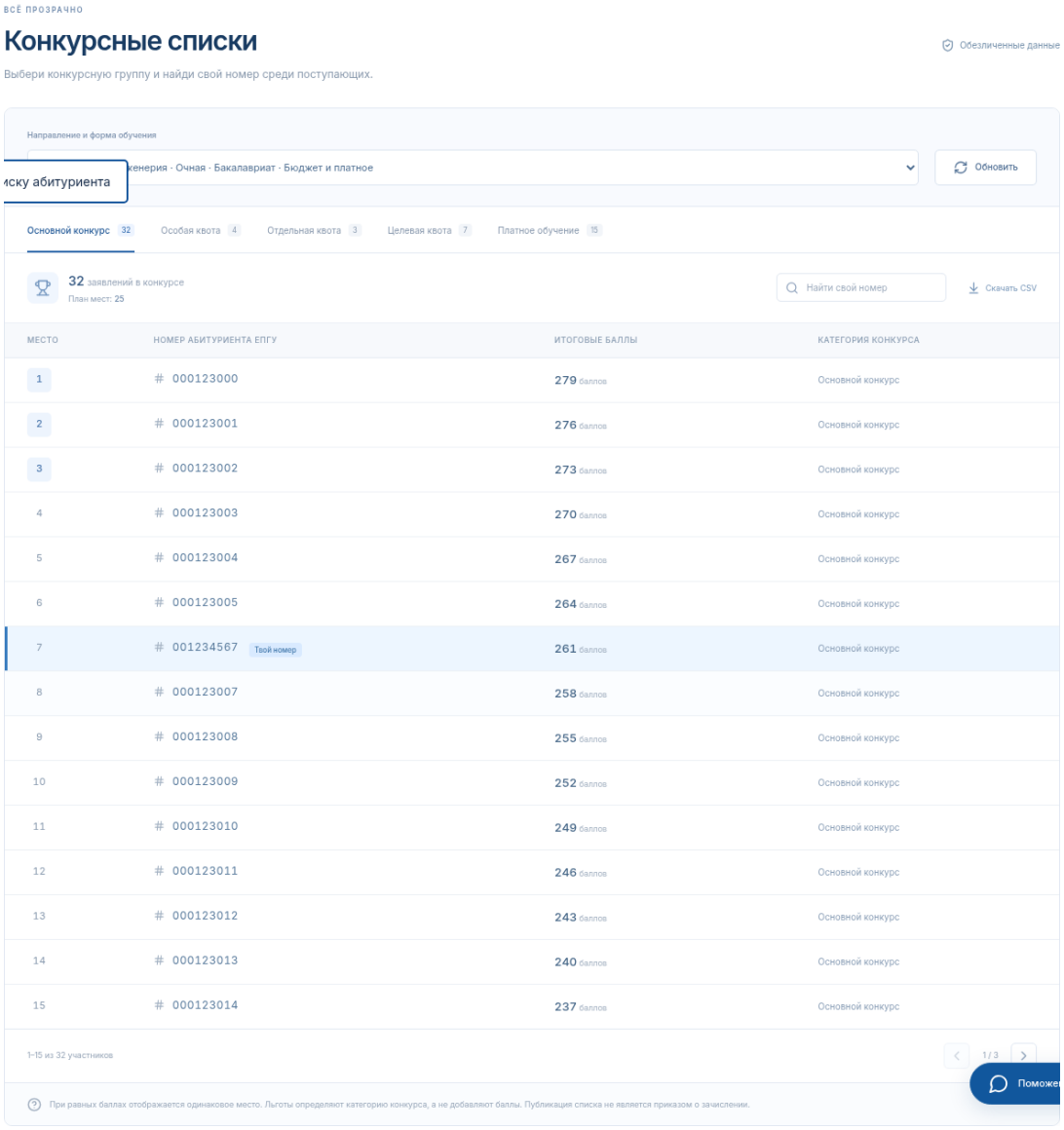


Рисунок 12.15 — Обезличенная конкурсная таблица публичного сайта

Изображение из исходной курсовой; данные на экране являются демонстрационными или синтетическими.

При отсутствии плана отображается «Недостаточно данных». Оценка внутри текущей границы не гарантирует зачисление. Официальное решение определяется приказом и действующими правилами приёма.

Для просмотра примера можно включить ?demo=1. Жёлтая полоса сообщает, что номера и баллы вымышлены. После импорта во внутреннем сайте реальные списки обновляются кнопкой «Обновить» или перезагрузкой страницы.

Помощник отвечает на типовые вопросы. В сообщении не следует указывать паспорт, СНИЛС и контакты; индивидуальные условия и юридические вопросы уточняются в приёмной комиссии.

12.4 Руководство программиста и API

Интерактивная спецификация доступна на /docs, JSON-схема — на /openapi.json. Закрытые методы используют Authorization: Bearer <token>; публичные методы находятся под /public/. Ошибки ввода возвращают 400/422, неизвестная запись — 404, конфликт уникальности — 409, отсутствие действительного токена — 401, нарушение сетевого ограничения — 403.

Таблица 12.2 — Группы API

Метод	Путь	Доступ	Назначение
GET	/health	Открытый	Health

Продолжение таблицы 12.2

Метод	Путь	Доступ	Назначение
POST	/auth/token	Локальная сеть; логин/пароль	Login
POST	/imports/epgu	JWT + локальная сеть	Epgu
GET	/applicants	JWT + локальная сеть	Applicants
POST	/applicants	JWT + локальная сеть	Add Applicant
GET	/applicants/{number}	JWT + локальная сеть	Applicant
PATCH	/applicants/{number}	JWT + локальная сеть	Edit Applicant
POST	/applications	JWT + локальная сеть	Add Application
POST	/registry	JWT + локальная сеть	Register
GET	/registry	JWT + локальная сеть	Registry
PATCH	/applications/{application_id}	JWT + локальная сеть	Edit Application
PUT	/seat-plans	JWT + локальная сеть	Plan
GET	/public/rankings	Открытый	Public Rankings
GET	/public/applicants/{number}	Открытый	Public Applicant
GET	/public/faq	Открытый	Faq
POST	/contracts	JWT + локальная сеть	New Contract
GET	/contracts	JWT + локальная сеть	Contracts
PATCH	/contracts/{contract_id}	JWT + локальная сеть	Contract Status

Продолжение таблицы 12.2

Метод	Путь	Доступ	Назначение
GET	/contracts/{contract_id}/history	JWT + локальная сеть	History
POST	/contracts/{contract_id}/document	JWT + локальная сеть	Contract Document
POST	/imports/payments	JWT + локальная сеть	Payments
GET	/calls/queue	JWT + локальная сеть	Calls
POST	/calls	JWT + локальная сеть	Log Call
POST	/applications/{application_id}/enroll	JWT + локальная сеть	Enroll
GET	/reports/statistics	JWT + локальная сеть	Stats
GET	/reports/ranking	JWT + локальная сеть	Report
GET	/reports/accounting	JWT + локальная сеть	Accounting
GET	/exports/1c	JWT + локальная сеть	One C
GET	/auth/me	JWT + локальная сеть	Me
GET	/dashboard	JWT + локальная сеть	Dashboard
GET	/activities	JWT + локальная сеть	Activities
GET	/catalog/groups	JWT + локальная сеть	Groups
GET	/rankings	JWT + локальная сеть	Rankings
GET	/public/catalog	Открытый	Catalog

Для внесения изменений в интерфейс редактируются frontend/src и frontend/src/portal. После npm run build сервер отдаёт новые assets; перед запуском должна существовать сборка. В режиме разработки Vite использует прокси /api на Python-сервер. Статические файлы не содержат рабочую базу; SQLite остаётся на сервере.

12.4.1 Подготовка рабочего окружения

Используйте Python 3.11+ и Node.js 20.19+ либо 22.12+. Создайте виртуальное окружение, установите requirements-dev.txt, затем выполните npm ci и npm run build в frontend. До запуска API задайте JWT_SECRET длиной не менее 32 байт.
Файл .env автоматически не загружается.

12.4.2 Проверка изменений

В корне проекта выполните .venv/bin/python -m pytest -q. Для интерфейса выполните npm run build в frontend. После запуска проверьте /health, открытие обоих сайтов и представительный сценарий сохранения данных. Тесты работают с временными базами и синтетическими сведениями.

12.4.3 Безопасное обновление

Перед изменением структуры базы выполните резервное копирование. create_all не заменяет миграции существующей базы. После обновления проверьте целостность данных, права доступа и экспорт документов; не редактируйте рабочую SQLite-базу одновременно с запущенным сервером.

Поясняющая схема: Транзакционная обработка импорта.

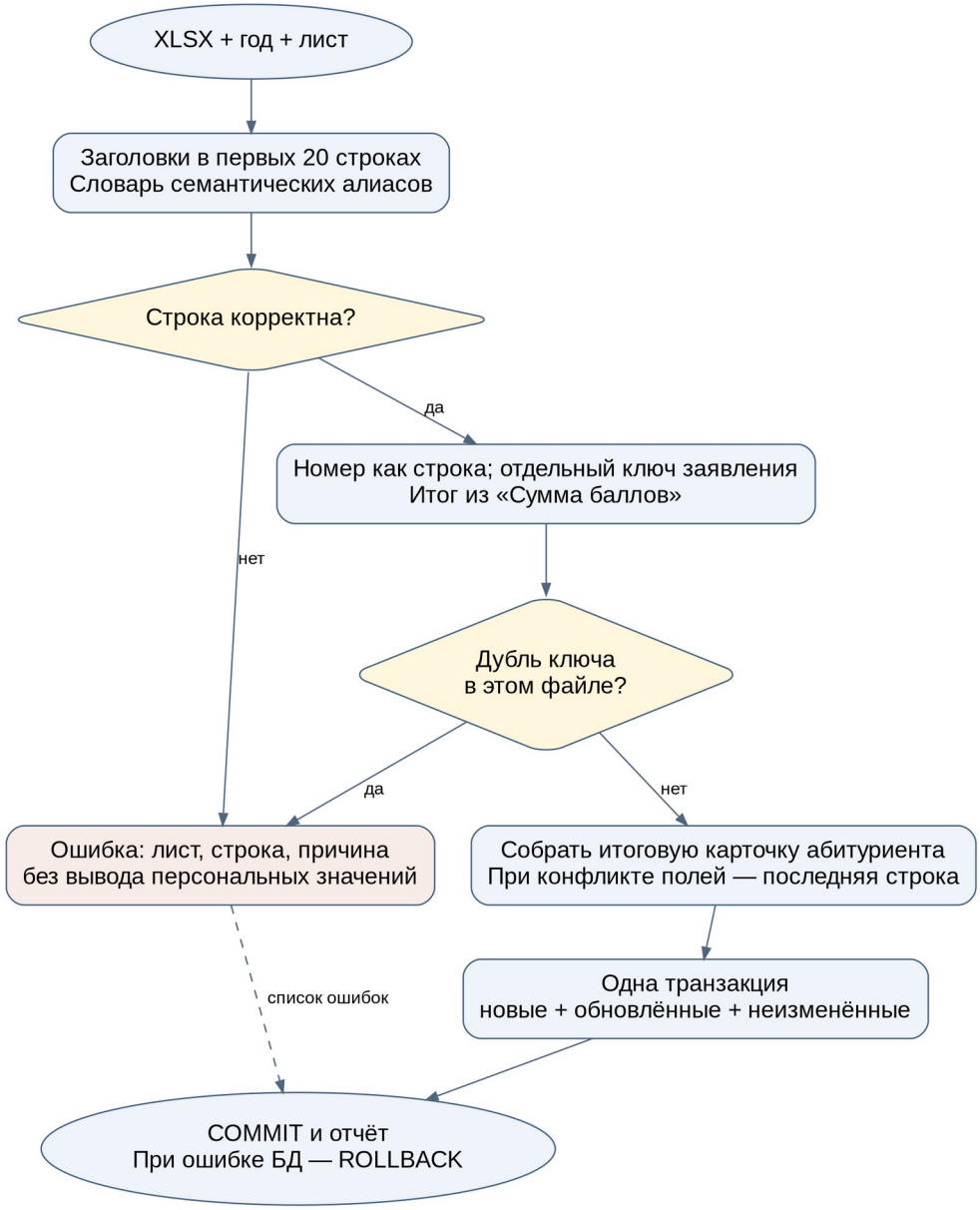


Рисунок 12.16 — Транзакционная обработка импорта

Поясняющая схема: Логика оценки конкурсного положения.

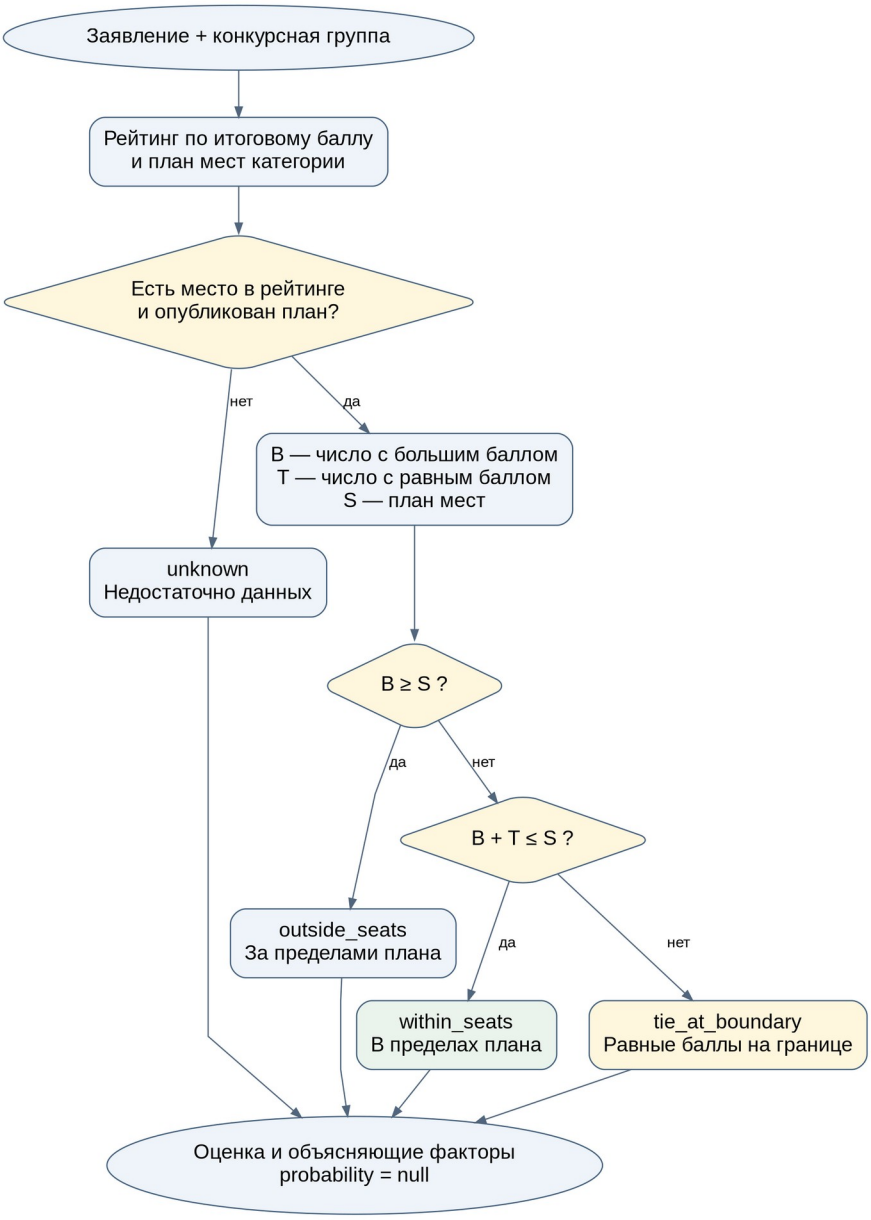


Рисунок 12.17 — Логика оценки конкурсного положения

Поясняющая схема: Изменение договора и аудит.

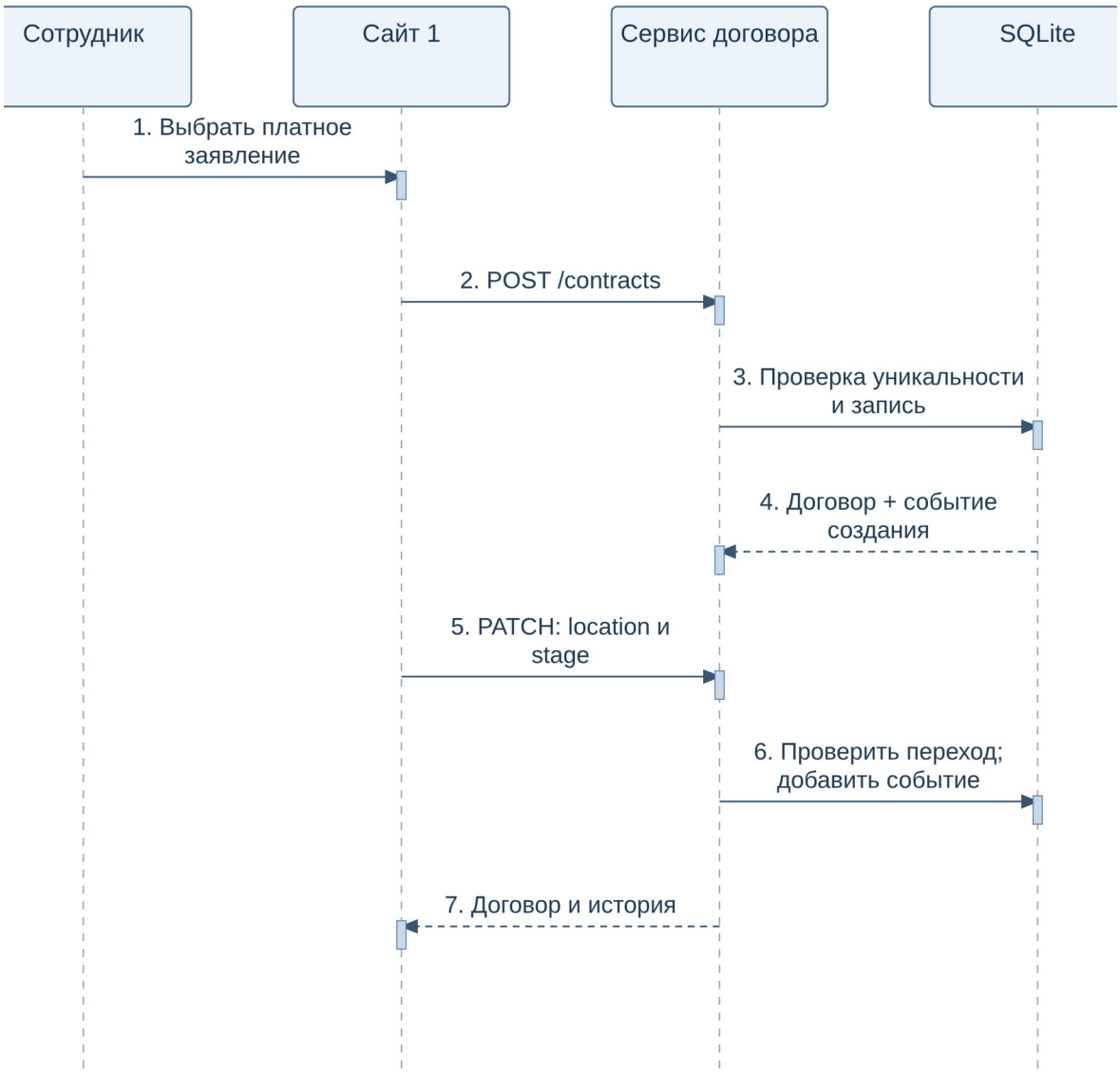


Рисунок 12.18 — Изменение договора и аудит

12.5 Диагностика типовых затруднений

Таблица 12.3 — Сообщения и действия

Ситуация	Причина	Действие
Путь не является командой	Папка введена без cd	Выполнить cd /d "путь" или открыть CMD в корне проекта.
requirements.txt не найден	Выбран внешний каталог распакованного ZIP	Перейти в папку, где находятся admissions и frontend.
Not Found на сайте	Нет frontend/dist или сервер запущен до сборки	Проверить два HTML-файла и перезапустить сервер.
Соединение отклонено	Uvicorn остановлен или порт отличается	Проверить окно сервера и /health.

Ситуация	Причина	Действие
После импорта видны примеры	Включён демо-режим	Войти в рабочую базу на сайте 1; убрать ?demo=1 на сайте 2.
Номер не найден	Другой год или потеря ведущих нулей	Сопоставить номер со столбцом «Идентификатор».
Нет оценки положения	Не опубликован план мест	Задать план для полного ключа конкурса.
Ошибка PDF в Windows	Нет шрифта по Linux-пути	Настроить переносимый путь шрифта в отдельной доработке.

Заключение

В ходе курсового проекта разработана АИС приёмной комиссии БГТУ, объединяющая Python-ядро, внутреннее рабочее место и открытый портал абитуриента. Единая база данных обеспечивает согласованность заявлений, договоров и публичных конкурсных списков.

Практически значимыми решениями стали разделение абитуриента и заявления, сохранение строкового номера ЕПГУ, семантический импорт, использование готового итогового балла источника, атомарные изменения связанных объектов и отдельная публичная сериализация без персональных данных.

Подтверждены обработка предоставленной выгрузки, устойчивость повторного импорта, 26 автоматических проверок и два браузерных сценария. Локальная установка и запуск обоих сайтов на Windows подтверждены ручной проверкой. Это подтверждает работоспособность основных сценариев прототипа, но не промышленное внедрение и не выполнение всех нагрузочных нормативов.

Документация соответствует версии 1. Код ядра приведён в приложении А, SQL — в приложении Б, структура проекта и результаты проверки — в приложении В, план спринтов — в приложении Г. Расширенная проектная модель и старые эскизы сохранены в архиве материалов. Для развития системы требуется согласование правил конкурса, переносимость PDF, миграции БД, регламент хранения, нагрузочное испытание и интеграционный контур 1С.

Отдельным направлением развития является обученная модель прогнозирования на исторических данных. В текущем продукте используется объяснимая расчётная оценка, что позволяет демонстрировать полезное поведение без неподтверждённых вероятностей и обещаний зачисления.

Список используемых источников

1. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления.
2. ГОСТ 19.201–78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению.
3. Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных».
4. Федеральный закон от 29.12.2012 № 273-ФЗ «Об образовании в Российской Федерации».
5. Sommerville I. Software Engineering. 10th ed. Pearson, 2016.
6. ГОСТ Р 2.105–2019. Единая система конструкторской документации. Общие требования к текстовым документам.
7. ГОСТ Р 2.104–2023. Единая система конструкторской документации. Основные надписи.
8. Репозиторий АИС приёмной комиссии БГТУ. URL: https://github.com/zarunast/r (дата обращения: 31.10.2026). Зафиксированное состояние: 853721f856ed214c1dc5e5e06df877ec8eebfc4b.
9. Прототип автора в Figma. URL: https://www.figma.com/design/ijwZl38V5hKkz6Gzsbakcc/Untitled?node-id=0-1 (дата обращения: 31.10.2026). Ссылка сохранена из исходного материала.
10. Python Documentation. URL: https://docs.python.org/3/ (дата обращения: 31.10.2026).
11. FastAPI Documentation. URL: https://fastapi.tiangolo.com/ (дата обращения: 31.10.2026).
12. SQLAlchemy 2.0 Documentation. URL: https://docs.sqlalchemy.org/en/20/ (дата обращения: 31.10.2026).
13. SQLite Documentation. URL: https://www.sqlite.org/docs.html (дата обращения: 31.10.2026).
14. React Documentation. URL: https://react.dev/ (дата обращения: 31.10.2026).
15. Vite Documentation. URL: https://vite.dev/guide/ (дата обращения: 31.10.2026).
16. openpyxl Documentation. URL: https://openpyxl.readthedocs.io/ (дата обращения: 31.10.2026).
17. python-docx Documentation. URL: https://python-docx.readthedocs.io/ (дата обращения: 31.10.2026).
18. pytest Documentation. URL: https://docs.pytest.org/ (дата обращения: 31.10.2026).
19. Playwright for Python Documentation. URL: https://playwright.dev/python/ (дата обращения: 31.10.2026).
20. БГТУ. Абитуриенту : официальный сайт. URL: https://www.tu-bryansk.ru/abiturient/ (дата обращения: 31.10.2026). Референс оформления; текущая страница из среды подготовки отчёта не проверена.

21. ГОСТ Р 7.0.100–2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления.

22. ГОСТ Р 7.0.5–2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления.

23. «Деканат» (ММИС). Официальный сайт. URL: https://dekanat.mmis.ru/. Содержимое страницы на дату подготовки 01.11.2026 из рабочей среды недоступно.

24. Информационная система «Контингент». Информация о системе. URL: http://gis-kontingent.ru/information.html. Содержимое страницы на дату подготовки 01.11.2026 из рабочей среды недоступно.

Приложение А
Листинги программного кода

В приложении приведены полные тексты пяти модулей Python-ядра версии 1: доступ к данным, ORM-модели, импорт, прикладные сервисы и командная строка. Исходный код API, двух сайтов и тестов хранится в репозитории [8]. Длинные строки перенесены для печати; для запуска используются файлы репозитория, а не копирование листинга из PDF.

Листинг А.1 — admissions/db.py. Подключение к SQLite и транзакции

```
from contextlib import contextmanager
from pathlib import Path

from sqlalchemy import create_engine, event
from sqlalchemy.orm import DeclarativeBase, sessionmaker


class Base(DeclarativeBase):
    pass


class Database:
    def __init__(self, url: str):
        if not url.startswith("sqlite:///"):
            raise ValueError("Поддерживается SQLite URL
sqlite:///...")
        path = url.removeprefix("sqlite:///")
        if path != ":memory:":
            Path(path).parent.mkdir(parents=True,
exist_ok=True)
        options = {"connect_args": {"check_same_thread":
False, "timeout": 30}}
        if path == ":memory:":
```

Продолжение листинга A.1

```
            from sqlalchemy.pool import StaticPool
            options["poolclass"] = StaticPool
        self.engine = create_engine(url, **options)

        @event.listens_for(self.engine, "connect")
        def configure(connection, _):
            # SQLite lower() по умолчанию не знает кириллицу.
            connection.create_function("lower", 1, lambda
value: value.lower() if isinstance(value, str) else value,
                                       deterministic=True)
            connection.execute("PRAGMA foreign_keys=ON")
            connection.execute("PRAGMA journal_mode=WAL")
            connection.execute("PRAGMA busy_timeout=30000")

        self.sessions = sessionmaker(self.engine,
expire_on_commit=False)

    def create_schema(self):
        from . import models  # noqa: F401
        Base.metadata.create_all(self.engine)

    @contextmanager
    def uow(self):
        """Один запрос/импорт — одна транзакция с откатом при
исключении."""
        with self.sessions.begin() as session:
            yield session
```

Листинг A.2 — admissions/models.py. Модели данных

```
from datetime import datetime, timezone
from enum import Enum

```

Продолжение листинга А.2

```
from sqlalchemy import Boolean, Date, DateTime, ForeignKey,
Index, Integer, JSON, String, UniqueConstraint
from sqlalchemy.orm import Mapped, mapped_column

from .db import Base


def now():
    return datetime.now(timezone.utc)


class Quota(str, Enum):
    SPECIAL = "special"
    SEPARATE = "separate"
    TARGET = "target"
    GENERAL = "general"
    PAID = "paid"


class Applicant(Base):
    __tablename__ = "applicants"
    number: Mapped[str] = mapped_column(String(64),
primary_key=True)
    surname: Mapped[str] = mapped_column(default="",
index=True)
    first_name: Mapped[str] = mapped_column(default="")
    patronymic: Mapped[str] = mapped_column(default="")
    profile: Mapped[dict] = mapped_column(JSON, default=dict)
    updated_at: Mapped[datetime] =
mapped_column(DateTime(timezone=True), default=now)


class Application(Base):
```

Продолжение листинга А.2

```
    __tablename__ = "applications"
    __table_args__ = (
        UniqueConstraint("applicant_number", "year",
"campaign", "direction", "study_form", "quota"),
        Index("ix_ranking", "year", "direction", "quota",
"status", "total_score"),
    )
    id: Mapped[int] = mapped_column(primary_key=True)
    applicant_number: Mapped[str] =
mapped_column(ForeignKey("applicants.number"), index=True)
    year: Mapped[int]
    campaign: Mapped[str]
    direction: Mapped[str]
    study_form: Mapped[str]
    quota: Mapped[str]
    priority: Mapped[int] = mapped_column(default=1)
    status: Mapped[str] =
mapped_column(default="participating")
    total_score: Mapped[int] = mapped_column(default=0)
    achievements: Mapped[int] = mapped_column(default=0)
    exams: Mapped[dict] = mapped_column(JSON, default=dict)
    consent: Mapped[bool] = mapped_column(Boolean,
default=False)
    enrolled_at: Mapped[datetime | None] =
mapped_column(DateTime(timezone=True), nullable=True)
    source: Mapped[dict] = mapped_column(JSON, default=dict)


class SeatPlan(Base):
    __tablename__ = "seat_plans"
    __table_args__ = (UniqueConstraint("year", "campaign",
"direction", "study_form", "quota"),)
    id: Mapped[int] = mapped_column(primary_key=True)
```

Продолжение листинга А.2

```
    year: Mapped[int]
    campaign: Mapped[str]
    direction: Mapped[str]
    study_form: Mapped[str]
    quota: Mapped[str]
    seats: Mapped[int]


class Contract(Base):
    __tablename__ = "contracts"
    id: Mapped[int] = mapped_column(primary_key=True)
    number: Mapped[str] = mapped_column(unique=True)
    application_id: Mapped[int] =
mapped_column(ForeignKey("applications.id"), unique=True)
    location: Mapped[str] =
mapped_column(default="admissions")
    stage: Mapped[str] = mapped_column(default="draft")
    paid: Mapped[bool] = mapped_column(default=False)
    created_at: Mapped[datetime] =
mapped_column(DateTime(timezone=True), default=now)


class ContractEvent(Base):
    __tablename__ = "contract_events"
    id: Mapped[int] = mapped_column(primary_key=True)
    contract_id: Mapped[int] =
mapped_column(ForeignKey("contracts.id"), index=True)
    at: Mapped[datetime] =
mapped_column(DateTime(timezone=True), default=now)
    actor: Mapped[str]
    changes: Mapped[dict] = mapped_column(JSON)

```

Продолжение листинга А.2

```
class Call(Base):
    __tablename__ = "calls"
    id: Mapped[int] = mapped_column(primary_key=True)
    application_id: Mapped[int] =
mapped_column(ForeignKey("applications.id"), index=True)
    result: Mapped[str]
    comment: Mapped[str] = mapped_column(default="")
    actor: Mapped[str]
    at: Mapped[datetime] =
mapped_column(DateTime(timezone=True), default=now)


class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    login: Mapped[str] = mapped_column(unique=True)
    password_hash: Mapped[str]
    active: Mapped[bool] = mapped_column(default=True)


class ActivityEvent(Base):
    __tablename__ = "activity_events"
    id: Mapped[int] = mapped_column(primary_key=True)
    at: Mapped[datetime] =
mapped_column(DateTime(timezone=True), default=now,
index=True)
    actor: Mapped[str]
    action: Mapped[str]
    object: Mapped[str]
    detail: Mapped[str] = mapped_column(default="")
    status: Mapped[str] = mapped_column(default="success")
```

Листинг А.3 — admissions/importer.py. Чтение и импорт Excel-выгрузки

```
"""Потоковое чтение ЕПГУ без потери ведущих нулей и
```

Продолжение листинга А.3

```
альтернативных экзаменов."""
from collections import Counter
from dataclasses import asdict, dataclass, field
from datetime import date, datetime
from pathlib import Path
import re


from openpyxl import load_workbook
from sqlalchemy import select


from .models import Applicant, Application, Quota, now


def normalize(value):
    return re.sub(r"[^a-яа-z0-9]", "", str(value or
"").lower().replace("ё", "е"))


ALIASES = {
    "number": ["Идентификатор", "Номер абитуриента", "ID
абитуриента"],
    "direction": ["Направление", "Направление подготовки"],
    "campaign": ["Приемная кампания", "Уровень образования"],
    "study_form": ["Форма обучения"],
    "category": ["Категория", "Квота", "Основание приема"],
    "total_score": ["Сумма баллов", "Итоговый балл",
"Суммарный балл"],
    "achievements": ["Общие ИД", "Баллы ИД", "Индивидуальные
достижения"],
    "surname": ["Фамилия"], "first_name": ["Имя"],
"patronymic": ["Отчество"],
    "priority": ["Приоритет"], "status": ["Статус"],
"consent": ["Согласие"],
```

Продолжение листинга А.3

```
}
REVERSE = {normalize(alias): name for name, aliases in
ALIASES.items() for alias in aliases}
REQUIRED = {"number", "direction", "campaign", "study_form",
"category", "total_score"}
EXAMS = {
    "Математика", "Прикладная математика", "Информатика",
"Прикладная информатика",
    "Физика", "Химия", "Механика", "Основы электротехники и
электроники",
    "Обществознание", "Основы экономики", "История", "Основы
педагогики",
    "Иностранный язык", "Русский язык", "Междисциплинарный
экзамен", "Специальная дисциплина",
}
STATUS = {normalize(k): v for k, v in {
    "Участвует в конкурсе": "participating", "На
рассмотрении": "pending",
    "Принято": "accepted", "Зачислен": "enrolled",
"Зачислена": "enrolled",
    "Отозвано": "withdrawn", "Отозван": "withdrawn",
"Отклонено": "rejected",
}.items()}


def text(value):
    if value is None:
        return ""
    if isinstance(value, (date, datetime)):
        return value.isoformat()
    if isinstance(value, float) and value.is_integer():
        return str(int(value))
    return str(value).strip()
```

Продолжение листинга А.3

```
def integer(value, name, low=0, high=1000):
    try:
        number = float(str(value).replace(",", "."))
        if not number.is_integer() or not low <= number <=
high:
            raise ValueError
        return int(number)
    except (ValueError, TypeError, OverflowError):
        raise ValueError(f"Некорректное числовое поле:
{name}") from None


def boolean(value):
    n = normalize(value)
    if n in {"да", "истина", "true", "1"}:
        return True
    if n in {"", "нет", "ложь", "false", "0"}:
        return False
    raise ValueError("Неизвестное значение логического поля")


def quota_for(value):
    n = normalize(value)
    aliases = {
        "бюдж": Quota.GENERAL, "бюджет": Quota.GENERAL,
"основнойнабор": Quota.GENERAL,
        "общийконкурс": Quota.GENERAL, "general":
Quota.GENERAL,
        "плат": Quota.PAID, "платное": Quota.PAID,
"платноеобучение": Quota.PAID,
        "договор": Quota.PAID, "paid": Quota.PAID,
```

Продолжение листинга А.3

```
        "особаяквота": Quota.SPECIAL, "special":
Quota.SPECIAL,
        "отдельнаяквота": Quota.SEPARATE, "separate":
Quota.SEPARATE,
        "целевое": Quota.TARGET, "целевоенаправление":
Quota.TARGET,
        "целеваяквота": Quota.TARGET, "target": Quota.TARGET,
    }
    if n not in aliases:
        raise ValueError("Неизвестная категория приема:
требуется явное сопоставление")
    return aliases[n].value


@dataclass
class ImportReport:
    rows: int = 0
    new_applicants: int = 0
    updated_applicants: int = 0
    new_applications: int = 0
    updated_applications: int = 0
    unchanged_applications: int = 0
    errors: list = field(default_factory=list)
    warnings: list = field(default_factory=list)

    def as_dict(self):
        return asdict(self)


def application_key(obj):
    get = obj.get if isinstance(obj, dict) else lambda k:
getattr(obj, k)
    return tuple(get(k) for k in ("applicant_number", "year",
```

Продолжение листинга А.3

```
"campaign", "direction", "study_form", "quota"))
```

```

```

```
def read_epgu(path, year, sheet=None):
    if not 2000 <= year <= 2100:
        raise ValueError("Укажите год приемной кампании
(2000–2100)")
    report = ImportReport()
    workbook = load_workbook(path, read_only=True,
data_only=True)
    parsed = []
    try:
        if sheet is None and len(workbook.sheetnames) != 1:
            raise ValueError("В книге несколько листов:
укажите sheet явно")
        ws = workbook[sheet] if sheet else workbook.active
        rows = ws.iter_rows(values_only=True)
        header = None
        for line, row in enumerate(rows, 1):
            fields = [REVERSE.get(normalize(v)) for v in row]
            if REQUIRED.issubset(set(fields)):
                duplicates = [k for k, v in Counter(x for x in
 fields if x).items() if v > 1]
                if duplicates:
                    raise ValueError("Неоднозначные заголовки:
 " + ", ".join(duplicates))
                header = [text(v) for v in row]
                positions = {v: i for i, v in
enumerate(fields) if v}
                header_line = line
                break
            if line == 20:
                break
```

Продолжение листинга А.3

```
        if header is None:
            raise ValueError("Не найдена строка заголовков.
Нужны: " + ", ".join(sorted(REQUIRED)))
        seen = set()
        for line, row in enumerate(rows, header_line + 1):
            if all(v is None for v in row):
                continue
            report.rows += 1
            get = lambda name, default=None:
row[positions[name]] if name in positions else default
            try:
                number = text(get("number"))
                if not re.fullmatch(r"[0-9A-Za-z_-]{1,64}",
number):
                    raise ValueError("Отсутствует или
некорректен номер абитуриента")
                # Числовая ячейка с форматом 000000000 иначе
потеряла бы ведущие нули.
                if isinstance(get("number"), (int, float)):
                    raise ValueError("Номер абитуриента должен
 быть текстом: иначе ведущие нули неизвестны")
                raw = {k: text(v) for k, v in zip(header, row)
 if k}
                personal = {"Пол", "Дата рождения", "Место
рождения", "Телефон", "Почта", "СНИЛС",
                            "ДУЛ", "Серия", "Номер", "Страна
выдачи", "Дата выдачи",
                            "Тип документа ОУ", "Дата выдачи
ОУ", "Организация, выдавшая документ ОУ",
                            "Наличие ОУ", "Документы доп.",
"Необходимость в общежитии"}
                profile = {k: v for k, v in raw.items() if k
in personal}
```

Продолжение листинга А.3

```
                applicant = {"number": number, "surname":
text(get("surname")),
                             "first_name":
text(get("first_name")), "patronymic":
text(get("patronymic")),
                             "profile": profile}
                status_raw = normalize(get("status",
"Участвует в конкурсе"))
                if status_raw not in STATUS:
                    raise ValueError("Неизвестный статус
заявления")
                app = dict(applicant_number=number, year=year,
 campaign=text(get("campaign")),
                           direction=text(get("direction")),
study_form=text(get("study_form")),
                           quota=quota_for(get("category")),
priority=integer(get("priority", 1), "Приоритет", 1, 100),
                           status=STATUS[status_raw],
total_score=integer(get("total_score"), "Сумма баллов"),

achievements=integer(get("achievements", 0) or 0, "Общие ИД",
0, 100),
                           exams={k: integer(v, k, 0, 100) for
 k, v in raw.items() if k in EXAMS and v},
                           consent=boolean(get("consent",
False)), source=raw)
                if any(not app[k] for k in ("campaign",
"direction", "study_form")):
                    raise ValueError("Не заполнены
направление, форма или кампания")
                key = application_key(app)
                if key in seen:
                    raise ValueError("Повтор заявления в одном
```

Продолжение листинга А.3

```
 файле")
                seen.add(key)
                parsed.append((applicant, app))
            except ValueError as error:
                report.errors.append({"sheet": ws.title,
"row": line, "error": str(error)})
        report.warnings.append("Сумма баллов взята из
итогового столбца; альтернативные экзамены не суммируются.")
        report.warnings.append("Выгрузка обновляет
присутствующие заявления; отсутствующие автоматически не
удаляются.")
        return parsed, report
    finally:
        workbook.close()


def import_epgu(db, path, year, sheet=None):
    parsed, report = read_epgu(path, year, sheet)
    with db.uow() as session:
        applicants = {a.number: a for a in
session.scalars(select(Applicant))}
        applications = {application_key(a): a for a in
session.scalars(select(Application).where(Application.year ==
year))}
        updated = set()
        fresh = set()
        desired = {}
        conflicts = set()
        for values, _ in parsed:
            previous = desired.get(values["number"])
            if previous and previous != values:
                conflicts.add(values["number"])
            desired[values["number"]] = values
```

Продолжение листинга А.3

```
        if conflicts:
            report.warnings.append(f"У {len(conflicts)}
абитуриентов различаются персональные поля в строках;
использована последняя строка файла.")
        for values in desired.values():
            number = values["number"]
            existing = applicants.get(number)
            if existing is None:
                existing = Applicant(**values)
                applicants[number] = existing
                session.add(existing)
                fresh.add(number)
            else:
                # Локальные поля (например, родитель)
сохраняются при ежедневном импорте.
                values = {**values, "profile":
{**existing.profile, **values["profile"]}}
                if any(getattr(existing, k) != v for k, v in
values.items()):
                    for k, v in values.items():
                        setattr(existing, k, v)
                    existing.updated_at = now()
                    updated.add(number)
        session.flush()
        for _, values in parsed:
            key = application_key(values)
            existing = applications.get(key)
            if existing is None:
                session.add(Application(**values))
                report.new_applications += 1
            else:
                # Приказ о зачислении — локальное действие,
его не отменяет старый Excel.
```

Продолжение листинга А.3

```
                if existing.enrolled_at:
                    values = {**values, "status": "enrolled",
"source": {
                        **values["source"],
"enrollment_order": existing.source.get("enrollment_order")}}
                if any(getattr(existing, k) != v for k, v in
values.items()):
                    for k, v in values.items():
                        setattr(existing, k, v)
                    report.updated_applications += 1
                else:
                    report.unchanged_applications += 1
        report.new_applicants = len(fresh)
        report.updated_applicants = len(updated - fresh)
    return report
```

Листинг А.4 — admissions/services.py. Рейтинги, договоры, оплата, звонки и экспорт

```
from datetime import date, datetime, timedelta, timezone
import io
import json
import sqlite3
from pathlib import Path
from xml.etree.ElementTree import Element, SubElement,
tostring
from zoneinfo import ZoneInfo

from docx import Document
from openpyxl import Workbook, load_workbook
from sqlalchemy import func, select

from .importer import integer, normalize, text
from .models import Applicant, Application, Call, Contract,
```

Продолжение листинга А.4

```
ContractEvent, Quota, SeatPlan, now



ACTIVE = ("participating", "accepted", "enrolled")
LOCATIONS = {"admissions", "applicant", "accounting",
"archive"}
TRANSITIONS = {
    "draft": {"issued", "cancelled"}, "issued": {"signed",
"cancelled"},
    "signed": {"registered", "cancelled"}, "registered":
{"cancelled"}, "cancelled": set(),
}



def require(session, model, key):
    obj = session.get(model, key)
    if obj is None:
        raise LookupError("Запись не найдена")
    return obj



def group_query(year, direction=None, campaign=None,
study_form=None, quota=None):
    q = select(Application).where(Application.year == year)
    for name, value in {"direction": direction, "campaign":
campaign, "study_form": study_form, "quota": quota}.items():
        if value is not None:
            q = q.where(getattr(Application, name) == value)
    return q



def ranking(session, year, direction, campaign, study_form,
quota):
```

Продолжение листинга А.4

```
    """Ничьи разрешаются номером для стабильности; это не
правило приказа о зачислении."""
    q = group_query(year, direction, campaign, study_form,
quota).where(Application.status.in_(ACTIVE))
    rows =
session.scalars(q.order_by(Application.total_score.desc(),
Application.applicant_number)).all()
    result = []
    previous = None
    place = 0
    for index, app in enumerate(rows, 1):
        if app.total_score != previous:
            place = index
        previous = app.total_score
        result.append({"applicant_number":
app.applicant_number, "score": app.total_score,
                       "quota": app.quota, "direction":
app.direction, "rank": place})
    return result


def all_rankings(session, year, direction, campaign,
study_form):
    return {q.value: ranking(session, year, direction,
campaign, study_form, q.value) for q in Quota}


def forecast(session, application):
    a = application
    rows = ranking(session, a.year, a.direction, a.campaign,
a.study_form, a.quota)
    own = next((r for r in rows if r["applicant_number"] ==
a.applicant_number), None)
```

Продолжение листинга А.4

```
    seats = session.scalar(select(SeatPlan).where(
        SeatPlan.year == a.year, SeatPlan.campaign ==
a.campaign,
        SeatPlan.direction == a.direction, SeatPlan.study_form
 == a.study_form, SeatPlan.quota == a.quota))
    result = {"method": "rules_v1", "probability": None,
"rank": own["rank"] if own else None,
              "seats": seats.seats if seats else None,
"assessment": "unknown",
              "factors": ["Расчёт по текущему рейтингу, без
обученной ИИ-модели и гарантии зачисления."]}
    if own is None:
        result["factors"].append("Заявление не участвует в
действующем конкурсе.")
    elif seats is None:
        result["factors"].append("План мест не задан.")
    else:
        better = sum(r["score"] > a.total_score for r in rows)
        tied = sum(r["score"] == a.total_score for r in rows)
        if better >= seats.seats:
            result["assessment"] = "outside_seats"
        elif better + tied <= seats.seats:
            result["assessment"] = "within_seats"
        else:
            result["assessment"] = "tie_at_boundary"
        result["factors"].append("Не учитывает будущие
заявления, отзыв согласий и правила разрешения равных
баллов.")
    return result


def create_contract(session, application_id, number, actor):
    app = require(session, Application, application_id)
```

Продолжение листинга А.4

```
    if app.quota != Quota.PAID.value:
        raise ValueError("Договор доступен только для платного
 заявления")
    if app.status in {"withdrawn", "rejected"}:
        raise ValueError("Заявление отозвано или отклонено")
    if session.scalar(select(Contract).where((Contract.number
== number) | (Contract.application_id == application_id))):
        raise ValueError("Договор с таким номером или для
этого заявления уже существует")
    contract = Contract(number=number,
application_id=application_id)
    session.add(contract)
    session.flush()
    session.add(ContractEvent(contract_id=contract.id,
actor=actor,
                              changes={"created": True,
"location": "admissions", "stage": "draft"}))
    return contract


def update_contract(session, contract, actor, *,
location=None, stage=None, paid=None):
    changes = {}
    if location is not None and location not in LOCATIONS:
        raise ValueError("Неизвестное местоположение
договора")
    if stage is not None and stage != contract.stage and stage
 not in TRANSITIONS[contract.stage]:
        raise ValueError("Недопустимый переход этапа
договора")
    if paid is True and (stage or contract.stage) ==
"cancelled":
        raise ValueError("Нельзя подтвердить оплату
```

Продолжение листинга А.4

```
отменённого договора")
    for field, value in {"location": location, "stage": stage,
 "paid": paid}.items():
        if value is not None and value != getattr(contract,
field):
            changes[field] = {"from": getattr(contract,
field), "to": value}
            setattr(contract, field, value)
    if changes:
        session.add(ContractEvent(contract_id=contract.id,
actor=actor, changes=changes))
    return contract


def reconcile_payments(session, file, actor):
    """Файл подтверждённых полных оплат: номер договора +
Оплачен. Отсутствие не отменяет оплату."""
    workbook = load_workbook(file, read_only=True,
data_only=True)
    from .importer import boolean
    report = {"confirmed": 0, "unchanged": 0, "errors": []}
    try:
        rows = workbook.active.iter_rows(values_only=True)
        header = [normalize(v) for v in next(rows, ())]
        if "номердоговора" not in header or "оплачен" not in
header:
            raise ValueError("В файле оплаты нужны столбцы
«Номер договора», «Оплачен»")
        n, p = header.index("номердоговора"),
header.index("оплачен")
        contracts = {c.number: c for c in
session.scalars(select(Contract))}
        for line, row in enumerate(rows, 2):
```

Продолжение листинга А.4

```
            if not any(v is not None for v in row):
                continue
            try:
                contract = contracts.get(text(row[n]))
                if contract is None:
                    raise ValueError("Договор не найден")
                if not boolean(row[p]) or contract.paid:
                    report["unchanged"] += 1
                    continue
                update_contract(session, contract, actor,
paid=True)
                report["confirmed"] += 1
            except ValueError as error:
                report["errors"].append({"row": line, "error":
 str(error)})
        return report
    finally:
        workbook.close()


def call_queue(session, year, change_deadline,
payment_deadline, today=None):
    today = today or
datetime.now(ZoneInfo("Europe/Moscow")).date()
    reasons = []
    if change_deadline - timedelta(days=7) <= today <=
change_deadline:
        reasons.append("direction_deadline")
    if payment_deadline - timedelta(days=7) <= today <=
payment_deadline:
        reasons.append("payment_deadline")
    if not reasons:
        return []
```

Продолжение листинга А.4

```
    apps = session.scalars(group_query(year,
quota=Quota.PAID.value).where(Application.status.in_(ACTIVE)))
    contracts = {c.application_id: c for c in session.scalars(
select(Contract).join(Application).where(Application.year ==
year))}
    last_calls = {}
    for call in session.scalars(select(Call).join(Application)
.where(Application.year == year).order_by(Call.at, Call.id)):
        last_calls[call.application_id] = call
    result = []
    for app in apps:
        contract = contracts.get(app.id)
        if contract and (contract.paid or contract.stage ==
"cancelled"):
            continue
        last = last_calls.get(app.id)
        if last and last.result == "declined":
            continue
        person = require(session, Applicant,
app.applicant_number)
        score = (40 if app.consent else 0) + (25 if
app.priority == 1 else 0)
        score += 30 if contract and contract.stage in
{"signed", "registered"} else 0
        result.append({"application_id": app.id,
"applicant_number": app.applicant_number,
                       "phone": person.profile.get("Телефон",
""), "direction": app.direction,
                       "priority_score": score, "method":
"rules_v1", "reasons": reasons,
                       "last_result": last.result if last else
 None})
    return sorted(result, key=lambda r: (-r["priority_score"],
```

Продолжение листинга А.4

```
 r["applicant_number"], r["application_id"]))


def statistics(session, year):
    groups = session.execute(select(Application.direction,
Application.campaign, Application.study_form,
                                    Application.quota,
func.count(Application.id)).where(Application.year == year)
                               .group_by(Application.direction,
Application.campaign, Application.study_form,
Application.quota)).all()
    plans = {(p.direction, p.campaign, p.study_form, p.quota):
 p.seats
             for p in
session.scalars(select(SeatPlan).where(SeatPlan.year ==
year))}
    statuses = dict(session.execute(select(Application.status,
 func.count()).where(Application.year == year)

.group_by(Application.status)).all())
    active_counts = {(d, c, f, q): count for d, c, f, q, count
 in session.execute(
        select(Application.direction, Application.campaign,
Application.study_form, Application.quota, func.count())
        .where(Application.year == year,
Application.status.in_(ACTIVE))
        .group_by(Application.direction, Application.campaign,
 Application.study_form, Application.quota))}
    return {"year": year, "applications":
sum(statuses.values()), "statuses": statuses,
            "applicants": session.scalar(select(func.count(fun
c.distinct(Application.applicant_number))).where(Application.y
ear == year)),
```

Продолжение листинга А.4

```
            "groups": [{"direction": d, "campaign": c,
"study_form": f, "quota": q, "applications": count,
                        "active": active_counts.get((d, c, f,
q), 0), "seats": plans.get((d, c, f, q)),
                        "overcrowded": active_counts.get((d,
c, f, q), 0) > plans[(d, c, f, q)]
                        if (d, c, f, q) in plans else None}
for d, c, f, q, count in groups]}


def xlsx_bytes(rows):
    workbook = Workbook()
    ws = workbook.active
    ws.title = "Отчёт"
    def safe(value):
        # Запрещаем формулы из внешнего Excel в экспортируемых
 строках.
        return "'" + value if isinstance(value, str) and
value.startswith(("=", "+", "-", "@")) else value
    if rows:
        columns = list(rows[0])
        ws.append(columns)
        for row in rows:
            ws.append([safe(row.get(k)) for k in columns])
    out = io.BytesIO()
    workbook.save(out)
    return out.getvalue()


def pdf_bytes(rows):
    from reportlab.lib import colors
    from reportlab.lib.pagesizes import A4, landscape
    from reportlab.pdfbase import pdfmetrics
```

Продолжение листинга А.4

```
    from reportlab.pdfbase.ttfonts import TTFont
    from reportlab.platypus import SimpleDocTemplate, Table,
TableStyle
    font =
Path("/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf")
    if not font.exists():
        raise ValueError("Для PDF нужен шрифт DejaVuSans.ttf
(fonts-dejavu-core)")
    pdfmetrics.registerFont(TTFont("DejaVu", str(font)))
    out = io.BytesIO()
    columns = ["rank", "applicant_number", "score", "quota"]
    data = [["Место", "Номер абитуриента", "Баллы", "Квота"]]
    data += [[str(row.get(k, "")) for k in columns] for row in
 rows]
    table = Table(data, repeatRows=1)
    table.setStyle(TableStyle([("FONTNAME", (0, 0), (-1, -1),
"DejaVu"),
                               ("BACKGROUND", (0, 0), (-1, 0),
 colors.lightgrey),
                               ("GRID", (0, 0), (-1, -1), .3,
colors.grey)]))
    SimpleDocTemplate(out, pagesize=A4).build([table])
    return out.getvalue()


def render_contract(session, contract, template):
    """Маркеры {{name}} могут быть разбиты Word на несколько
runs; оформление абзаца сохраняется частично."""
    app = require(session, Application,
contract.application_id)
    person = require(session, Applicant, app.applicant_number)
    values = {"contract_number": contract.number,
"applicant_number": person.number,
```

Продолжение листинга А.4

```
                "full_name": " ".join(filter(None,
[person.surname, person.first_name, person.patronymic])),
                "direction": app.direction, "study_form":
app.study_form, "year": str(app.year),
                "snils": person.profile.get("СНИЛС", ""),
"passport_series": person.profile.get("Серия", ""),
                "passport_number": person.profile.get("Номер",
""), "date": now().date().isoformat()}
    document = Document(template)
    def replace_container(container):
        for paragraph in container.paragraphs:
            original = paragraph.text
            replaced = original
            for name, value in values.items():
                replaced = replaced.replace("{{" + name +
"}}", value)
            if replaced != original:
                if paragraph.runs:
                    paragraph.runs[0].text = replaced
                    for run in paragraph.runs[1:]:
                        run.text = ""
                else:
                    paragraph.add_run(replaced)
            if "{{" in replaced:
                raise ValueError("В шаблоне есть
неподдерживаемый маркер")
        for table in container.tables:
            for row in table.rows:
                for cell in row.cells:
                    replace_container(cell)
    replace_container(document)
    for section in document.sections:
        replace_container(section.header)
```

Продолжение листинга А.4

```
        replace_container(section.footer)
    out = io.BytesIO()
    document.save(out)
    return out.getvalue()


def export_1c(session, year, format="json"):
    records = []
    contracts = {c.application_id: c for c in
session.scalars(select(Contract))}
    for app, person in session.execute(select(Application,
Applicant).join(Applicant)
                                       .where(Application.year
== year, Application.status == "enrolled",

Application.enrolled_at.is_not(None))):
        contract = contracts.get(app.id)
        records.append({"applicant_number": person.number,
                        "full_name": " ".join(filter(None,
[person.surname, person.first_name, person.patronymic])),
                        "passport_series":
person.profile.get("Серия", ""),
                        "passport_number":
person.profile.get("Номер", ""), "snils":
person.profile.get("СНИЛС", ""),
                        "direction": app.direction,
"study_form": app.study_form,
                        "enrolled_at":
app.enrolled_at.date().isoformat(), "contract_number":
contract.number if contract else None})
    if format == "json":
        return json.dumps({"schema_version": 1, "students":
records}, ensure_ascii=False).encode()
```

Продолжение листинга А.4

```
    if format != "xml":
        raise ValueError("Формат должен быть json или xml")
    root = Element("students", schema_version="1")
    for record in records:
        child = SubElement(root, "student")
        for key, value in record.items():
            SubElement(child, key).text = value or ""
    return tostring(root, encoding="utf-8",
xml_declaration=True)


def backup_database(db, destination):
    path = Path(destination)
    source_path = db.engine.url.database
    if source_path == ":memory:" or path.resolve() ==
Path(source_path).resolve():
        raise ValueError("Нужны файловая БД и отдельный путь
резервной копии")
    path.parent.mkdir(parents=True, exist_ok=True)
    if path.exists():
        raise ValueError("Резервная копия по этому пути уже
существует")
    # sqlite backup API корректно учитывает WAL и параллельные
 транзакции.
    connection = db.engine.raw_connection()
    try:
        with sqlite3.connect(path) as target:
            connection.driver_connection.backup(target)
            if target.execute("PRAGMA
integrity_check").fetchone()[0] != "ok":
                raise ValueError("Проверка резервной копии не
пройдена")
    finally:
```

Продолжение листинга А.4

```
        connection.close()
    path.chmod(0o600)
    return str(path)


FAQ = {
    "номер": "Номер абитуриента указан в заявлении в личном
кабинете Госуслуг (ЕПГУ). Введите его полностью, включая нули
в начале. ФИО, паспорт и СНИЛС для поиска на портале не
нужны.",
    "епгу": "Номер абитуриента находится в заявлении на
Госуслугах. Сохраните все ведущие нули при поиске. Не
отправляйте паспортные данные и СНИЛС в чат.",
    "шанс": "Оценка сравнивает текущее место в рейтинге с
планом мест по выбранной категории. Она не гарантирует
зачисления и не является обученной ИИ-моделью. Без плана мест
оценка недоступна.",
    "прогноз": "Оценка основана на текущем рейтинге и плане
мест. Будущие заявления, отзыв согласий и официальное
разрешение ничьих могут изменить результат. Решение о
зачислении оформляется приказом.",
    "документ": "Подготовьте паспорт, документ об образовании
и СНИЛС. Перечень льготных документов уточните в комиссии.",
    "договор": "Для платного обучения договор оформляет
приёмная комиссия; статус и оплату уточняйте у секретаря.",
    "рейтинг": "Рейтинг доступен по номеру абитуриента,
направлению, форме обучения и категории конкурса.",
    "срок": "Даты устанавливаются ежегодными правилами приёма
БГТУ. Уточните актуальные сроки в комиссии.",
    "соглас": "Порядок подачи согласия уточняйте в действующих
 правилах приёма БГТУ и в личном кабинете Госуслуг. Открытый
портал не принимает согласия и не заменяет официальное
заявление.",
```

Продолжение листинга А.4

```
    "зачисл": "Зачисление подтверждается приказом
университета. Место в конкурсном списке и оценка положения не
являются приказом о зачислении.",
}


def chatbot(question):
    question = question.lower()
    if "договор" in question or "оплат" in question:
        return FAQ["договор"]
    if "когда" in question:
        return FAQ["срок"]
    for word, answer in FAQ.items():
        if word in question:
            return answer
    return "Уточните вопрос о документах, договоре, рейтинге
или сроках. За индивидуальной консультацией обратитесь в
комиссию."
```

Листинг А.5 — admissions/__main__.py. Команды запуска и обслуживания

```
import argparse
import getpass
import json
import os
from pathlib import Path

from .db import Database
from .importer import import_epgu
from .security import add_user
from .services import backup_database


```

Продолжение листинга А.5

```
def main():
    parser = argparse.ArgumentParser(description="АИС приёмной
 комиссии БГТУ")
    parser.add_argument("--database",
default=os.environ.get("DATABASE_URL",
"sqlite:///data/admissions.sqlite3"))
    commands = parser.add_subparsers(dest="command",
required=True)
    commands.add_parser("init-db")
    user = commands.add_parser("create-user")
    user.add_argument("login")
    importer = commands.add_parser("import-epgu")
    importer.add_argument("file", type=Path)
    importer.add_argument("--year", type=int, required=True)
    importer.add_argument("--sheet")
    backup = commands.add_parser("backup")
    backup.add_argument("destination", type=Path)
    args = parser.parse_args()
    db = Database(args.database)
    db.create_schema()
    try:
        if args.command == "create-user":
            password = getpass.getpass("Пароль: ")
            if password != getpass.getpass("Повтор пароля: "):
                parser.error("Пароли не совпадают")
            with db.uow() as session:
                add_user(session, args.login, password)
            print("Пользователь создан")
        elif args.command == "import-epgu":
            report = import_epgu(db, args.file, args.year,
args.sheet)
            print(json.dumps(report.as_dict(),
ensure_ascii=False, indent=2))
```

Продолжение листинга А.5

```
            if report.errors:
                return 2
        elif args.command == "backup":
            print(backup_database(db, args.destination))
        else:
            print("Схема создана")
    except (ValueError, OSError) as error:
        parser.error(str(error))
    finally:
        db.engine.dispose()
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Приложение Б
SQL-скрипты создания базы данных

Листинг Б.1 содержит структуру восьми таблиц работающей SQLite-базы и индексы, сформированные из SQLAlchemy metadata. Скрипт не содержит персональных записей или паролей. Штатная установка выполняется командой init-db. Пример запроса в листинге Б.2 показывает выборку конкурсной группы без персональных полей.

Листинг Б.1 — Создание таблиц и индексов SQLite

```
-- Реальная схема версии 1. Сгенерирована из SQLAlchemy metadata.

PRAGMA foreign_keys = ON;

CREATE TABLE activity_events (
        id INTEGER NOT NULL,
        at DATETIME NOT NULL,
        actor VARCHAR NOT NULL,
        action VARCHAR NOT NULL,
        object VARCHAR NOT NULL,
        detail VARCHAR NOT NULL,
        status VARCHAR NOT NULL,
        PRIMARY KEY (id)
);

CREATE INDEX ix_activity_events_at ON activity_events (at);

CREATE TABLE applicants (
        number VARCHAR(64) NOT NULL,
        surname VARCHAR NOT NULL,
        first_name VARCHAR NOT NULL,
        patronymic VARCHAR NOT NULL,
        profile JSON NOT NULL,
```

Продолжение листинга Б.1

```
        updated_at DATETIME NOT NULL,
        PRIMARY KEY (number)
);

CREATE INDEX ix_applicants_surname ON applicants (surname);

CREATE TABLE seat_plans (
        id INTEGER NOT NULL,
        year INTEGER NOT NULL,
        campaign VARCHAR NOT NULL,
        direction VARCHAR NOT NULL,
        study_form VARCHAR NOT NULL,
        quota VARCHAR NOT NULL,
        seats INTEGER NOT NULL,
        PRIMARY KEY (id),
        UNIQUE (year, campaign, direction, study_form, quota)
);

CREATE TABLE users (
        id INTEGER NOT NULL,
        login VARCHAR NOT NULL,
        password_hash VARCHAR NOT NULL,
        active BOOLEAN NOT NULL,
        PRIMARY KEY (id),
        UNIQUE (login)
);

CREATE TABLE applications (
        id INTEGER NOT NULL,
        applicant_number VARCHAR(64) NOT NULL,
        year INTEGER NOT NULL,
        campaign VARCHAR NOT NULL,
        direction VARCHAR NOT NULL,
```

Продолжение листинга Б.1

```
        study_form VARCHAR NOT NULL,
        quota VARCHAR NOT NULL,
        priority INTEGER NOT NULL,
        status VARCHAR NOT NULL,
        total_score INTEGER NOT NULL,
        achievements INTEGER NOT NULL,
        exams JSON NOT NULL,
        consent BOOLEAN NOT NULL,
        enrolled_at DATETIME,
        source JSON NOT NULL,
        PRIMARY KEY (id),
        UNIQUE (applicant_number, year, campaign, direction,
study_form, quota),
        FOREIGN KEY(applicant_number) REFERENCES applicants
(number)
);

CREATE INDEX ix_ranking ON applications (year, direction,
quota, status, total_score);

CREATE INDEX ix_applications_applicant_number ON applications
(applicant_number);

CREATE TABLE calls (
        id INTEGER NOT NULL,
        application_id INTEGER NOT NULL,
        result VARCHAR NOT NULL,
        comment VARCHAR NOT NULL,
        actor VARCHAR NOT NULL,
        at DATETIME NOT NULL,
        PRIMARY KEY (id),
        FOREIGN KEY(application_id) REFERENCES applications
(id)
```

Продолжение листинга Б.1

```
);

CREATE INDEX ix_calls_application_id ON calls
(application_id);

CREATE TABLE contracts (
        id INTEGER NOT NULL,
        number VARCHAR NOT NULL,
        application_id INTEGER NOT NULL,
        location VARCHAR NOT NULL,
        stage VARCHAR NOT NULL,
        paid BOOLEAN NOT NULL,
        created_at DATETIME NOT NULL,
        PRIMARY KEY (id),
        UNIQUE (number),
        UNIQUE (application_id),
        FOREIGN KEY(application_id) REFERENCES applications
(id)
);

CREATE TABLE contract_events (
        id INTEGER NOT NULL,
        contract_id INTEGER NOT NULL,
        at DATETIME NOT NULL,
        actor VARCHAR NOT NULL,
        changes JSON NOT NULL,
        PRIMARY KEY (id),
        FOREIGN KEY(contract_id) REFERENCES contracts (id)
);

CREATE INDEX ix_contract_events_contract_id ON contract_events
 (contract_id);
```

Листинг Б.2 — Обезличенная выборка одной конкурсной группы

```
SELECT applicant_number, total_score, priority, consent
FROM applications
WHERE year = :year AND campaign = :campaign
  AND direction = :direction AND study_form = :study_form
  AND quota = :quota AND status = 'participating'
ORDER BY total_score DESC, applicant_number ASC;
```

Выборка в листинге Б.2 ограничена параметрами конкурса и разрешёнными полями. Год, форма обучения и квота задаются явно. Первичные и внешние ключи поддерживают связи записей, а условия WHERE определяют состав конкретной выдачи. При расширении запроса нужно проверять, не попадают ли в публичный ответ персональные сведения. Структура исходников и результаты тестов в следующем приложении показывают, где реализованы эти правила и чем подтверждена их работа.

Приложение В
Структура проекта и результаты проверки

На рисунках В.1–В.3 представлены фактическая структура репозитория, результат локального прогона автоматических тестов и история коммитов. Снимки терминала сформированы по реальному выводу команд. Успешный локальный прогон не является свидетельством настроенного CI или промышленной приёмки.

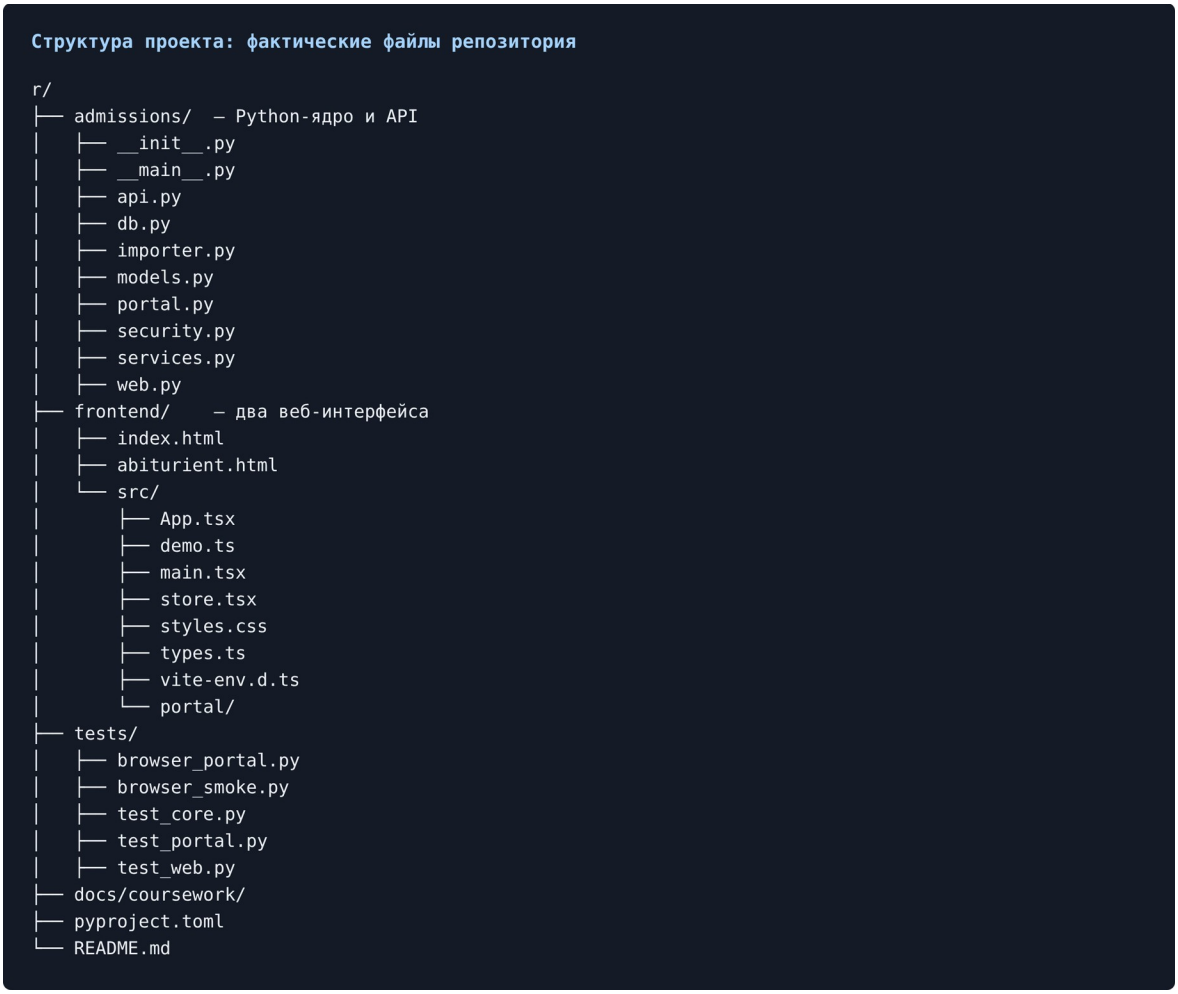


Рисунок В.1 — Структура каталогов и исходных файлов

Источник: локальная проверка репозитория zarunast/r при подготовке этой редакции.

```
python -m pytest -q

.......................                                                  [100%]
============================== warnings summary ===============================
../../opt/codex/runtimes/codex-primary-runtime/dependencies/python/lib/python3.12/site-
packages/fastapi/testclient.py:1
  /opt/codex/runtimes/codex-primary-runtime/dependencies/python/lib/python3.12/site-
packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with
`starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

.venv/lib/python3.12/site-packages/passlib/utils/__init__.py:854
  /workspace/r/.venv/lib/python3.12/site-packages/passlib/utils/__init__.py:854: DeprecationWarning:
'crypt' is deprecated and slated for removal in Python 3.13
    from crypt import crypt as _crypt

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
26 passed, 2 warnings in 8.60s
```

Рисунок В.2 — Результат локального прогона: 26 тестов пройдены

Источник: локальная проверка репозитория zarunast/r при подготовке этой редакции.

```
git log -6 --format="%h %s"

de4f6af  Revise coursework report with frames, concise introduction and complete illustrations
26e9717  Add coursework report, PDF and diagram materials
853721f  Add public applicant portal with anonymous rankings and admission guidance
224d1e0  Add internal admissions website matching dashboard reference
6ff00a7  Add admissions backend core, tests, documentation and source archive
```

Рисунок В.3 — История изменений репозитория Git

Источник: локальная проверка репозитория zarunast/r при подготовке этой редакции.

Приложение Г
План разработки по спринтам

Ниже приведена учебная модель четырёх двухнедельных спринтов MVP. Задачи сгруппированы по инкрементам, а не по фактическим датам работы команды. Задачи оцениваются по относительной сложности в Story Points; затраты в часах не приводятся.

Таблица Г.1 — Первый спринт

Задача	Story Points	Статус	Исполнитель
US-1: Аутентификация и настройка инфраструктуры	5	В плане	Backend
US-2: Загрузка Excel-выгрузки из ЕПГУ	8	В плане	Backend
US-3: Реестр абитуриентов (CRUD, поиск, карточка)	8	В плане	Fullstack
Итого	21		

Таблица Г.2 — Второй спринт

Задача	Story Points	Статус	Исполнитель
US-4, US-5, US-6: Расчёт рейтинга и 5 конкурсных списков	13	В плане	Backend

Продолжение таблицы Г.2

Задача	Story Points	Статус	Исполнитель
US-7, US-8: Создание договора (python-docx) и статусы («ГДЕ?», «ЭТАП»)	13	В плане	Fullstack
Итого	26		

Таблица Г.3 — Третий спринт

Задача	Story Points	Статус	Исполнитель
US-12, US-13: Сайт 2 (поиск, личный кабинет, обезличенный рейтинг)	8	В плане	Frontend
US-9, US-10: Подтверждение оплаты и списки для обзвона	8	В плане	Frontend
US-11: ИИ-прогноз вероятности заключения договора (Яндекс GPT / DeepSeek)	13	В плане	Backend
Итого	29		

Таблица Г.4 — Четвёртый спринт

Задача	Story Points	Статус	Исполнитель
US-14: ИИ-прогноз шансов поступления (Яндекс GPT / DeepSeek)	13	В плане	Backend
US-15: Скриптовый чат-бот (FAQ)	5	В плане	Frontend
US-16, US-17, US-18, US-19: Отчётность, архивация, выгрузка в 1С	13	В плане	Backend
US-20: Продуктовые события и аналитика	8	В плане	Backend + Fullstack
Тестирование (pytest, интеграционное) и документация	8	В плане	Тестировщик
Итого	47		

Завершение спринта подтверждается проверкой сквозного сценария: импорт, реестр, рейтинг, договор и отчётность. Результат фиксируется на синтетических данных; будущие функции и неподтверждённые нормативы рассматриваются отдельно от реализованных возможностей прототипа.